

Introduction to Matplotlib

- Matplotlib is a fundamental library in Python for creating static, interactive, and animated visualizations.
- It provides a flexible and powerful way to generate various types of plots, making it an essential tool for data analysis and presentation.

Key Highlights:

- **Versatile Plotting:** Capable of generating line plots, scatter plots, bar charts, histograms, and more.
- **Customizable:** Offers extensive control over plot aesthetics, including labels, titles, legends, colors, and styles.
- **Integration:** Works seamlessly with other scientific Python libraries like NumPy for numerical operations and Pandas for data manipulation.
- **matplotlib.pyplot** : A module that provides a MATLAB-like interface for easy plotting.

Basic Workflow (using `pyplot`):

1. Import `pyplot` .
2. Prepare Data using Python lists or NumPy arrays.
3. Create Plot(s) using `plt` functions (e.g., `plt.plot()` , `plt.scatter()` , `plt.bar()`).
4. Customize plots with labels, titles, and legends using `plt` functions.
5. Display the plot using `plt.show()` or save it using `plt.savefig()` .

- In essence, Matplotlib empowers you to tell compelling visual stories with your data in Python.

In [2]:

```
import matplotlib.pyplot as plt
```

Creating Basic Line Plots

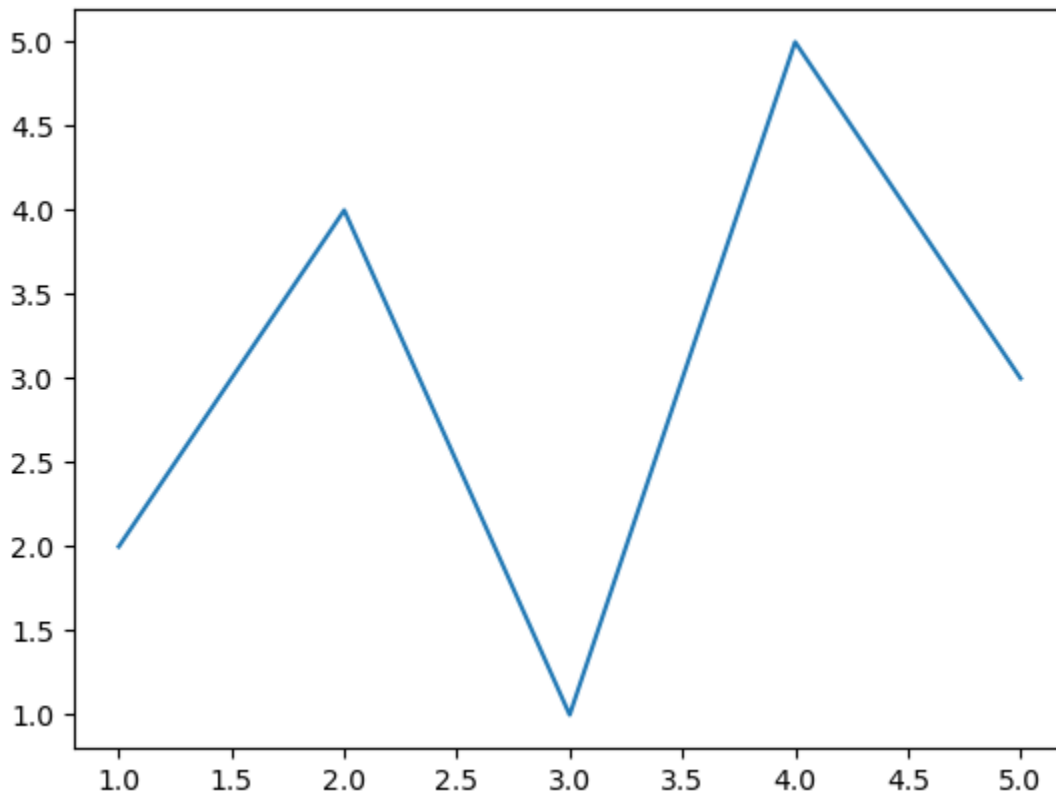
- The `plt.plot()` function is your workhorse for creating line plots.
- It typically takes two main arguments: the data for the **x-axis** and the data for the **y-axis**.
- You can provide Python lists or NumPy arrays as input.
- By default, `plt.plot()` will connect the data points with a solid line.

In [3]:

```
# Sample data
x = [1, 2, 3, 4, 5]
y = [2, 4, 1, 5, 3]

# Create a basic line plot
plt.plot(x, y)

# Display the plot
plt.show()
```



Customizing Basic Plot Elements

Titles: Use `plt.title('Your Plot Title')` to add a title to your plot, enhancing its context.

X-axis Label: Use `plt.xlabel('X-axis Label')` to describe the data represented on the horizontal axis.

Y-axis Label: Use `plt.ylabel('Y-axis Label')` to describe the data on the vertical axis.

Legends: For plots with multiple lines, use `plt.legend()` to display a legend. You'll need to provide a `label` argument within the `plt.plot()` function for each line to make the legend work.

In [4]:

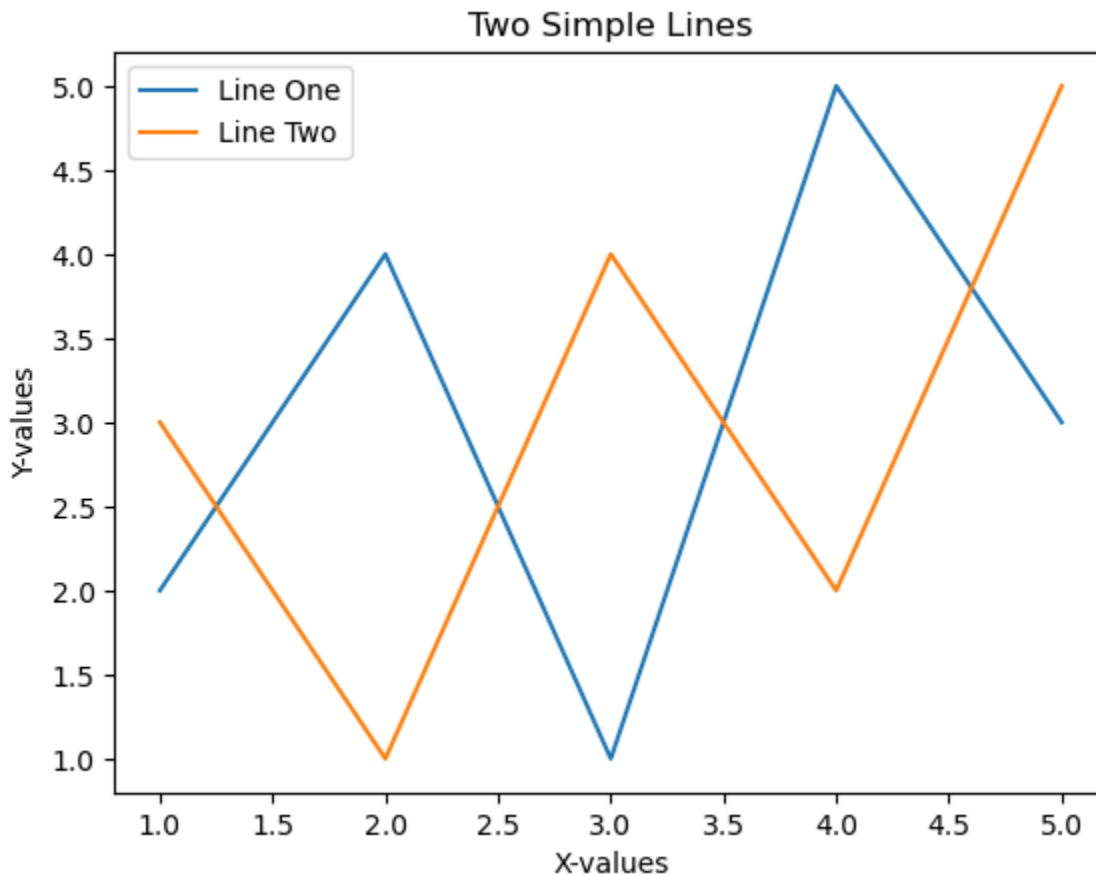
```
# Sample data for two lines
x = [1, 2, 3, 4, 5]
y1 = [2, 4, 1, 5, 3]
y2 = [3, 1, 4, 2, 5]

# Create two line plots with labels
plt.plot(x, y1, label='Line One')
plt.plot(x, y2, label='Line Two')

# Add title and labels
plt.title('Two Simple Lines')
plt.xlabel('X-values')
plt.ylabel('Y-values')

# Display the legend
plt.legend()
```

```
# Show the plot
plt.show()
```



Displaying Plots

- The `plt.show()` function is crucial for displaying the generated plot.
- It opens a window (or displays inline in environments like Jupyter notebooks) containing your visualization.
- Any code for further modifications to the plot *after* `plt.show()` might not be displayed in the same figure.

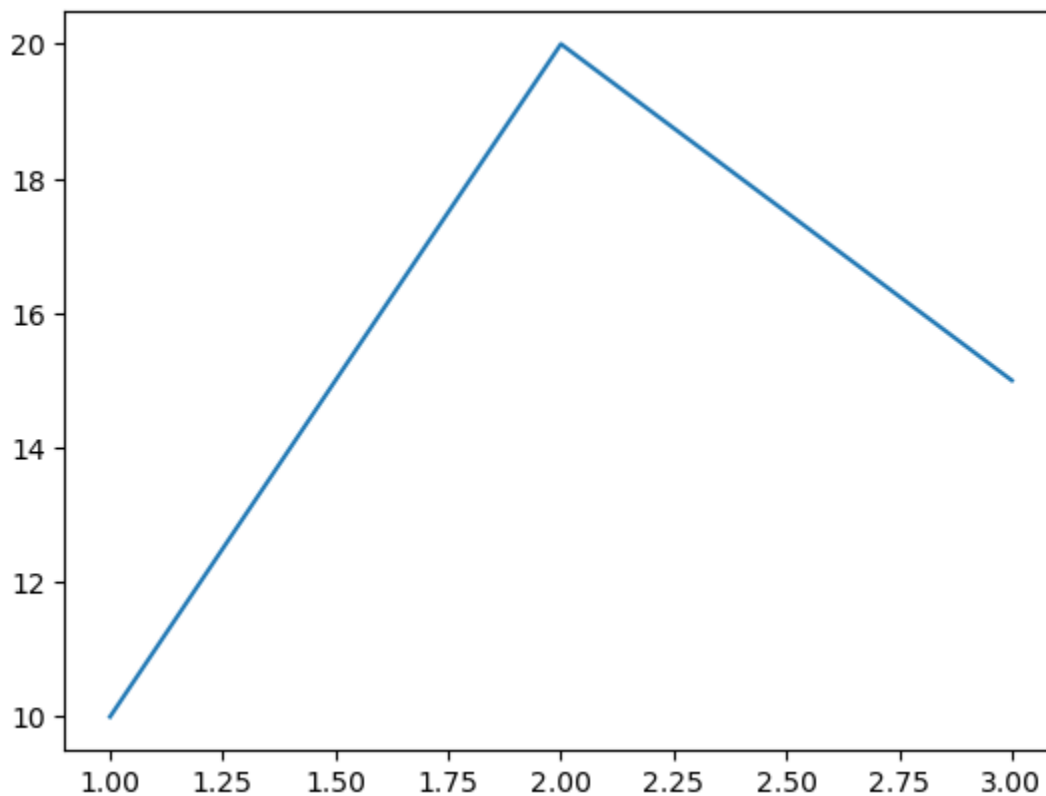
In [6]:

```
# Sample data
x = [1, 2, 3]
y = [10, 20, 15]

# Create a plot
plt.plot(x, y)

# Display the plot
plt.show()

# Any subsequent plotting commands will typically create a new figure
```



Saving Plots

- The `plt.savefig('filename.format')` function allows you to save your plot to a file.
- Replace `'filename'` with your desired file name and `.format` with the desired file format (e.g., `.png`, `.pdf`, `.jpg`, `.svg`).
- You can also specify additional arguments like `dpi` (dots per inch) to control the image resolution.

In [7]:

```
# Sample data
x = [1, 2, 3, 4]
y = [1, 4, 2, 3]

# Create a plot
plt.plot(x, y)
plt.title('My Saved Plot')
plt.xlabel('X')
plt.ylabel('Y')

# Save the plot as a PNG file
plt.savefig('my_line_plot.png')

# Save the plot as a PDF with higher DPI
plt.savefig('my_line_plot.pdf', dpi=300)

# plt.show() is not needed if you only want to save the figure
```

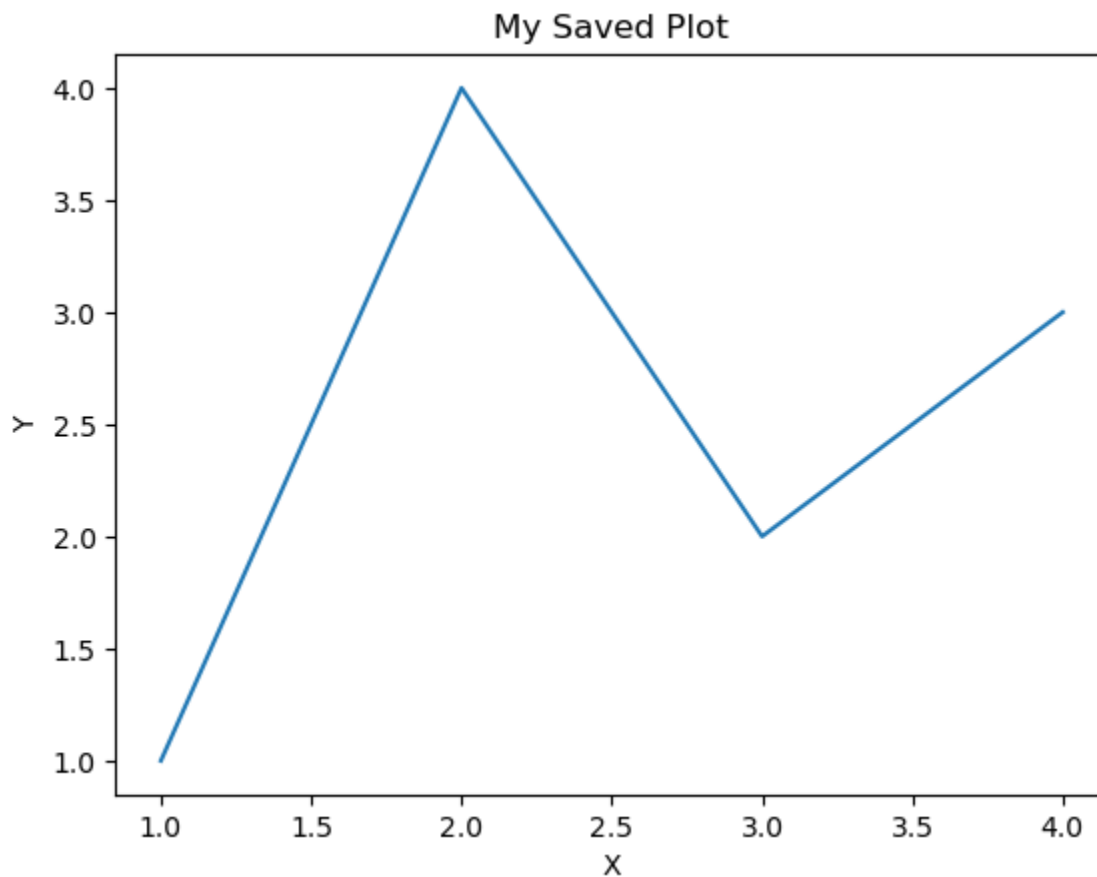


Figure and Axes

Understanding the Figure Object:

- The **Figure** is the top-level container in Matplotlib. Think of it as the entire window or page where your plots will be drawn.
- A Figure can contain one or more **Axes** objects (which represent the actual plots).
- You can control the overall size, layout, and other high-level properties of the Figure.

In [8]:

```
import matplotlib.pyplot as plt

# Create a new figure with a specific size (width=8 inches, height=6 inches)
fig = plt.figure(figsize=(8, 6))
```

<Figure size 800x600 with 0 Axes>

Understanding the Axes Object (Subplots):

- The **Axes** object is what you typically think of as "a plot". It's the region of the Figure where your data is displayed.
- Each Axes has an x-axis, a y-axis, a data-plotting area, and can have a title, labels, etc.
- A single Figure can contain multiple Axes, arranged as subplots.

Creating Figures and Axes Explicitly:

- You can create a **Figure** object using `plt.figure()` . This gives you more control over the Figure's properties right from the start.
- Once you have a Figure, you can add **Axes** to it using the `fig.add_subplot()` method.
- `fig.add_subplot(nrows, ncols, index)` allows you to specify the grid layout of subplots (number of rows, number of columns) and the position of the new subplot (1-based index).

In [10]:

```
import numpy as np

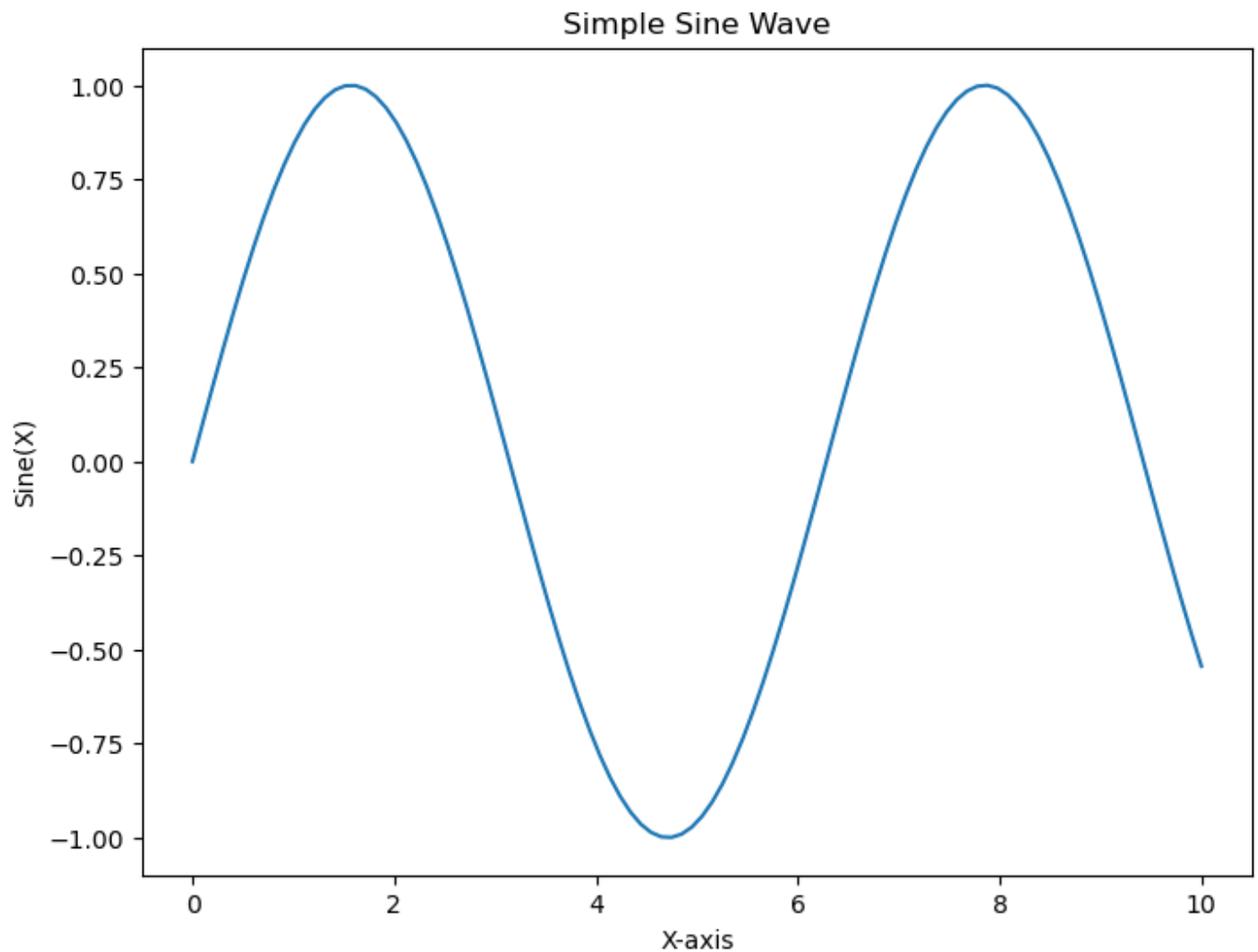
# Create a new figure
fig = plt.figure(figsize=(8, 6)) # Optional: set figure size

# Add a subplot to the figure (1 row, 1 column, first subplot)
ax1 = fig.add_subplot(1, 1, 1)

# Sample data
x = np.linspace(0, 10, 100)
y = np.sin(x)

# Plot on the axes
ax1.plot(x, y)
ax1.set_xlabel('X-axis')
ax1.set_ylabel('Sine(X)')
ax1.set_title('Simple Sine Wave')

# Show the figure
plt.show()
```



In [11]:

```
# Create a new figure
fig = plt.figure(figsize=(10, 5))

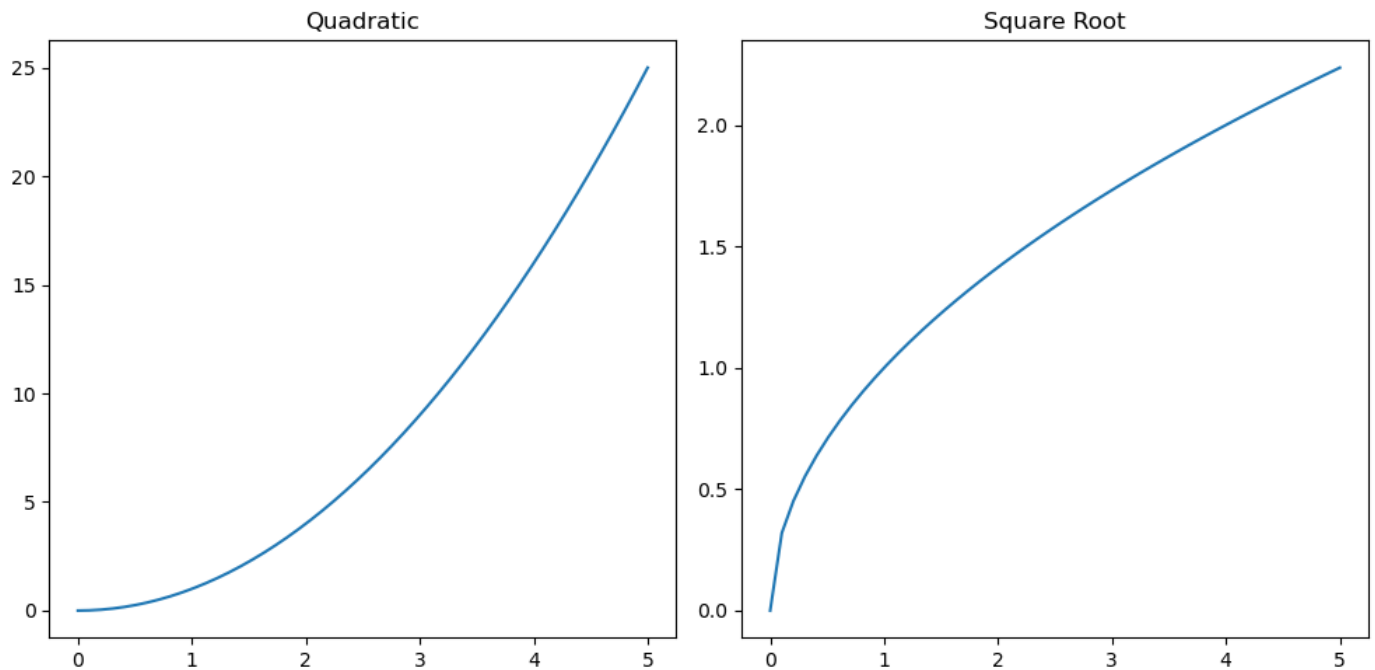
# Add two subplots side by side
ax1 = fig.add_subplot(1, 2, 1) # 1 row, 2 columns, first subplot
ax2 = fig.add_subplot(1, 2, 2) # 1 row, 2 columns, second subplot

# Data for the first subplot
x1 = np.linspace(0, 5, 50)
y1 = x1**2
ax1.plot(x1, y1)
ax1.set_title('Quadratic')

# Data for the second subplot
x2 = np.linspace(0, 5, 50)
y2 = np.sqrt(x2)
ax2.plot(x2, y2)
ax2.set_title('Square Root')

# Adjust layout to prevent overlapping
plt.tight_layout()

# Show the figure
plt.show()
```



Multiple Subplots using `plt.subplots()` :

- The `plt.subplots(nrows, ncols, sharex=False, sharey=False, squeeze=True, subplot_kw=None, gridspec_kw=None, **fig_kw)` function provides a convenient way to create a **Figure** and a set of subplots in one go.
- It returns the **Figure** object and a single **Axes** object (if only one subplot is created) or an array of **Axes** objects (for multiple subplots).
- `sharex` and `sharey` can be set to `True` to share the x or y axis limits and ticks across subplots.

In [12]:

```
# Create a figure with 2 rows and 2 columns of subplots
fig, axes = plt.subplots(2, 2, figsize=(10, 8))

# Sample data
x = np.linspace(0, 5, 50)

# Plot in the first subplot (row 0, column 0)
axes[0, 0].plot(x, np.sin(x))
axes[0, 0].set_title('Sine')

# Plot in the second subplot (row 0, column 1)
axes[0, 1].plot(x, np.cos(x))
axes[0, 1].set_title('Cosine')

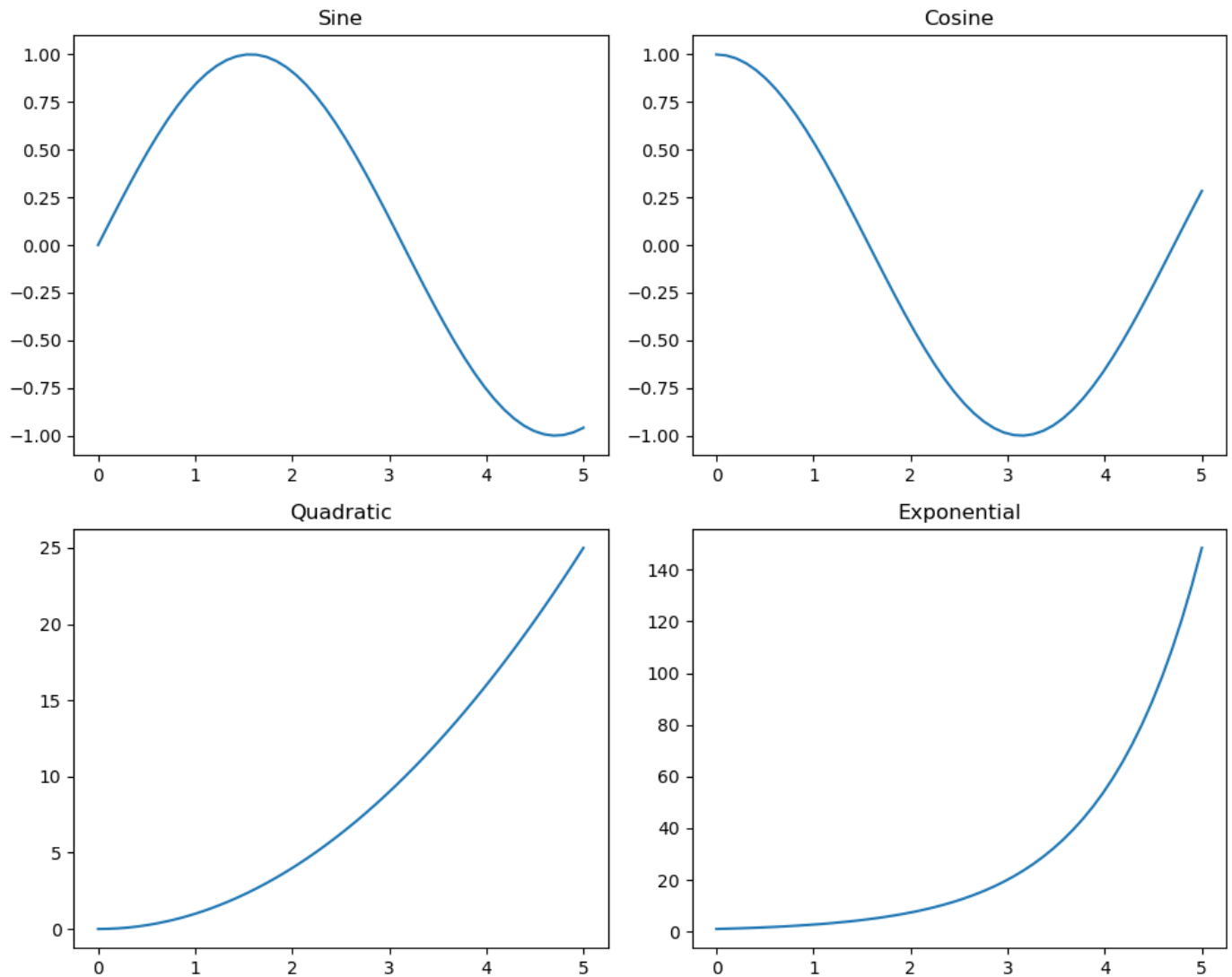
# Plot in the third subplot (row 1, column 0)
axes[1, 0].plot(x, x**2)
axes[1, 0].set_title('Quadratic')

# Plot in the fourth subplot (row 1, column 1)
axes[1, 1].plot(x, np.exp(x))
axes[1, 1].set_title('Exponential')

# Adjust layout
plt.tight_layout()
```



```
# Show the figure
plt.show()
```



Plotting Elements

Lines: Styling

You can customize the appearance of lines in your plots using various parameters within the `plt.plot()` function:

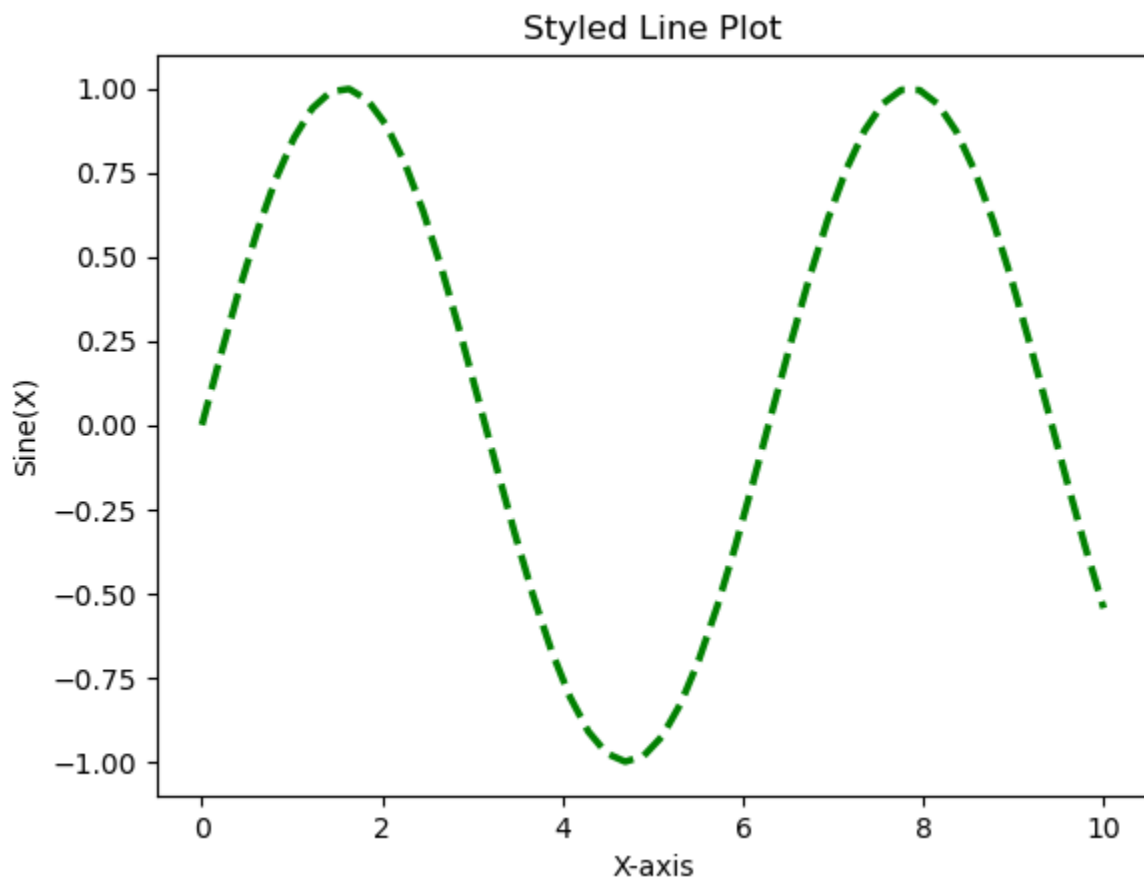
- **color** (or **c**): Specifies the line color (e.g., 'red', 'green', 'blue', or hex codes like '#FF0000').
- **linestyle** (or **ls**): Defines the line style (e.g., '-' for solid, '--' for dashed, '-.' for dash-dot, ':' for dotted, '' for no line).
- **linewidth** (or **lw**): Controls the thickness of the line.

In [14]:

```
x = np.linspace(0, 10, 50)
y = np.sin(x)

plt.plot(x, y, color='green', linestyle='--', linewidth=2.5)
```

```
plt.xlabel('X-axis')
plt.ylabel('Sine(X)')
plt.title('Styled Line Plot')
plt.show()
```



Lines:

Markers

Markers can be added to your line plots to highlight individual data points:

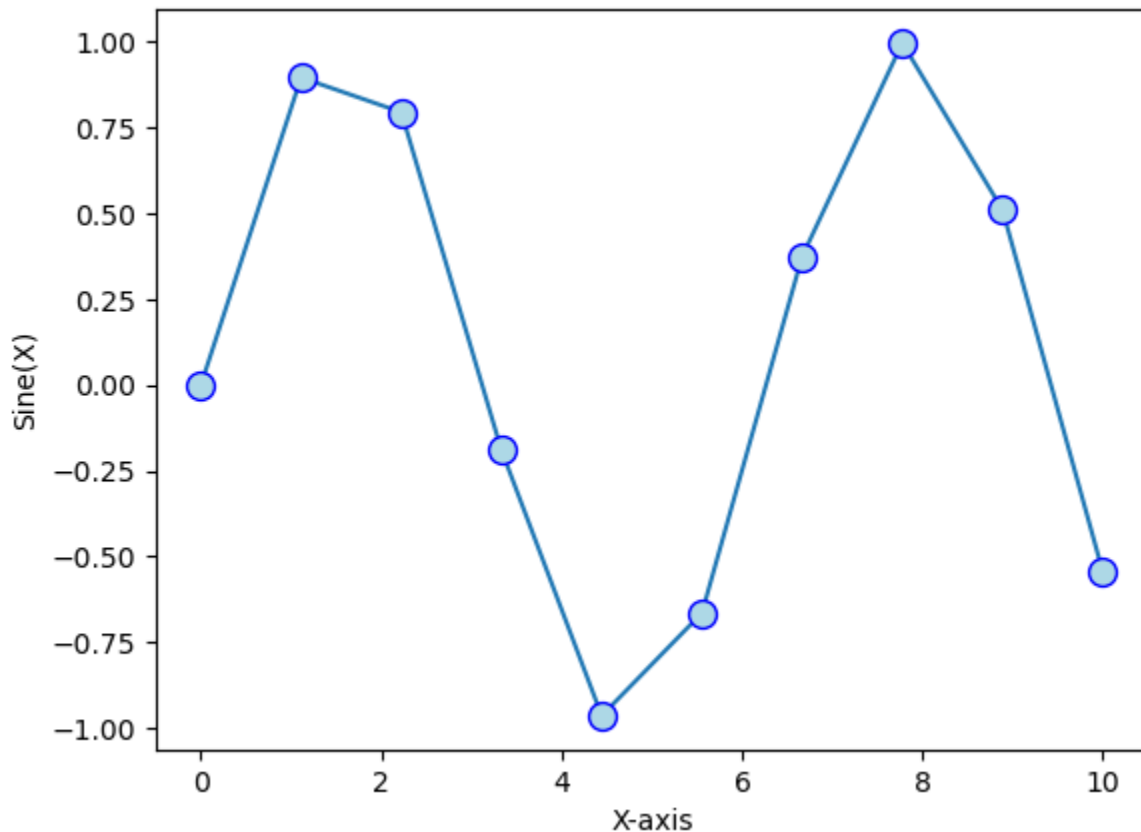
- **marker**: Specifies the marker style (e.g., 'o' for circle, 's' for square, '^' for triangle, '.' for point).
- **markersize** (or **ms**): Controls the size of the markers.
- **markerfacecolor** (or **mfc**): Sets the fill color of the markers.
- **markeredgecolor** (or **mec**): Sets the color of the marker edges.

In [15]:

```
x = np.linspace(0, 10, 10)
y = np.sin(x)

plt.plot(x, y, marker='o', markersize=10, markerfacecolor='lightblue', markeredgecolor='')
plt.xlabel('X-axis')
plt.ylabel('Sine(X)')
plt.title('Line Plot with Markers')
plt.show()
```

Line Plot with Markers



Scatter Plots:

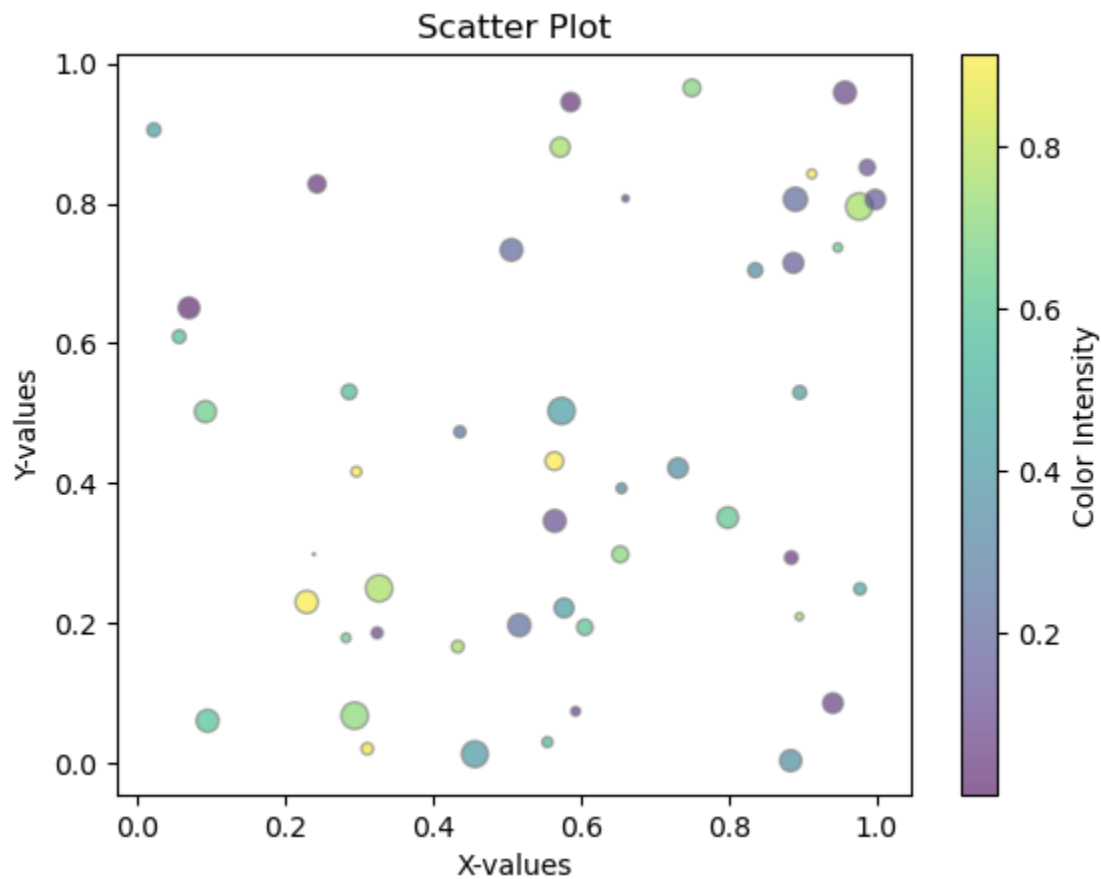
For individual data points without connecting lines, use `plt.scatter()` :

- It takes separate arrays for x and y coordinates.
- You can customize marker properties like:
 - `s` : size
 - `c` : color (can be a single color or an array for different colors)
 - `marker` : marker style
 - `alpha` : transparency
 - `edgecolors` : color of the marker edges

In [16]:

```
x = np.random.rand(50)
y = np.random.rand(50)
colors = np.random.rand(50)
sizes = 100 * np.random.rand(50)

plt.scatter(x, y, s=sizes, c=colors, alpha=0.6, cmap='viridis', edgecolors='gray')
plt.xlabel('X-values')
plt.ylabel('Y-values')
plt.title('Scatter Plot')
plt.colorbar(label='Color Intensity')
plt.show()
```



Bars:

Vertical Bars:

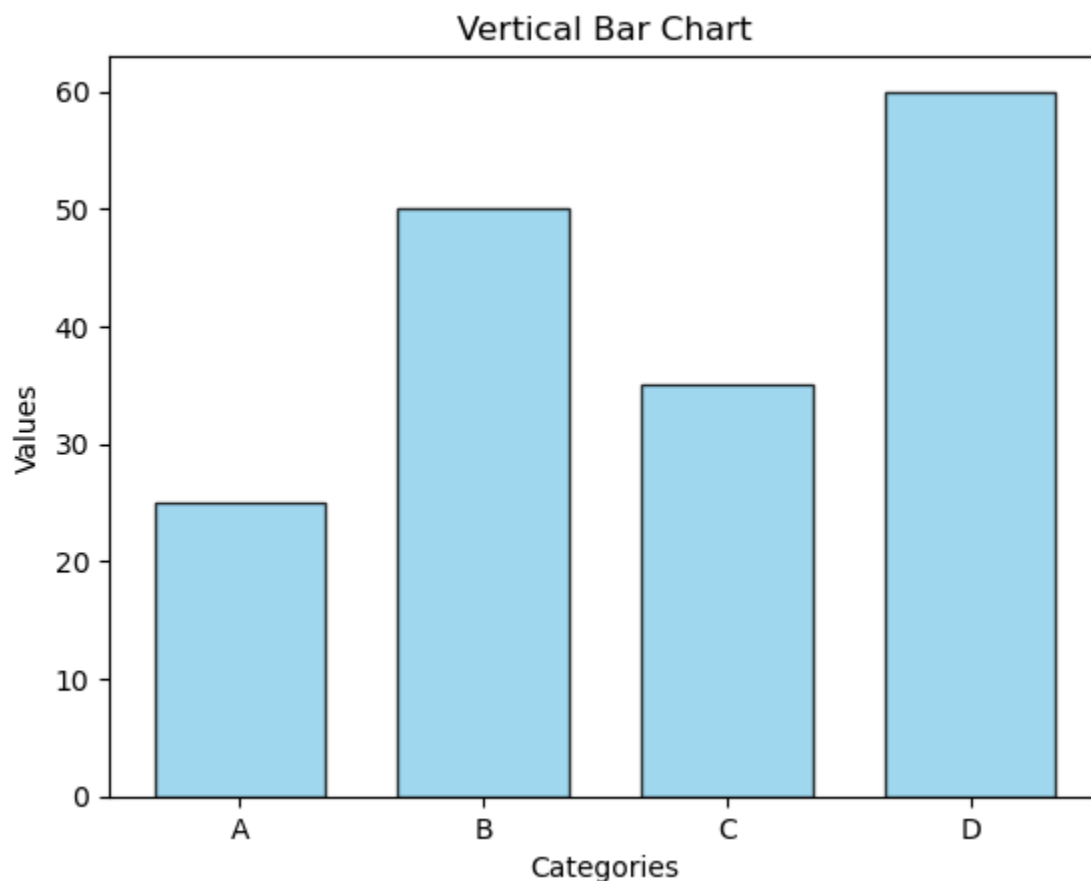
Use `plt.bar()` to create vertical bar charts:

- Provide x-coordinates (usually labels or categories) and the height of the bars.
- Customize properties like `width`, `color`, `edgecolor`, `linewidth`, `alpha`.

In [17]:

```
categories = ['A', 'B', 'C', 'D']
values = [25, 50, 35, 60]

plt.bar(categories, values, color='skyblue', width=0.7, edgecolor='black', alpha=0.8)
plt.xlabel('Categories')
plt.ylabel('Values')
plt.title('Vertical Bar Chart')
plt.show()
```



Horizontal Bars:

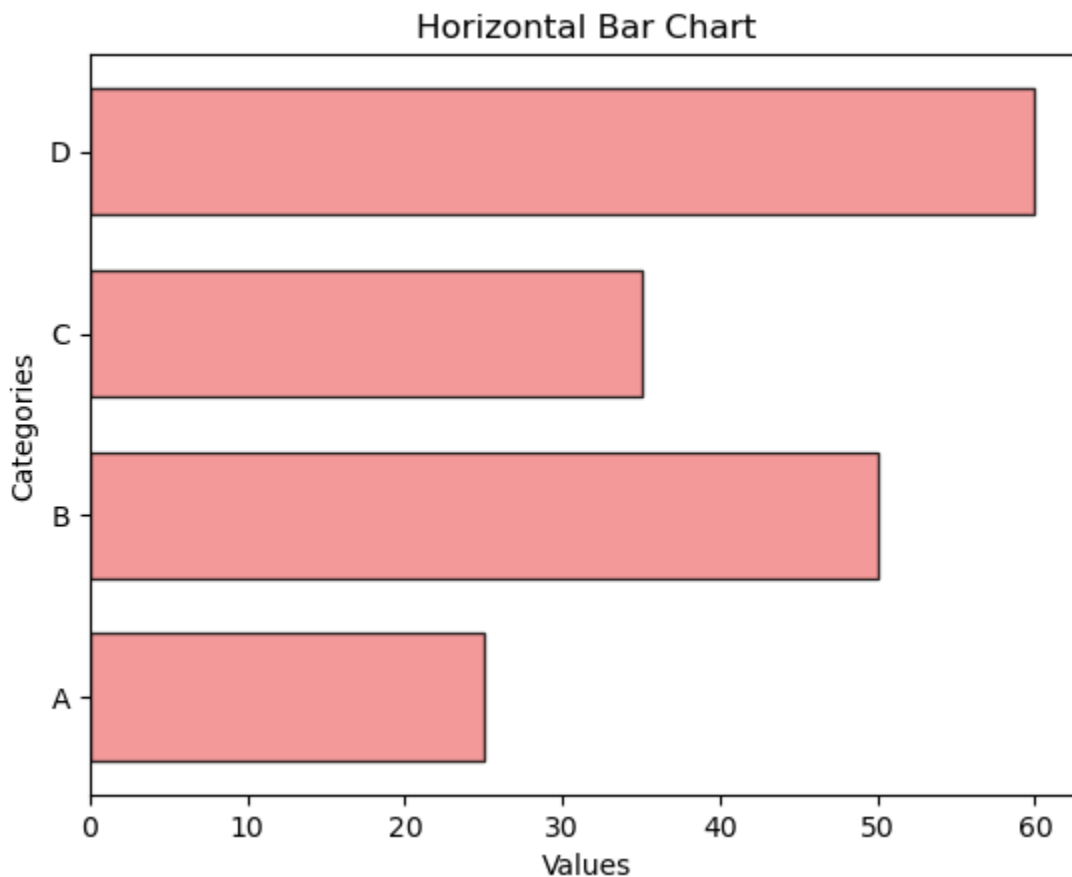
Use `plt.barh()` to create horizontal bar charts:

- Provide y-coordinates (usually labels or categories) and the width (length) of the bars.
- Customization options are similar to `plt.bar()`.

In [18]:

```
categories = ['A', 'B', 'C', 'D']
values = [25, 50, 35, 60]

plt.barh(categories, values, color='lightcoral', height=0.7, edgecolor='black', alpha=0.5)
plt.ylabel('Categories')
plt.xlabel('Values')
plt.title('Horizontal Bar Chart')
plt.show()
```



Histograms:

Use `plt.hist()` to visualize the distribution of a dataset:

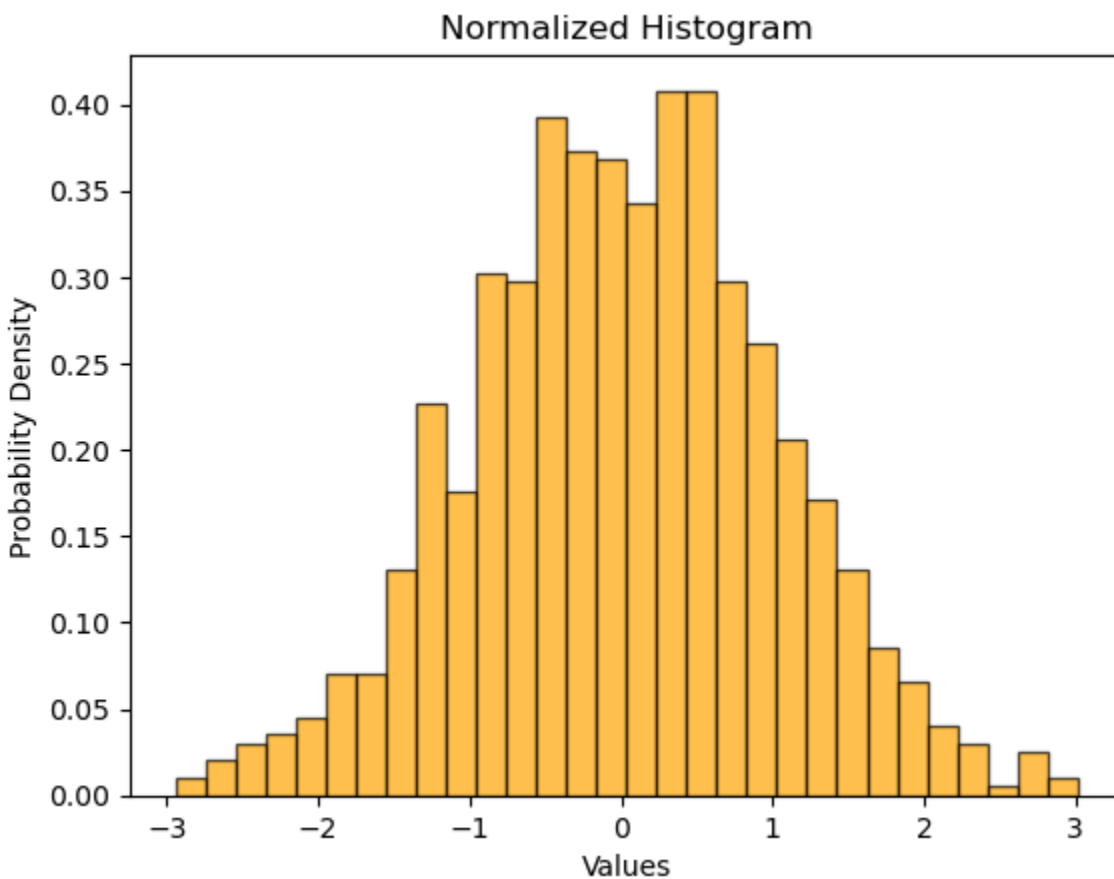
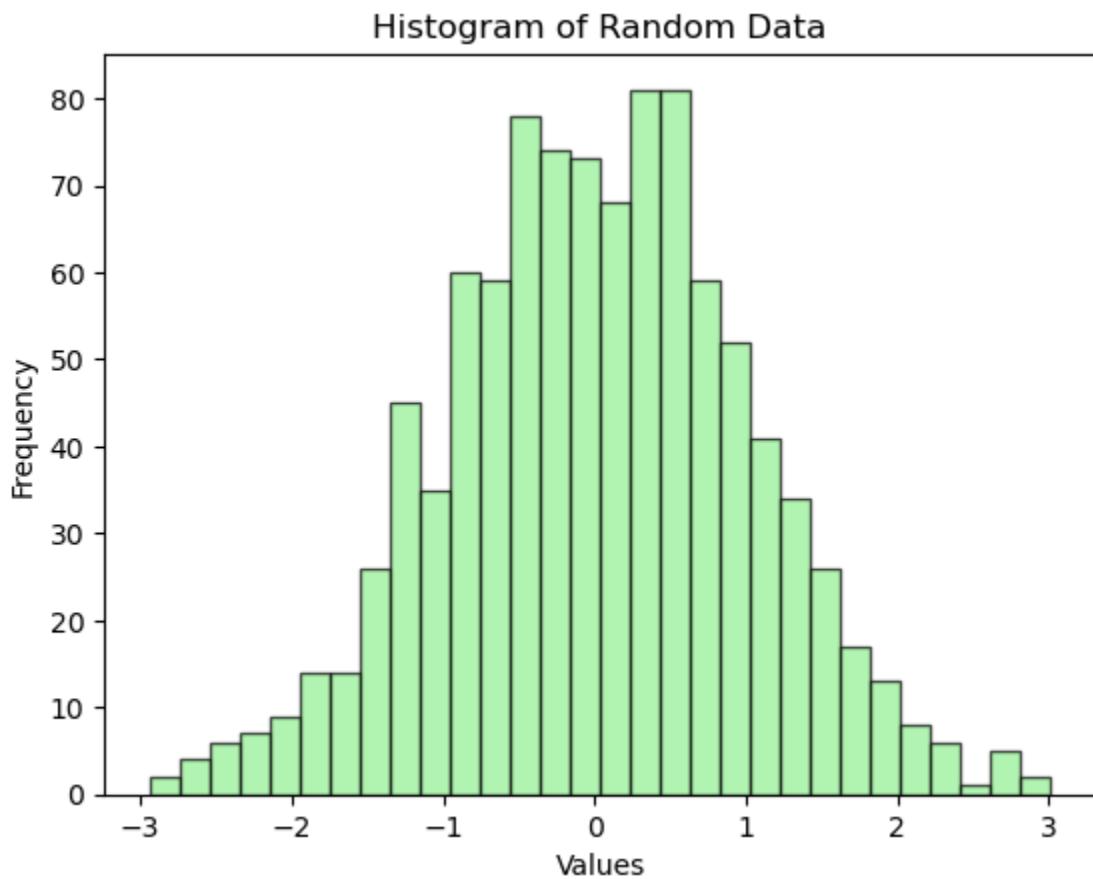
- Provide the data array and the number of `bins` .
- `density=True` can normalize the histogram to represent probability density.
- Other customizations include `color` , `alpha` , `edgecolor` , `histtype` .

In [19]:

```
data = np.random.randn(1000) # Generate random data from a normal distribution
```

```
plt.hist(data, bins=30, color='lightgreen', edgecolor='black', alpha=0.7)
plt.xlabel('Values')
plt.ylabel('Frequency')
plt.title('Histogram of Random Data')
plt.show()
```

```
plt.hist(data, bins=30, density=True, color='orange', edgecolor='black', alpha=0.7)
plt.xlabel('Values')
plt.ylabel('Probability Density')
plt.title('Normalized Histogram')
plt.show()
```



Pie Charts:

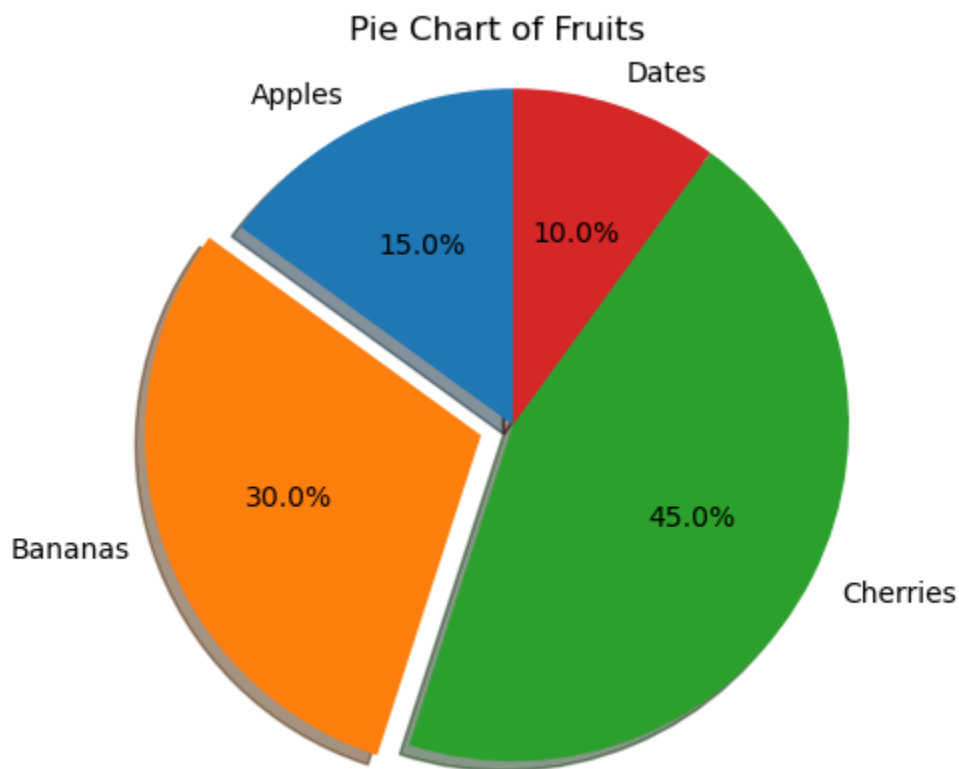
Use `plt.pie()` to represent proportions of different categories in a circle:

- Provide the `sizes` of each wedge.
- `labels` for each slice.
- `explode` to offset slices.
- `autopct` to display percentages (e.g., `'%.1f%%'`).
- `shadow=True` to add a shadow effect.

In [20]:

```
labels = ['Apples', 'Bananas', 'Cherries', 'Dates']
sizes = [15, 30, 45, 10]
explode = (0, 0.1, 0, 0) # Only explode the second slice (Bananas)

plt.pie(sizes, explode=explode, labels=labels, autopct='%1.1f%%', shadow=True, startangle=0)
plt.title('Pie Chart of Fruits')
plt.axis('equal') # Equal aspect ratio ensures that pie is drawn as a circle.
plt.show()
```



Area Plots:

- `plt.fill_between(x, y1, y2=0)` fills the area between two horizontal curves (`y1` and `y2` , defaulting `y2` to 0).
- `plt.stackplot(x, y1, y2, ...)` stacks multiple area plots.

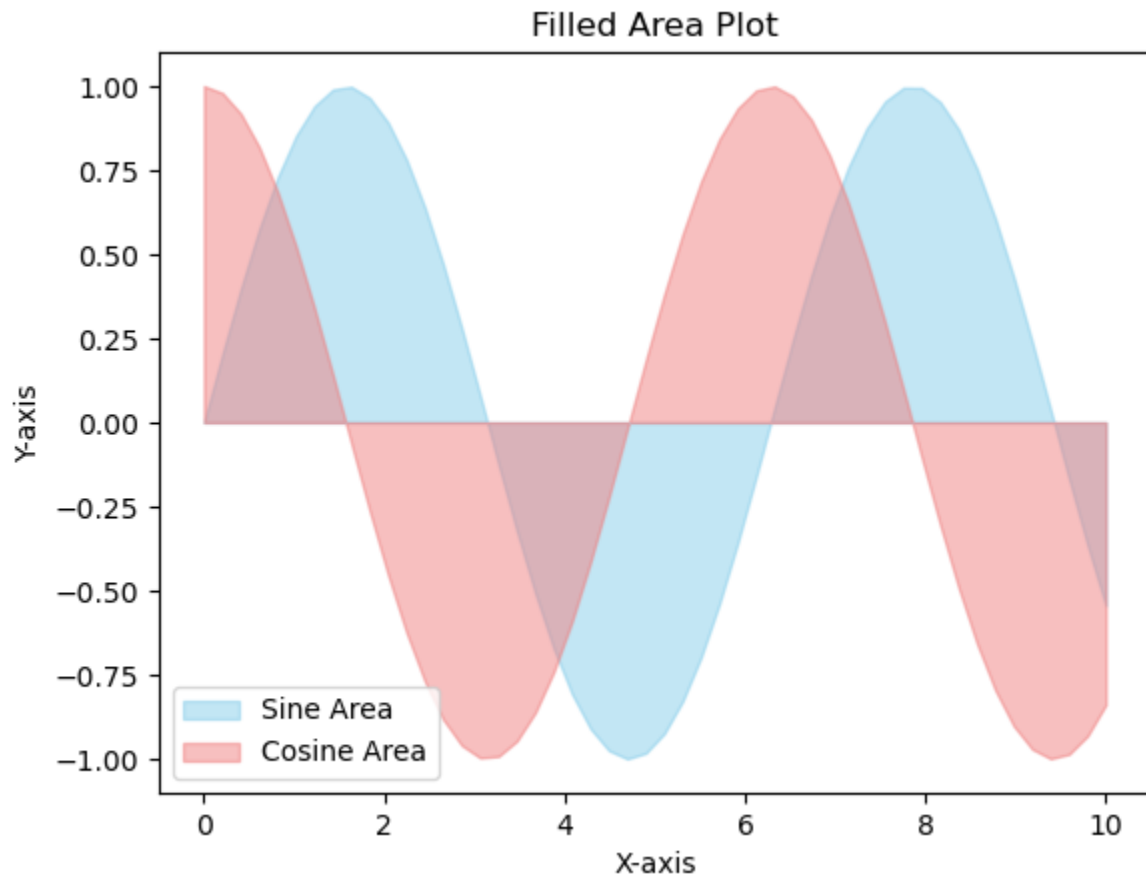
In [21]:

```
x = np.linspace(0, 10, 50)
y1 = np.sin(x)
y2 = np.cos(x)

plt.fill_between(x, y1, color='skyblue', alpha=0.5, label='Sine Area')
plt.fill_between(x, y2, color='lightcoral', alpha=0.5, label='Cosine Area')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
```



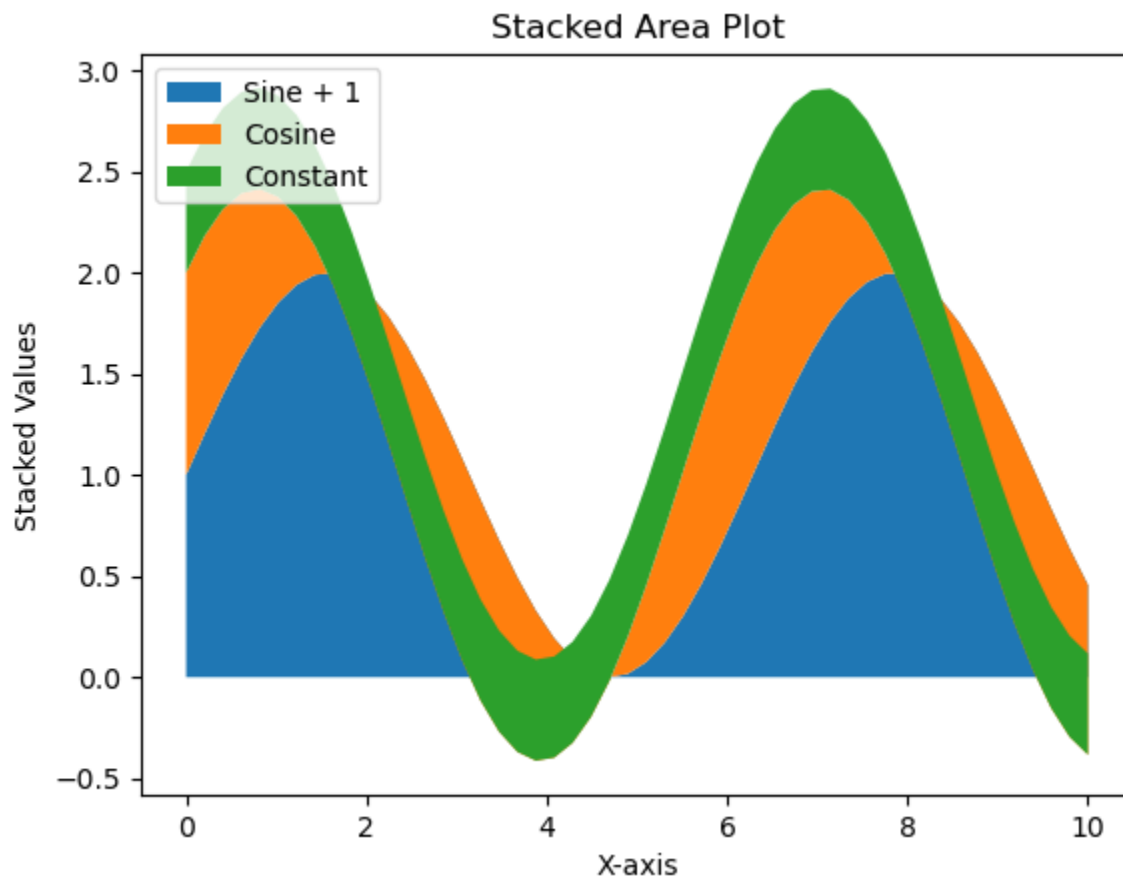
```
plt.title('Filled Area Plot')
plt.legend()
plt.show()
```



In [22]:

```
x = np.linspace(0, 10, 50)
y1 = np.sin(x) + 1
y2 = np.cos(x)
y3 = np.ones_like(x) * 0.5

plt.stackplot(x, y1, y2, y3, labels=['Sine + 1', 'Cosine', 'Constant'])
plt.xlabel('X-axis')
plt.ylabel('Stacked Values')
plt.title('Stacked Area Plot')
plt.legend(loc='upper left')
plt.show()
```



Labels, Titles, and Legends

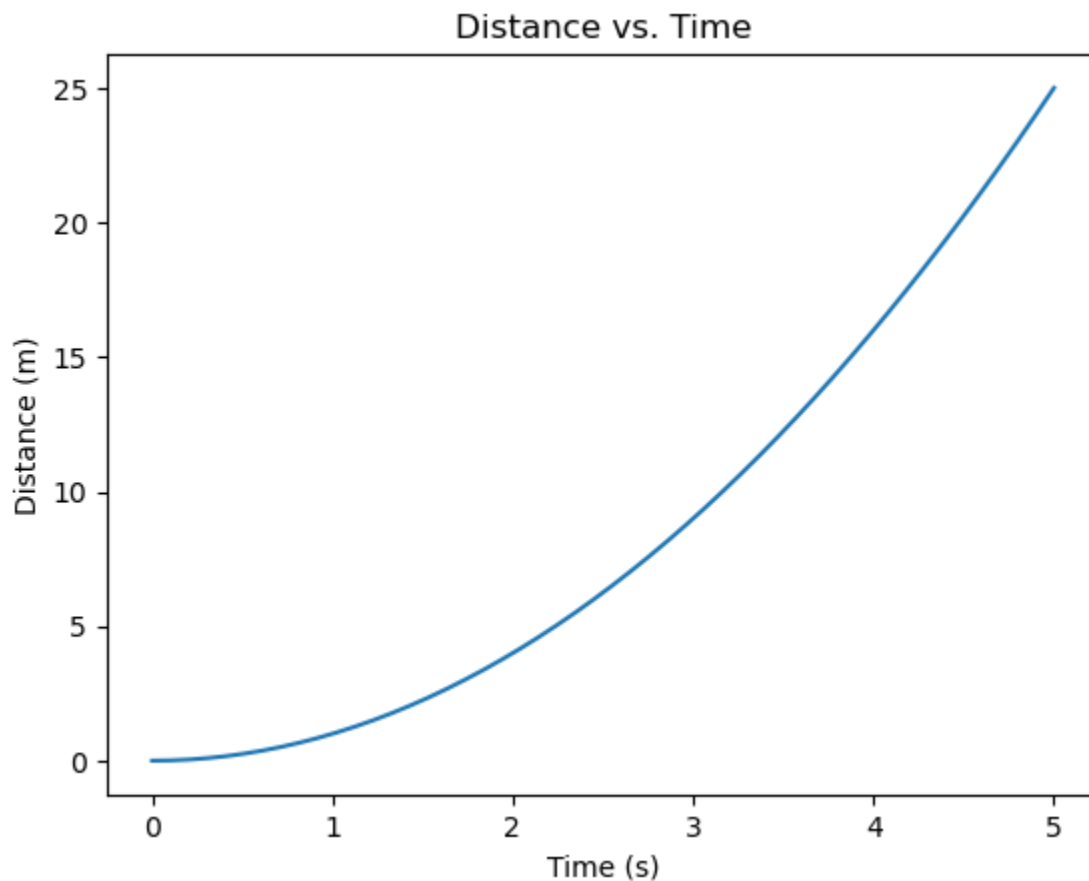
Setting Axis Labels:

- You can add descriptive labels to the x and y axes using the `plt.xlabel()` and `plt.ylabel()` functions, respectively.
- These labels help clarify what the axes represent.

In [23]:

```
x = np.linspace(0, 5, 50)
y = x**2

plt.plot(x, y)
plt.xlabel('Time (s)')
plt.ylabel('Distance (m)')
plt.title('Distance vs. Time')
plt.show()
```



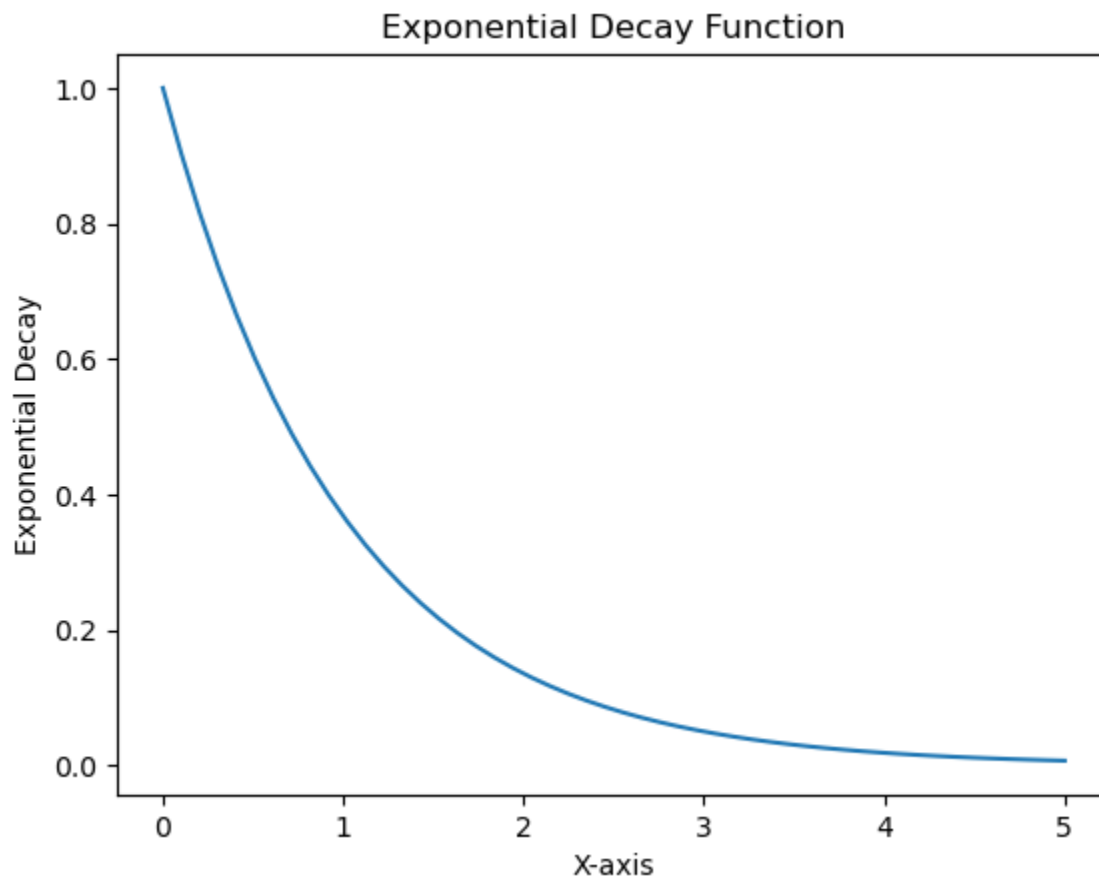
Setting Plot Titles:

- The `plt.title()` function allows you to add a title to your plot.
- A well-chosen title provides context and summarizes the information presented in the visualization.

In [24]:

```
x = np.linspace(0, 5, 50)
y = np.exp(-x)

plt.plot(x, y)
plt.xlabel('X-axis')
plt.ylabel('Exponential Decay')
plt.title('Exponential Decay Function')
plt.show()
```



Adding and Customizing Legends:

- Legends are essential when you have multiple plots within the same Axes.
- To add a legend, you need to provide a `label` argument to each plotting function (`plt.plot()`, `plt.scatter()`, etc.), and then call `plt.legend()`.
- You can customize the legend's appearance and location using arguments within `plt.legend()`.

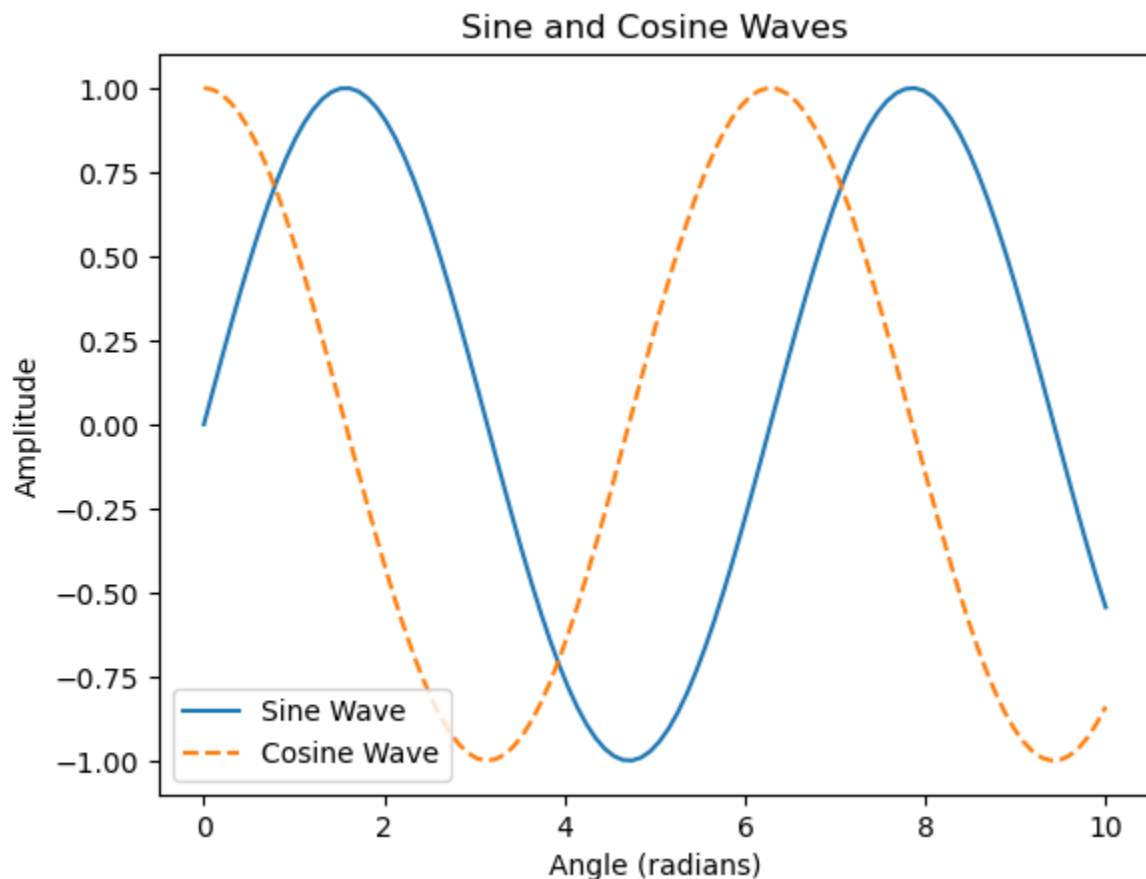
In [25]:

```
x = np.linspace(0, 10, 100)
y1 = np.sin(x)
y2 = np.cos(x)

plt.plot(x, y1, label='Sine Wave', linestyle='-')
plt.plot(x, y2, label='Cosine Wave', linestyle='--')

plt.xlabel('Angle (radians)')
plt.ylabel('Amplitude')
plt.title('Sine and Cosine Waves')
plt.legend() # Display the legend

plt.show()
```



Customizing the Legend:

You can pass various arguments to `plt.legend()` to customize its appearance and placement:

- **loc:** Specifies the location of the legend. Common options include:
 - 'upper left', 'upper right', 'lower left', 'lower right'
 - 'center left', 'center right', 'lower center', 'upper center', 'center'
 - 'best' (Matplotlib automatically tries to find the least obtrusive location)
- **fontsize:** Controls the size of the text in the legend (e.g., 'small', 'large', or a numerical value).
- **ncol:** Sets the number of columns in the legend.
- **shadow:** If `True`, adds a shadow to the legend box.
- **frameon:** If `True` (default), draws a frame around the legend. Set to `False` to remove the frame.
- **facecolor:** Sets the background color of the legend box.
- **edgecolor:** Sets the color of the legend box border.

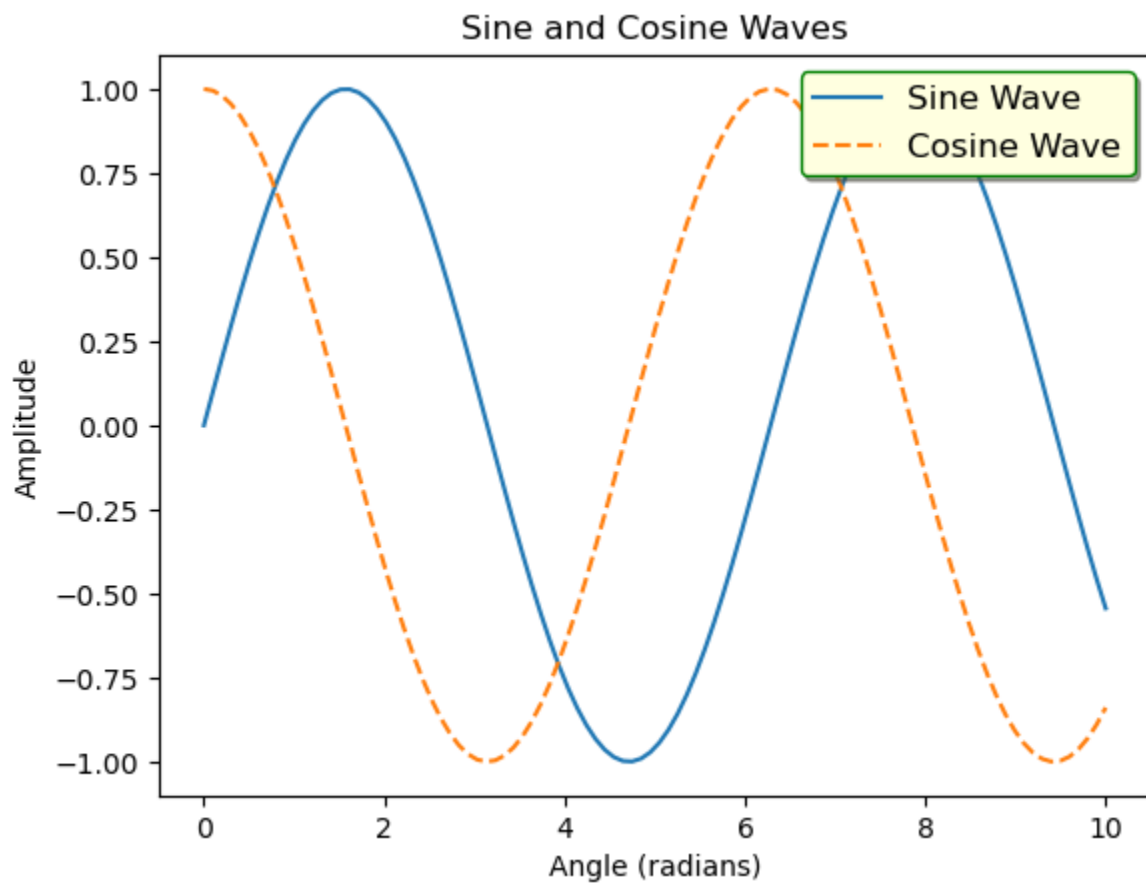
In [26]:

```
x = np.linspace(0, 10, 100)
y1 = np.sin(x)
y2 = np.cos(x)

plt.plot(x, y1, label='Sine Wave', linestyle='-')
plt.plot(x, y2, label='Cosine Wave', linestyle='--')

plt.xlabel('Angle (radians)')
plt.ylabel('Amplitude')
```

```
plt.title('Sine and Cosine Waves')
plt.legend(loc='upper right', fontsize='large', ncol=1, shadow=True, frameon=True, facec
plt.show()
```



Customization and Styling

Color Maps

Understanding and Using Different Colormaps (`cmap`):

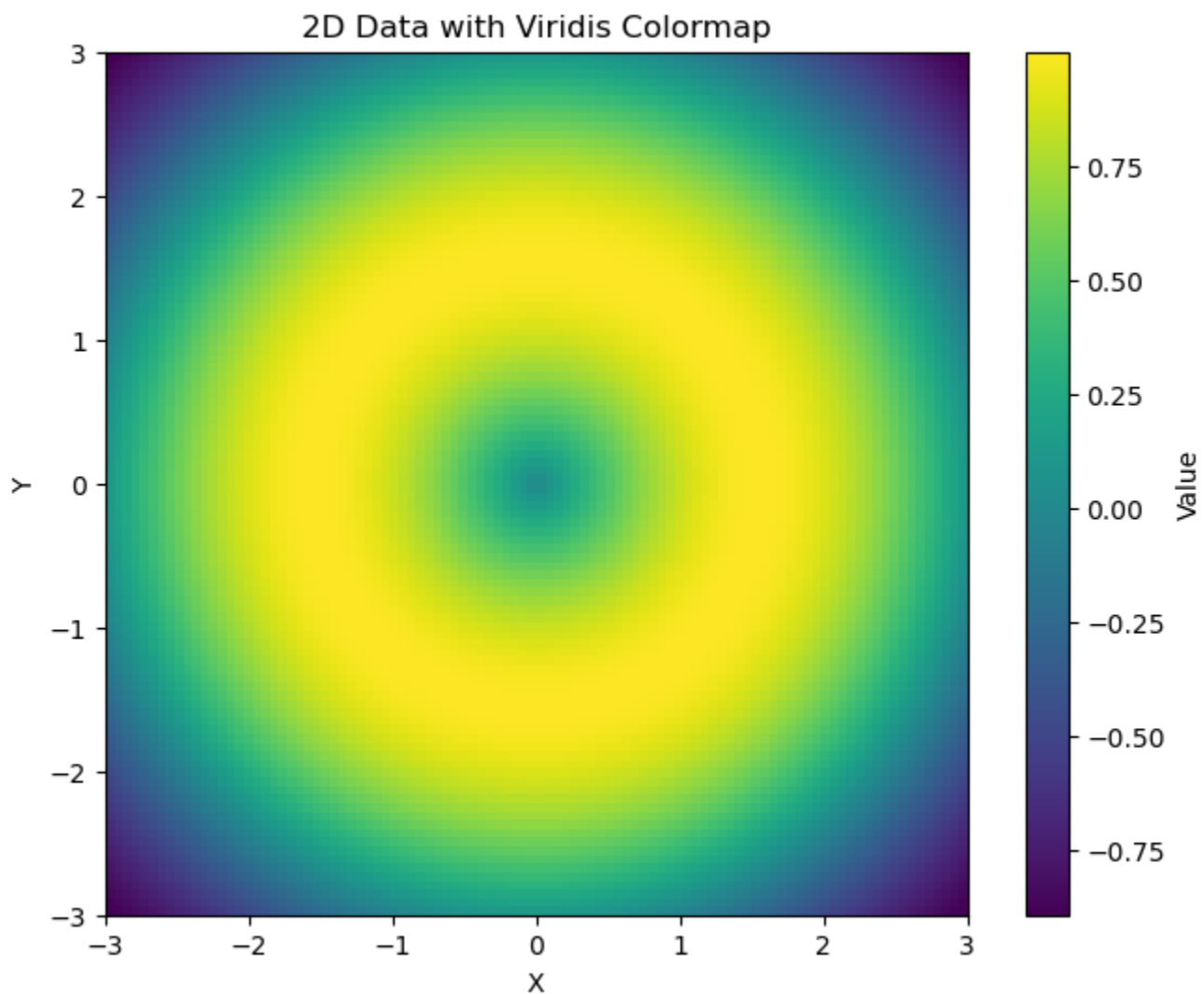
- A **colormap** maps data values to colors. Matplotlib offers diverse built-in colormaps (sequential, diverging, qualitative, cyclic).
- Use the `cmap` argument in functions like `plt.scatter()`, `plt.imshow()`, `plt.pcolormesh()` for 2D or scalar data visualization.
- Popular colormaps include:
 - **Sequential (perceptually uniform):** `'viridis'`, `'plasma'`, `'magma'`, `'inferno'`
 - **Diverging:** `'coolwarm'`, `'seismic'`
 - **Qualitative:** `'Set1'`, `'tab10'`
- **Perceptually uniform colormaps** are recommended for sequential data to avoid misinterpretations of data variations.

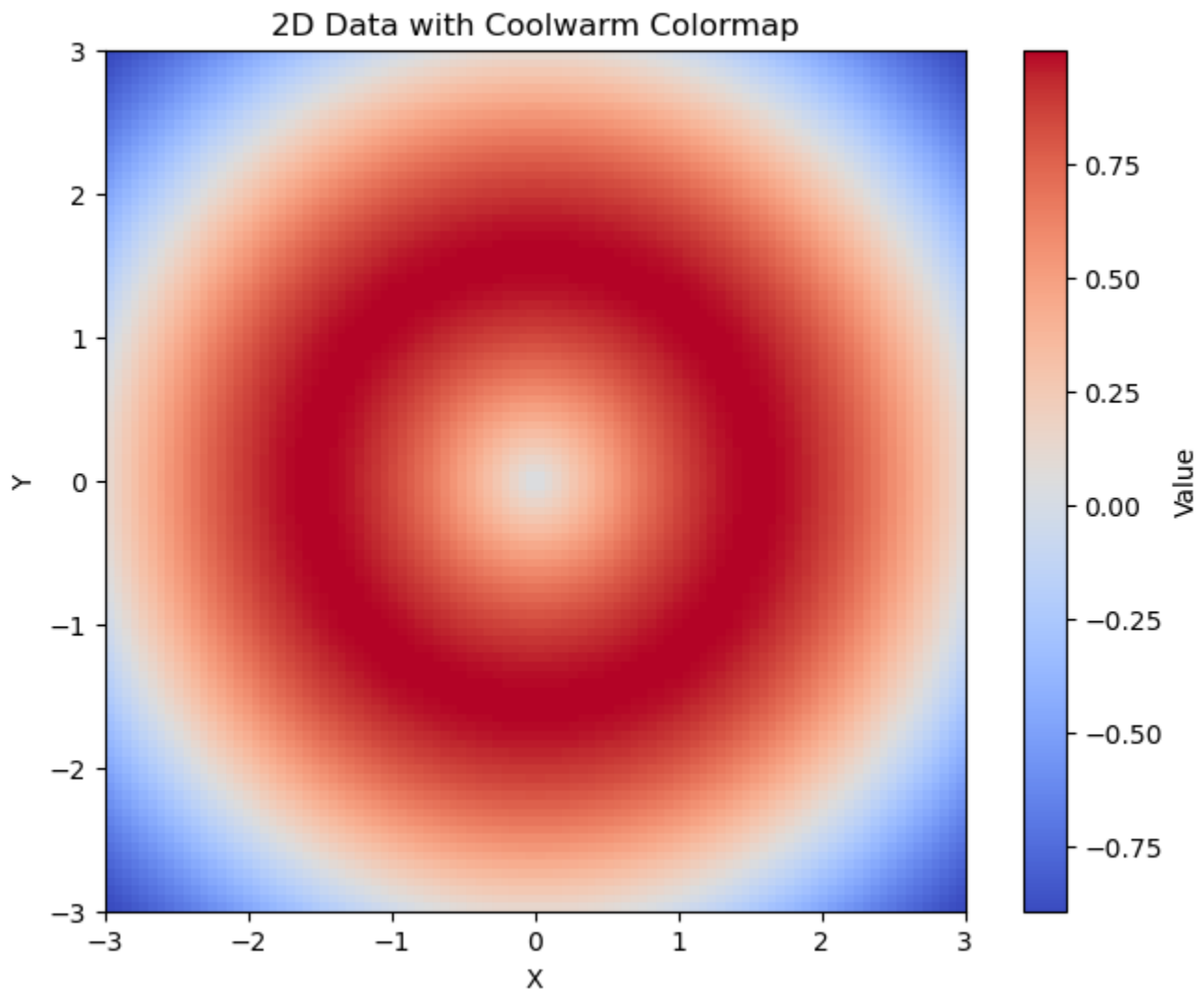
In [27]:

```
# Sample 2D data
X, Y = np.meshgrid(np.linspace(-3, 3, 100), np.linspace(-3, 3, 100))
Z = np.sin(np.sqrt(X**2 + Y**2))
```

```
# Visualize with a colormap using plt.imshow()
plt.figure(figsize=(8, 6))
plt.imshow(Z, cmap='viridis', origin='lower', extent=[-3, 3, -3, 3])
plt.colorbar(label='Value')
plt.title('2D Data with Viridis Colormap')
plt.xlabel('X')
plt.ylabel('Y')
plt.show()

# Try a different colormap
plt.figure(figsize=(8, 6))
plt.imshow(Z, cmap='coolwarm', origin='lower', extent=[-3, 3, -3, 3])
plt.colorbar(label='Value')
plt.title('2D Data with Coolwarm Colormap')
plt.xlabel('X')
plt.ylabel('Y')
plt.show()
```





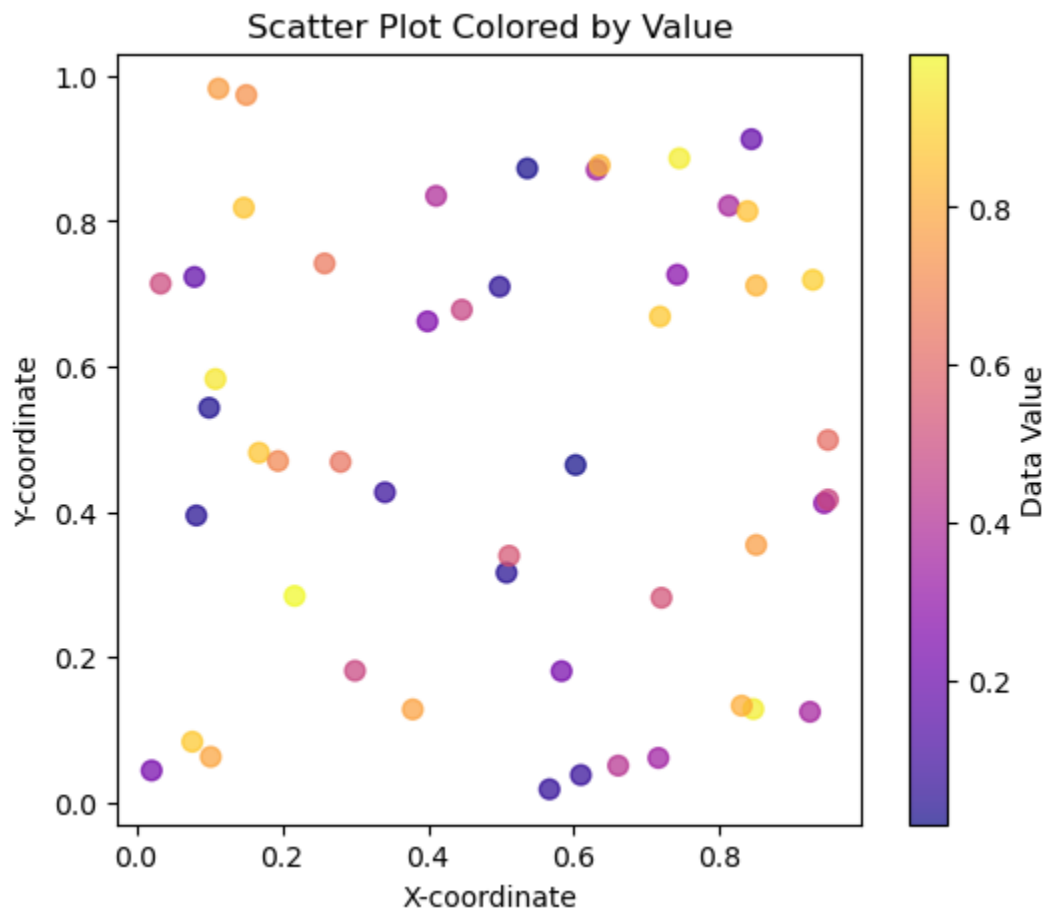
Applying Colormaps to Scatter Plots:

- In `plt.scatter()`, the `c` argument can take an array of values.
- These values are then mapped to colors according to the specified `cmap`, allowing visualization of a third data dimension through color.

In [28]:

```
# Sample data with a value associated with each point
x = np.random.rand(50)
y = np.random.rand(50)
values = np.random.rand(50)

# Scatter plot with colors based on 'values'
plt.figure(figsize=(6, 5))
scatter = plt.scatter(x, y, c=values, cmap='plasma', s=50, alpha=0.7)
plt.colorbar(scatter, label='Data Value')
plt.xlabel('X-coordinate')
plt.ylabel('Y-coordinate')
plt.title('Scatter Plot Colored by Value')
plt.show()
```

Creating Custom Colormaps (Brief Introduction):

- The `matplotlib.colors` module enables the creation of custom colormaps.
- This involves defining a list of color tuples (RGB or RGBA).
- Two main classes for custom colormaps:
 - **LinearSegmentedColormap** : Defines color transitions linearly between key points.
 - **ListedColormap** : Creates a colormap from a discrete list of colors.

In [29]:

```
from matplotlib.colors import LinearSegmentedColormap

# Define the color segments as a dictionary
cdict = {'red': [(0.0, 0.0, 0.0), # (value, start_intensity, end_intensity)
                (0.5, 1.0, 1.0),
                (1.0, 1.0, 1.0)],

         'green': [(0.0, 0.0, 0.0),
                  (0.5, 0.0, 1.0),
                  (1.0, 0.0, 0.0)],

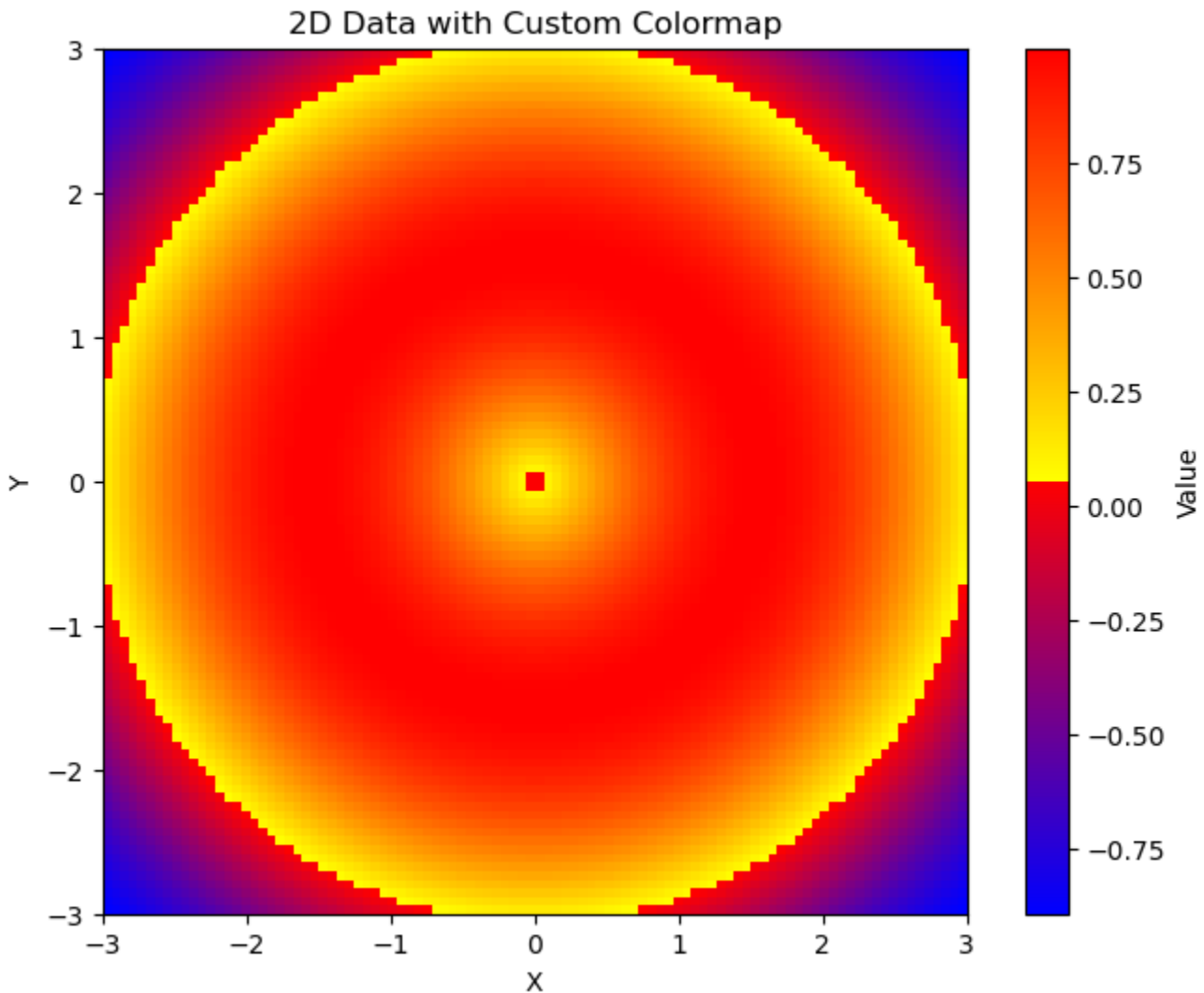
         'blue': [(0.0, 1.0, 1.0),
                 (0.5, 0.0, 0.0),
                 (1.0, 0.0, 0.0)]}

# Create the custom colormap
custom_cmap = LinearSegmentedColormap('my_cmap', cdict)

# Sample 2D data
```

```
X, Y = np.meshgrid(np.linspace(-3, 3, 100), np.linspace(-3, 3, 100))
Z = np.sin(np.sqrt(X**2 + Y**2))

# Visualize with the custom colormap
plt.figure(figsize=(8, 6))
plt.imshow(Z, cmap=custom_cmap, origin='lower', extent=[-3, 3, -3, 3])
plt.colorbar(label='Value')
plt.title('2D Data with Custom Colormap')
plt.xlabel('X')
plt.ylabel('Y')
plt.show()
```



Line Styles and Marker Styles

Exploring Various Line Styles:

The `linestyle` (or `ls`) parameter in `plt.plot()` controls the appearance of connecting lines. Common styles include:

Style String	Description
'-'	solid line
'--'	dashed line

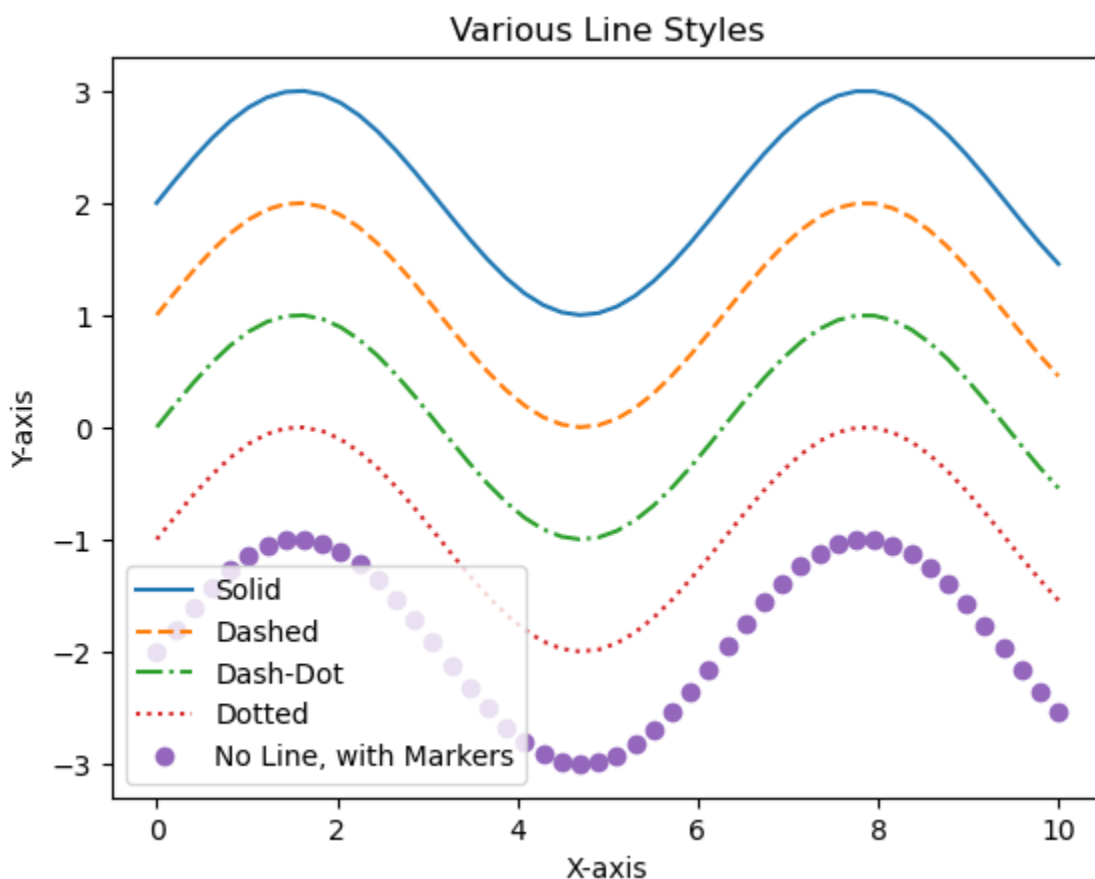
Style String	Description
'-.'	dash-dot line
':'	dotted line
' ' or ''	draw nothing

In [30]:

```
x = np.linspace(0, 10, 50)
y = np.sin(x)

plt.plot(x, y + 2, linestyle='-', label='Solid')
plt.plot(x, y + 1, linestyle='--', label='Dashed')
plt.plot(x, y, linestyle='-.', label='Dash-Dot')
plt.plot(x, y - 1, linestyle=':', label='Dotted')
plt.plot(x, y - 2, linestyle=' ', marker='o', label='No Line, with Markers') # Example w

plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.title('Various Line Styles')
plt.legend()
plt.show()
```



Exploring Various Marker Styles:

The `marker` parameter in `plt.plot()` and `plt.scatter()` specifies symbols for data points. Common styles include:

Marker String	Description
'o'	circle
'.'	point
','	pixel
'x'	cross
'+'	plus
's'	square
'd'	diamond
'D'	filled diamond
'^'	triangle_up
'v'	triangle_down
'<'	triangle_left
'>'	triangle_right
'*'	star
'h'	hexagon1
'H'	hexagon2
'p'	pentagon

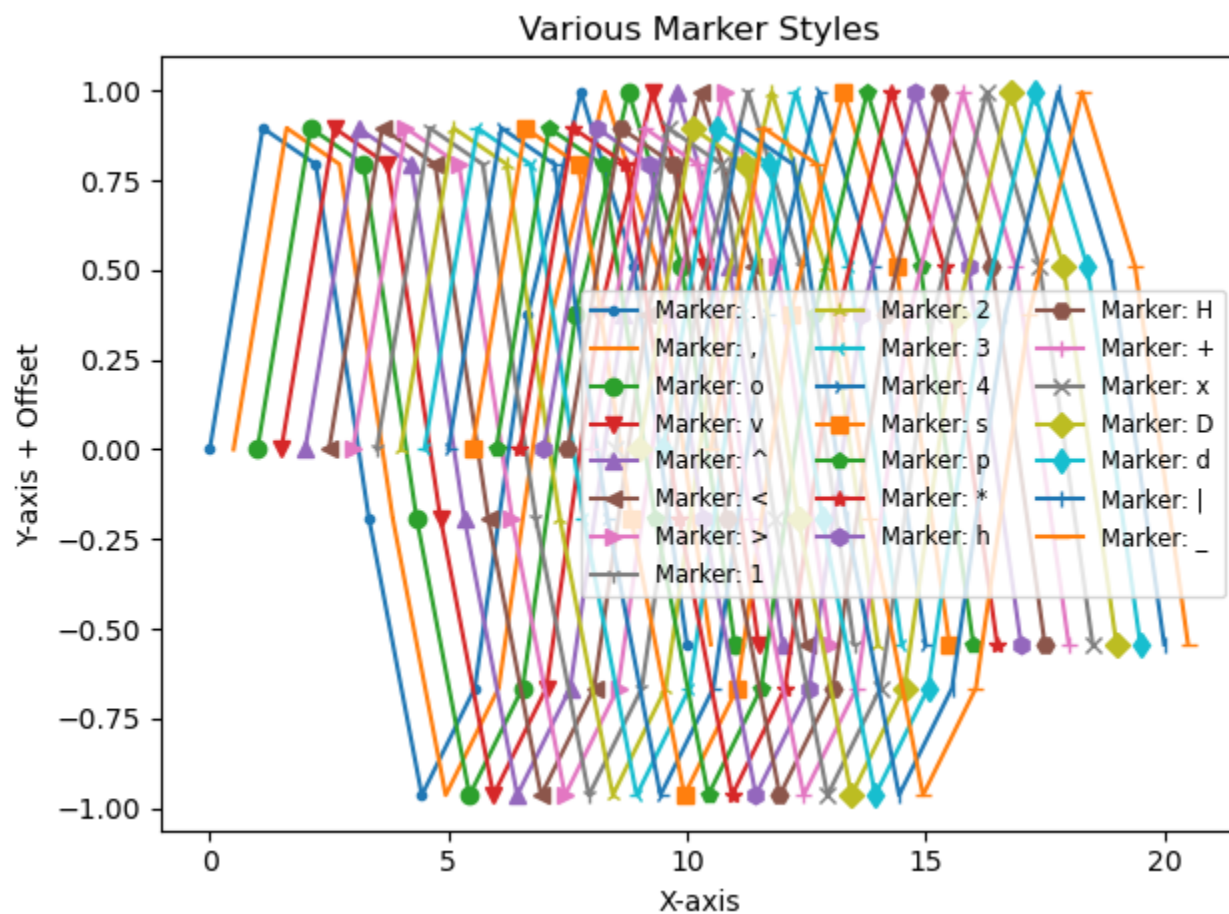
In [31]:

```
x = np.linspace(0, 10, 10)
y = np.sin(x)

markers = ['.', ',', 'o', 'v', '^', '<', '>', '1', '2', '3', '4', 's', 'p', '*', 'h', 'H']

for i, marker in enumerate(markers):
    plt.plot(x + i * 0.5, y, marker=marker, linestyle='-', label=f'Marker: {marker}')

plt.xlabel('X-axis')
plt.ylabel('Y-axis + Offset')
plt.title('Various Marker Styles')
plt.legend(ncol=3, fontsize='small')
plt.tight_layout()
plt.show()
```



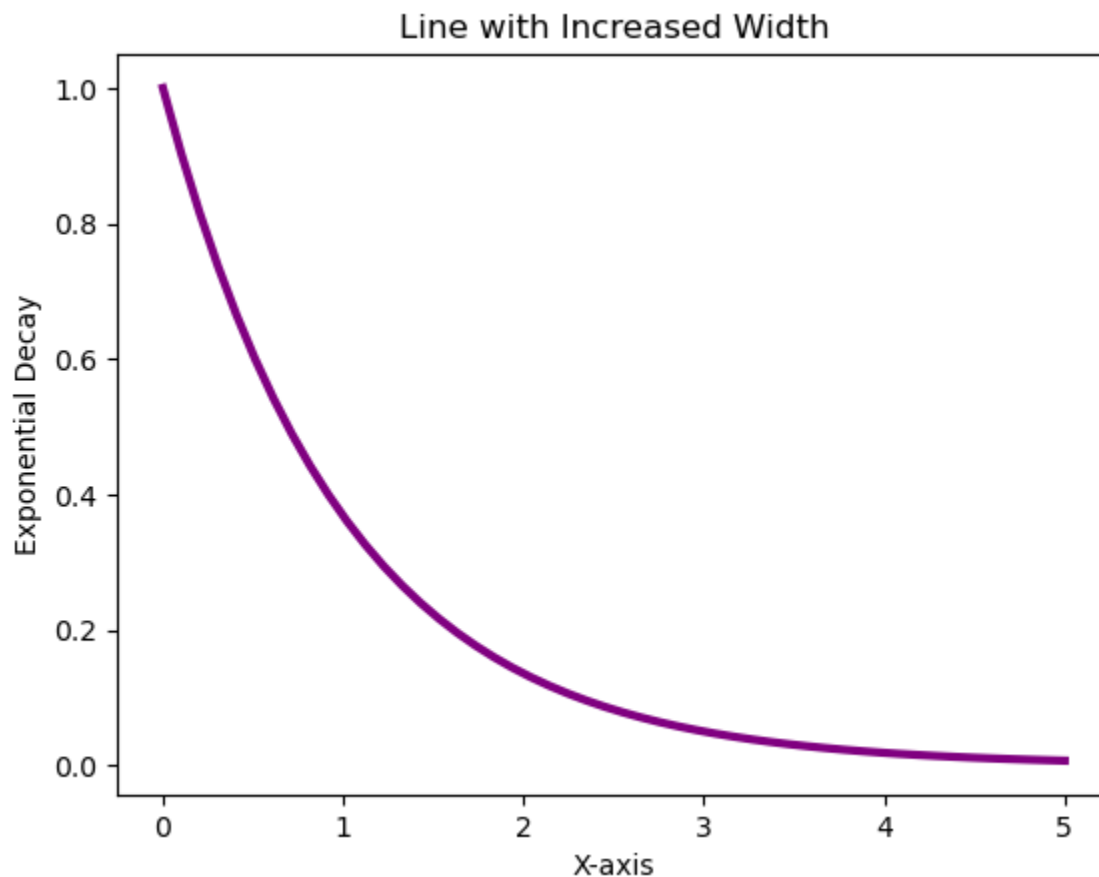
Customizing Line Width:

- The `linewidth` (or `lw`) parameter in `plt.plot()` controls line thickness.
- Specify a floating-point number for the desired width.

In [32]:

```
x = np.linspace(0, 5, 50)
y = np.exp(-x)

plt.plot(x, y, linewidth=3, color='purple')
plt.xlabel('X-axis')
plt.ylabel('Exponential Decay')
plt.title('Line with Increased Width')
plt.show()
```



Customizing Marker Size:

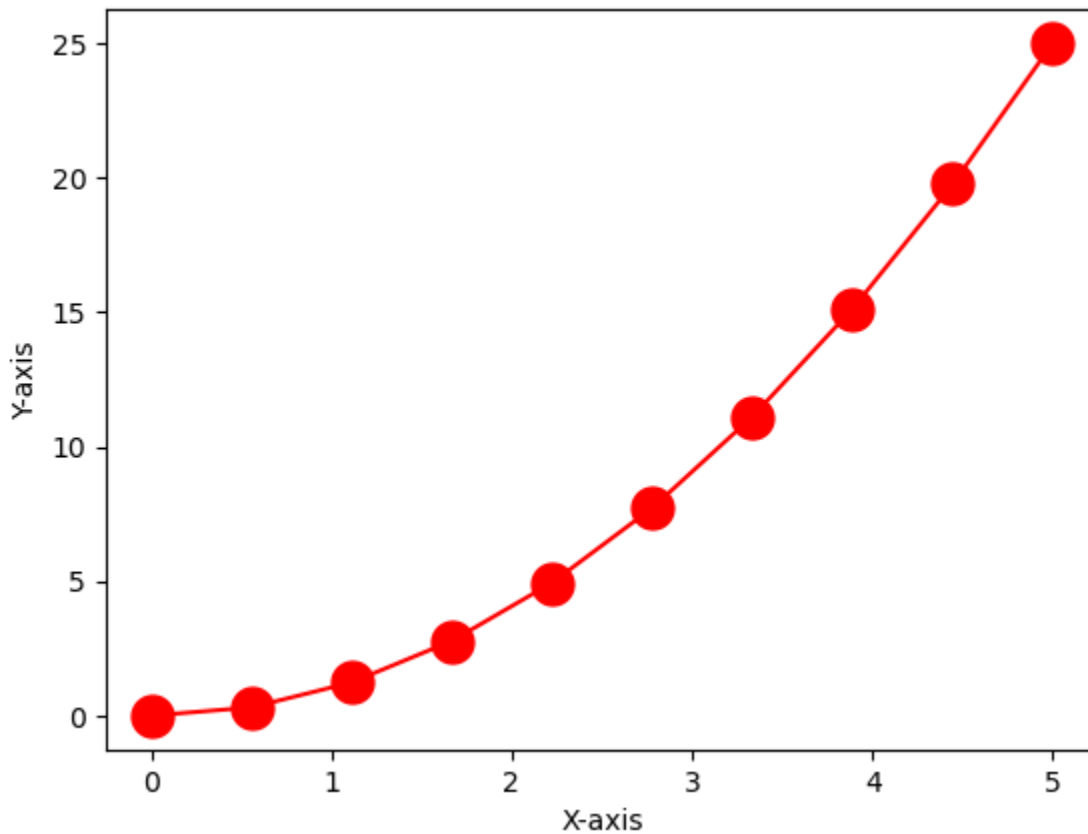
- The `markersize` (or `ms`) parameter in `plt.plot()` controls marker size. It's typically a scalar representing the marker's area.
- The `s` parameter in `plt.scatter()` also controls marker size, usually representing the area in points squared.

In [33]:

```
x = np.linspace(0, 5, 10)
y = x**2

plt.plot(x, y, marker='o', markersize=15, color='red')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.title('Markers with Increased Size (plot)')
plt.show()
```

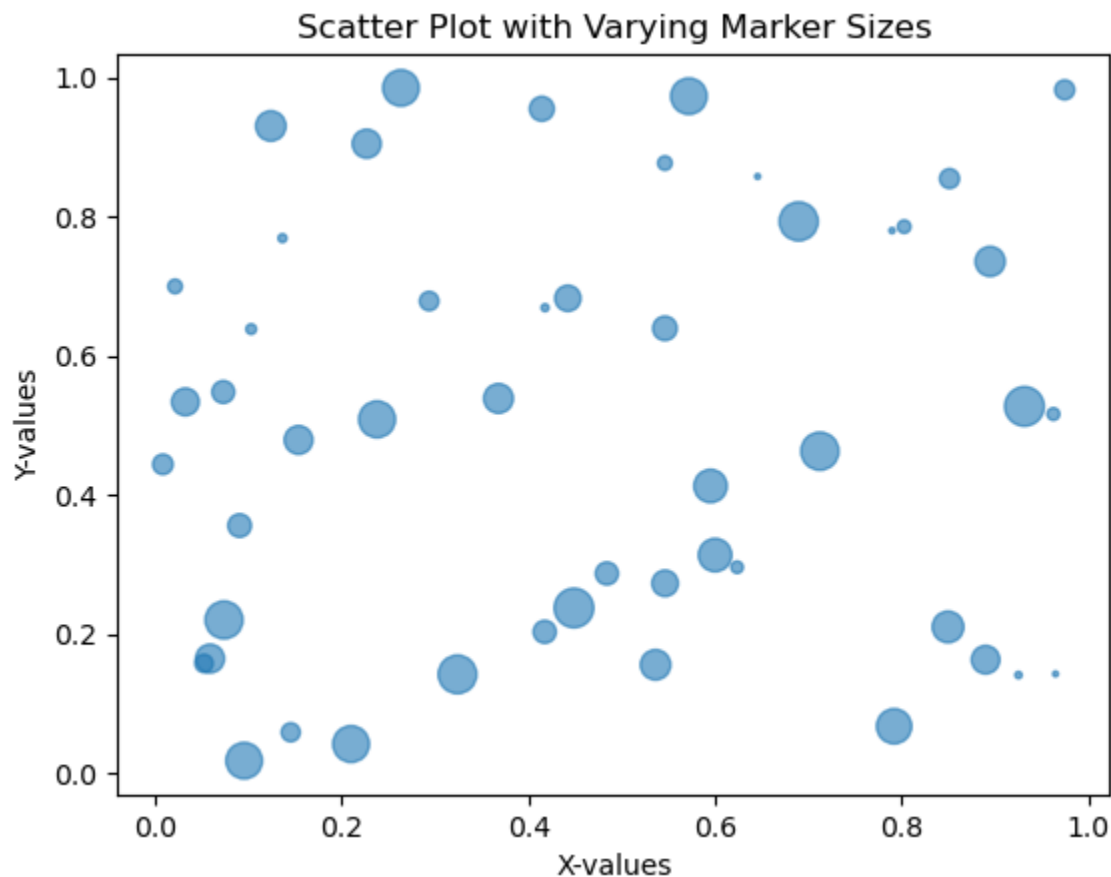
Markers with Increased Size (plot)



In [34]:

```
x = np.random.rand(50)
y = np.random.rand(50)
sizes = 200 * np.random.rand(50)

plt.scatter(x, y, s=sizes, alpha=0.6)
plt.xlabel('X-values')
plt.ylabel('Y-values')
plt.title('Scatter Plot with Varying Marker Sizes')
plt.show()
```



Tick Marks and Axis Limits

Setting and Formatting Tick Locations and Labels:

- The `plt.xticks()` and `plt.yticks()` functions control tick mark locations and labels.
- Pass a list/array of values to the first argument to set specific tick locations.
- Pass a corresponding list of strings to the second argument for custom labels.
- Customize label appearance using keyword arguments like `fontsize`, `rotation`, and `color`.

In [35]:

```
x = np.linspace(0, 10, 50)
y = np.sin(x)

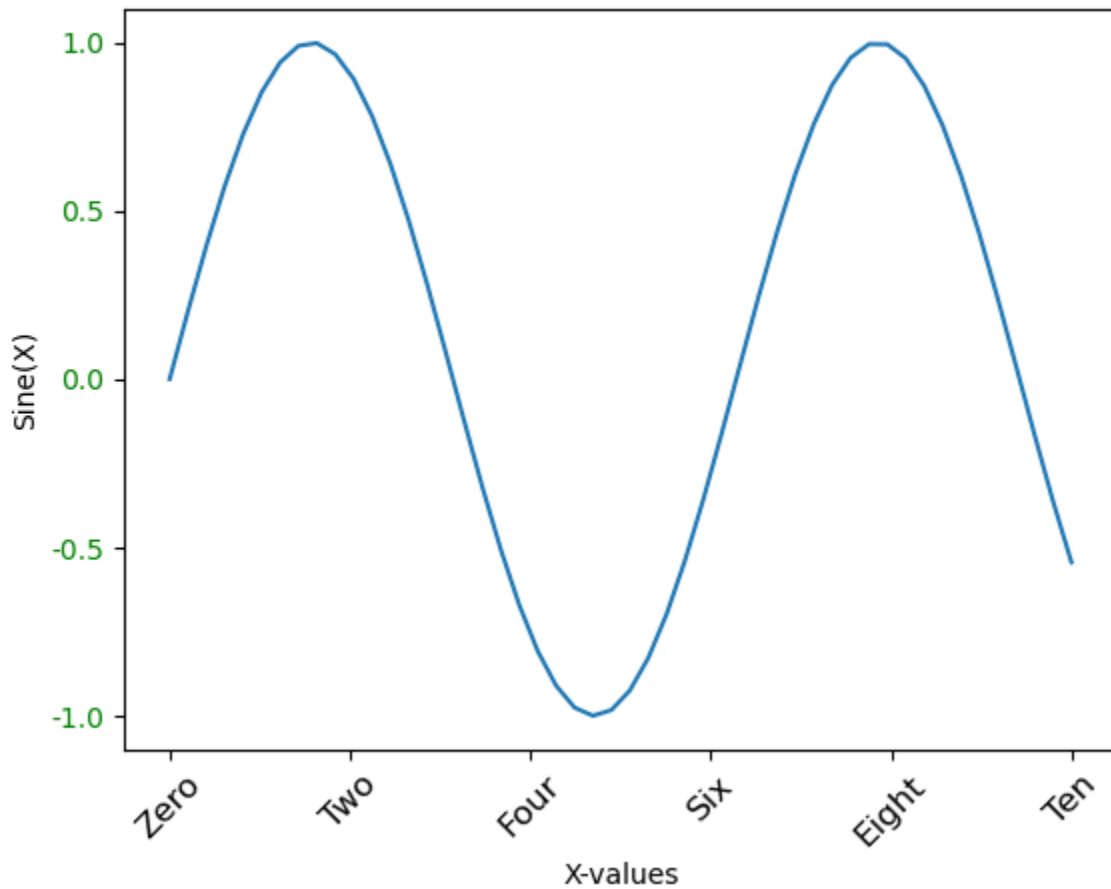
plt.plot(x, y)

# Customize x-axis ticks
plt.xticks([0, 2, 4, 6, 8, 10], ['Zero', 'Two', 'Four', 'Six', 'Eight', 'Ten'], fontsize

# Customize y-axis ticks
plt.yticks([-1, -0.5, 0, 0.5, 1], ['-1.0', '-0.5', '0.0', '0.5', '1.0'], color='green')

plt.xlabel('X-values')
plt.ylabel('Sine(X)')
plt.title('Customizing Tick Marks and Labels')
plt.show()
```


Customizing Tick Marks and Labels



Setting Axis Limits:

- The `plt.xlim(xmin, xmax)` function sets the minimum and maximum values for the x-axis. Calling it without arguments returns the current limits.
- Similarly, `plt.ylim(ymin, ymax)` sets the y-axis limits and returns the current limits if called without arguments.
- Explicitly setting axis limits helps focus on specific data regions or maintain consistency across plots.

In [36]:

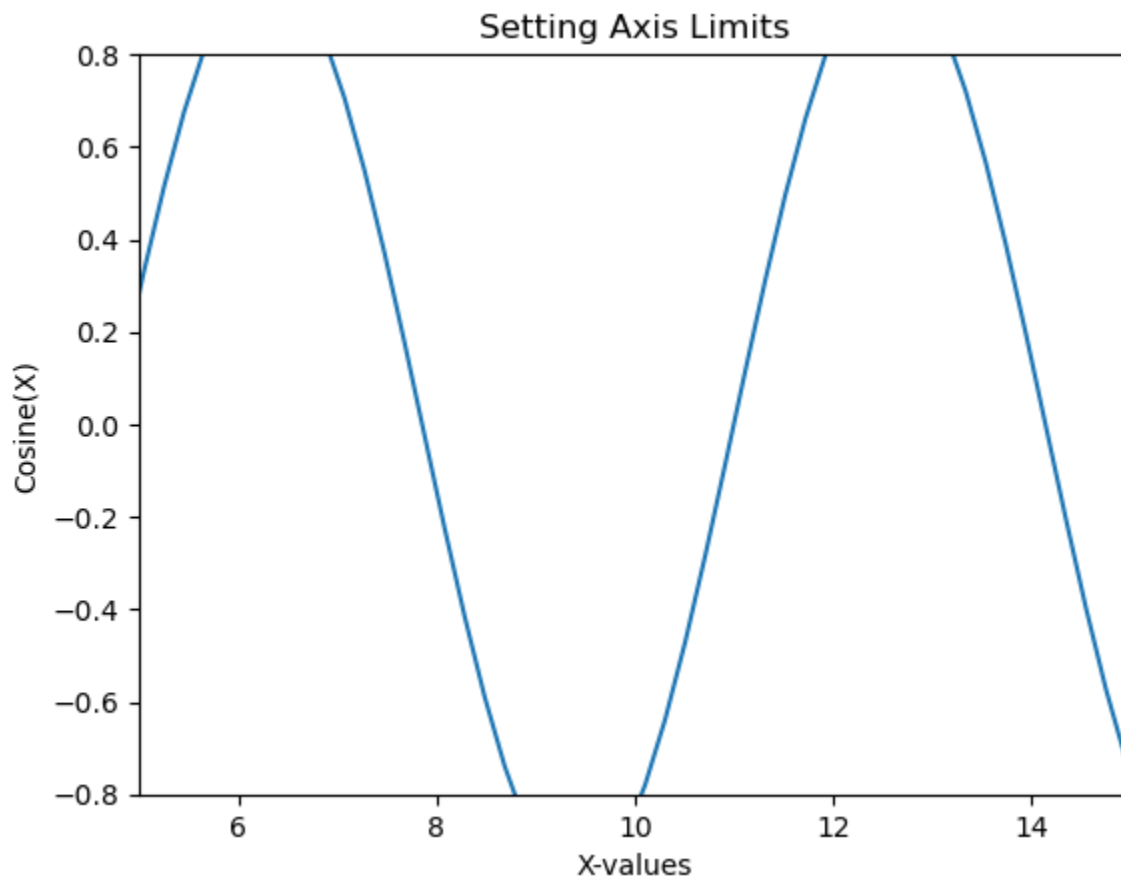
```
x = np.linspace(0, 20, 100)
y = np.cos(x)

plt.plot(x, y)

# Set x-axis limits
plt.xlim(5, 15)

# Set y-axis limits
plt.ylim(-0.8, 0.8)

plt.xlabel('X-values')
plt.ylabel('Cosine(X)')
plt.title('Setting Axis Limits')
plt.show()
```



Axis Scaling:

- Matplotlib supports various axis scales, with linear and logarithmic being common.
- Logarithmic scales are useful for data spanning several orders of magnitude.
- `plt.xscale('linear')` (default) sets a linear x-axis scale.
- `plt.xscale('log')` sets a logarithmic x-axis scale.
- Similarly, `plt.yscale('linear')` and `plt.yscale('log')` control the y-axis scaling.

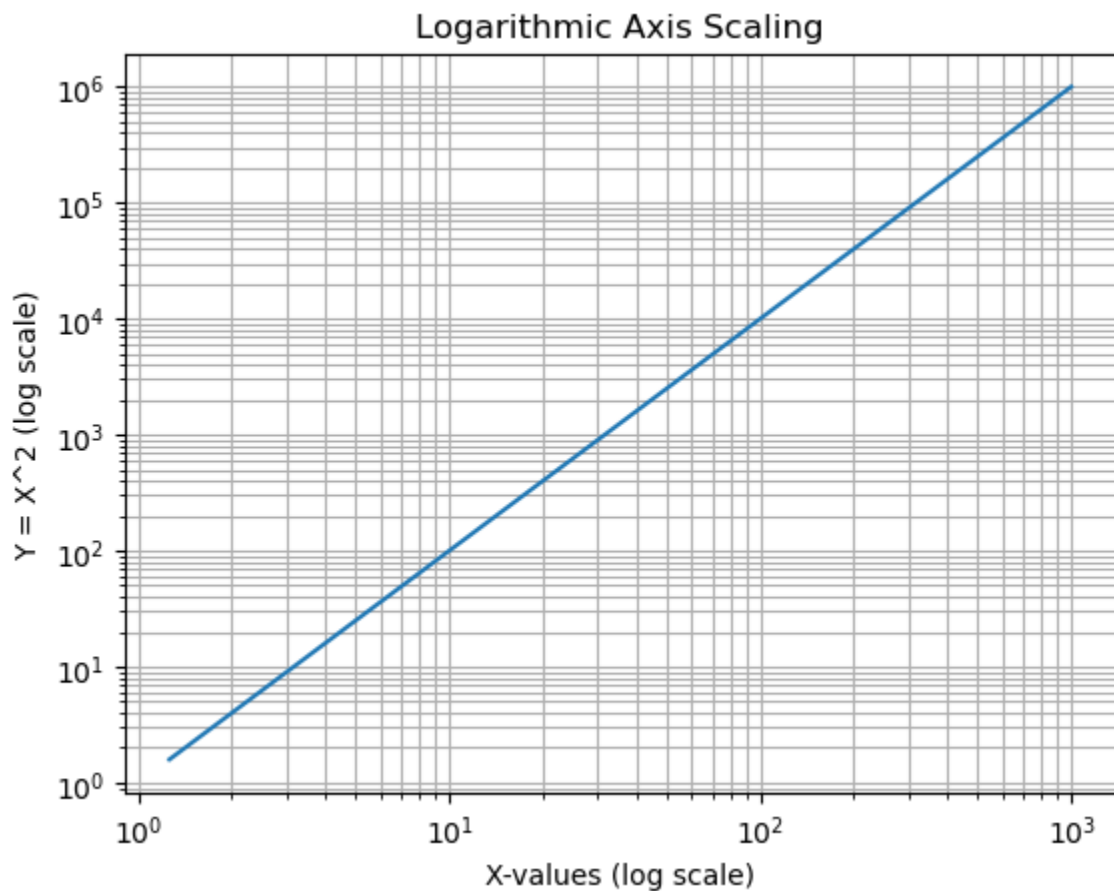
In [37]:

```
x = np.logspace(0.1, 3, 20) # Create logarithmically spaced x-values
y = x**2

plt.plot(x, y)
plt.xlabel('X-values (log scale)')
plt.ylabel('Y = X^2 (log scale)')
plt.title('Logarithmic Axis Scaling')

# Set both axes to logarithmic scale
plt.xscale('log')
plt.yscale('log')

plt.grid(True, which="both", ls="-") # Add grid for better readability on log scale
plt.show()
```



Grid Lines and Annotations

Adding and Customizing Grid Lines:

- Grid lines enhance plot readability by providing visual axis references. Add them using `plt.grid()`.
- `plt.grid(True)` displays default grid lines.
- Customize appearance with keyword arguments: `color`, `linestyle (ls)`, `linewidth (lw)`, and `alpha`.
- The `which` parameter controls which grid lines to display: `'major'`, `'minor'`, or `'both'`.
Note that minor ticks might need to be explicitly set for `'minor'` to be visible.

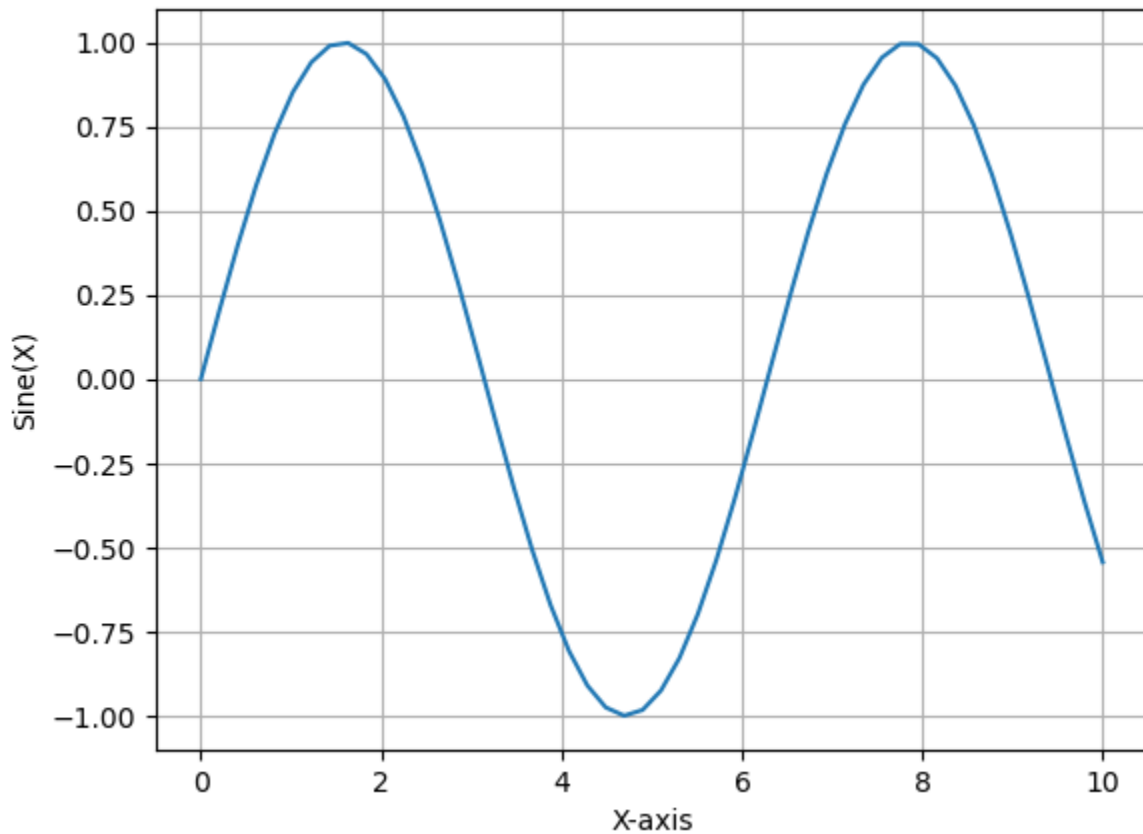
In [38]:

```
x = np.linspace(0, 10, 50)
y = np.sin(x)

plt.plot(x, y)
plt.xlabel('X-axis')
plt.ylabel('Sine(X)')
plt.title('Plot with Grid Lines')

# Add default grid lines
plt.grid(True)
plt.show()
```

Plot with Grid Lines



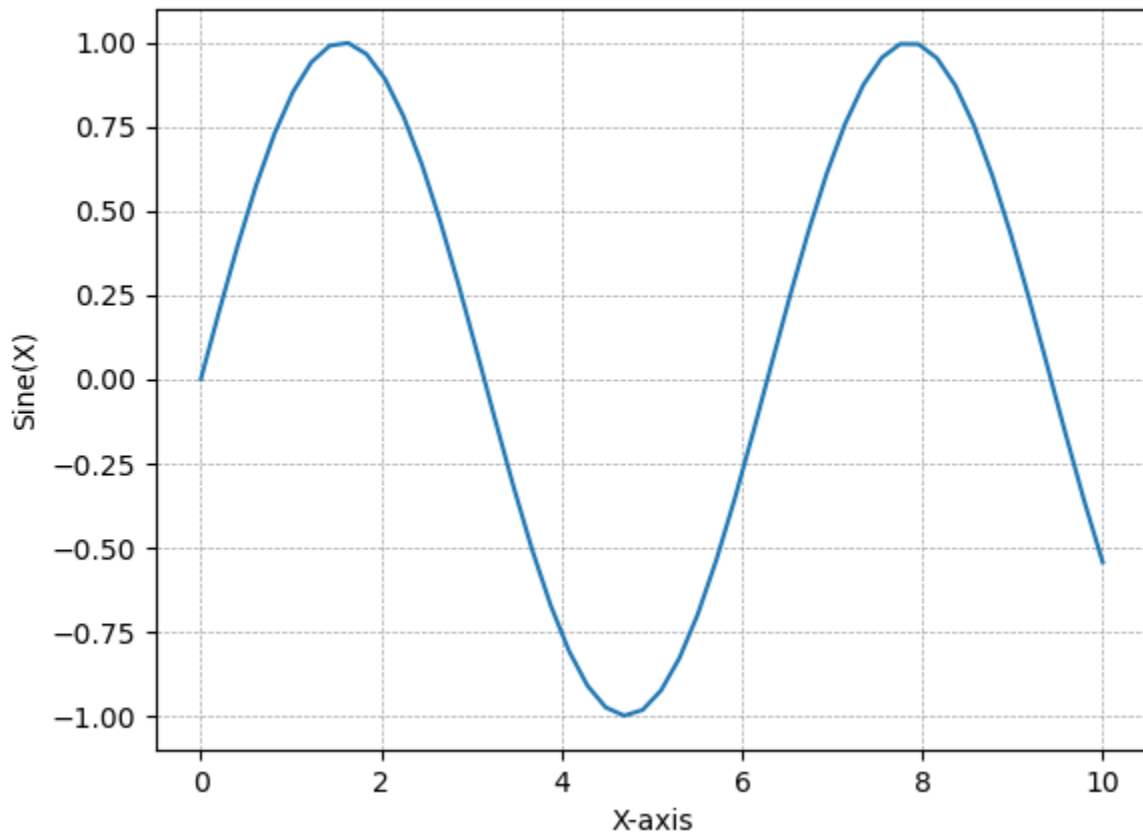
In [39]:

```
x = np.linspace(0, 10, 50)
y = np.sin(x)

plt.plot(x, y)
plt.xlabel('X-axis')
plt.ylabel('Sine(X)')
plt.title('Customized Grid Lines')

# Customize grid lines
plt.grid(True, color='gray', linestyle='--', linewidth=0.5, alpha=0.7)
plt.show()
```

Customized Grid Lines



Adding Text Annotations:

- Use `plt.text(x, y, 'text')` to add text at a specific (x, y) location on the plot (default coordinates are in data space).
- Customize text appearance using keyword arguments like `fontsize`, `color`, `fontweight`, `ha` (horizontal alignment), `va` (vertical alignment), and `bbox` (to add a box around the text).

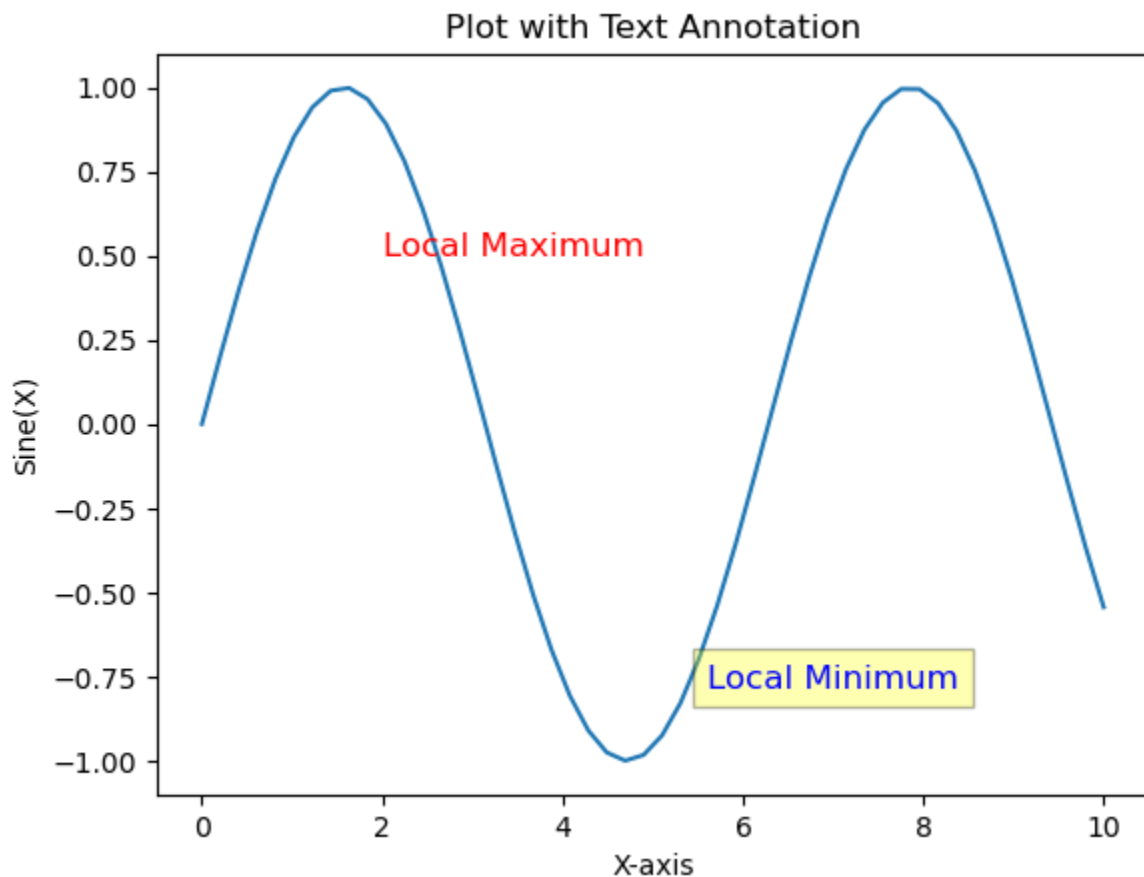
In [40]:

```
x = np.linspace(0, 10, 50)
y = np.sin(x)

plt.plot(x, y)
plt.xlabel('X-axis')
plt.ylabel('Sine(X)')
plt.title('Plot with Text Annotation')

# Add text annotation
plt.text(2, 0.5, 'Local Maximum', fontsize=12, color='red')
plt.text(7, -0.8, 'Local Minimum', fontsize=12, color='blue', ha='center', va='bottom',
        bbox={'facecolor': 'yellow', 'alpha': 0.3, 'pad': 5})

plt.show()
```



Adding Arrows and Other Annotations:

- The `plt.annotate('text', xy=(x_point, y_point), xytext=(x_text, y_text), arrowprops=dict(...))` function adds text with an optional arrow.
- **xy** : Coordinates of the annotated point (arrow target).
- **xytext** : Coordinates of the text placement (defaults to `xy`).
- **arrowprops** : A dictionary defining arrow properties like `arrowstyle`, `color`, `linewidth`, `shrink`, and `facecolor`.
- You can customize the `arrowprops` dictionary extensively to create various arrow styles. Some common `arrowstyle` options include:
 - `'->'` (standard arrow)
 - `'|-|'`
 - `'-|>'`
 - `'<->'`
 - `'fancy'`
 - `'simple'`
 - `'wedge'`

In [41]:

```
x = np.linspace(0, 10, 50)
y = np.sin(x)

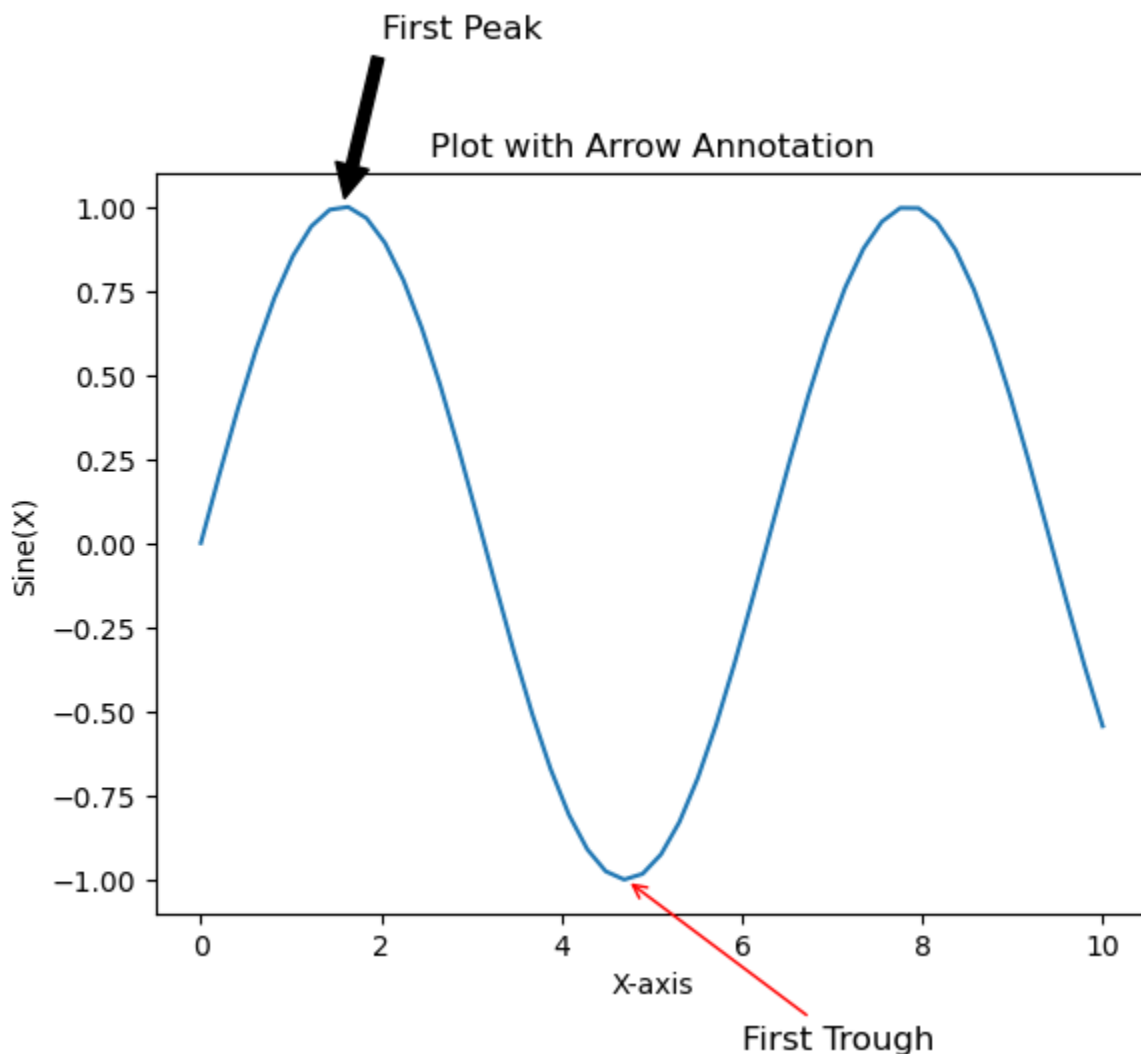
plt.plot(x, y)
plt.xlabel('X-axis')
```

```
plt.ylabel('Sine(X)')
plt.title('Plot with Arrow Annotation')

# Add an arrow annotation
plt.annotate('First Peak', xy=(np.pi/2, 1), xytext=(2, 1.5),
            arrowprops=dict(facecolor='black', shrink=0.05),
            fontsize=12)

plt.annotate('First Trough', xy=(3*np.pi/2, -1), xytext=(6, -1.5),
            arrowprops=dict(arrowstyle='->', color='red', linewidth=1),
            fontsize=12)

plt.show()
```



Stylesheets

Using Predefined Matplotlib Stylesheets:

- Matplotlib offers predefined stylesheets to control plot appearance (colors, fonts, line styles, etc.).
- Apply a stylesheet using `plt.style.use('stylesheet_name')`.

In [42]:

```
# Sample data
x = np.linspace(0, 5, 50)
```

```

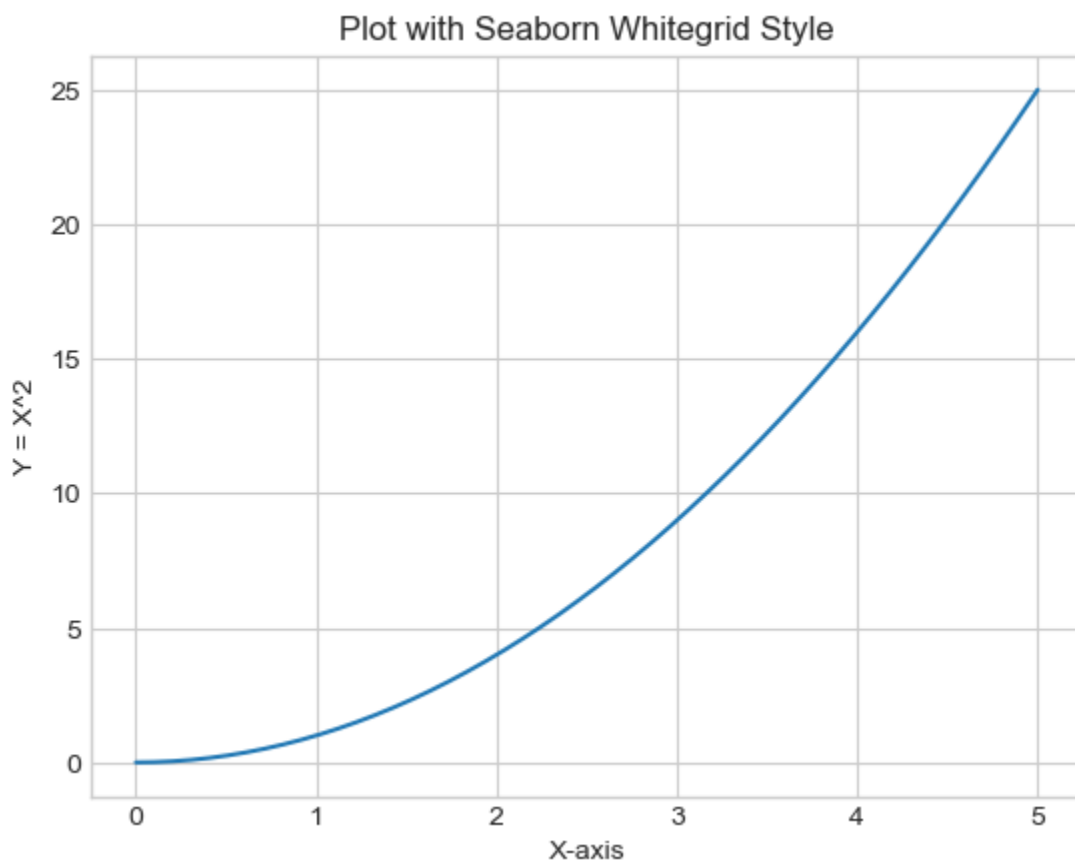
y = x**2

# Apply a stylesheet
plt.style.use('seaborn-v0_8-whitegrid') # Try different stylesheets

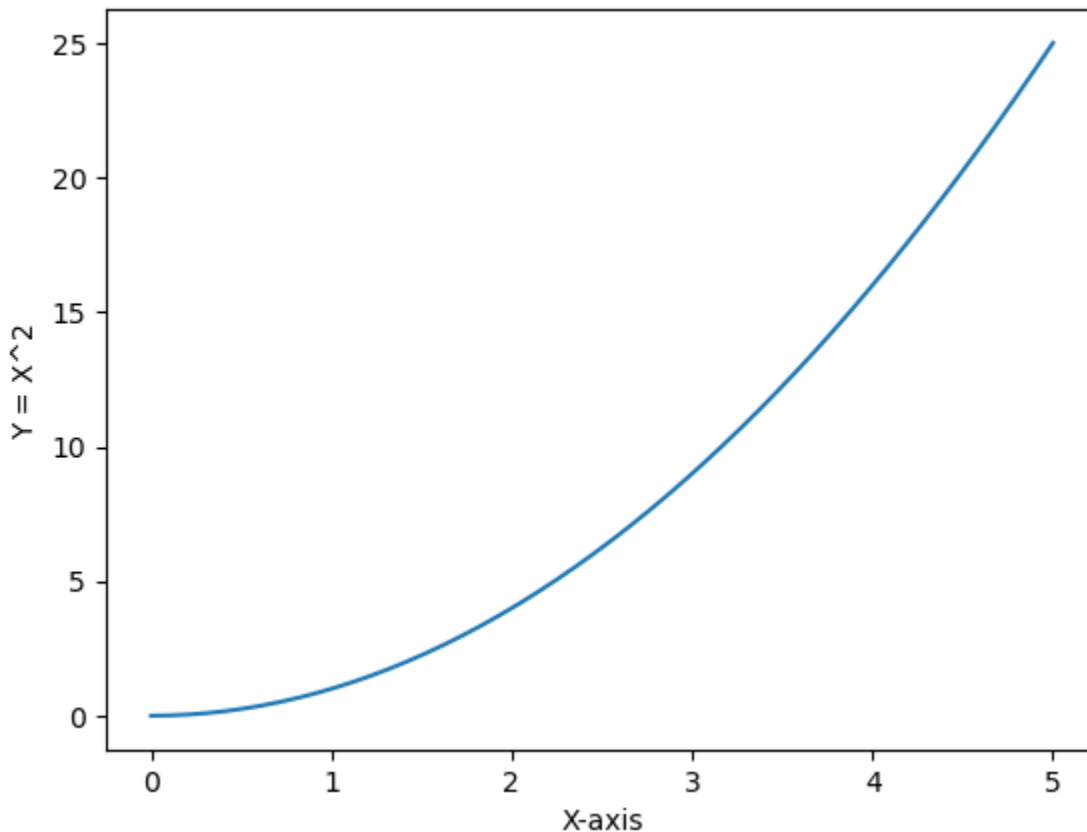
plt.plot(x, y)
plt.xlabel('X-axis')
plt.ylabel('Y = X^2')
plt.title('Plot with Seaborn Whitegrid Style')
plt.show()

# Revert to the default style (optional)
plt.style.use('default')
plt.plot(x, y)
plt.xlabel('X-axis')
plt.ylabel('Y = X^2')
plt.title('Plot with Default Style')
plt.show()

```



Plot with Default Style



Exploring Available Stylesheets:

- You can get a list of all available predefined stylesheets using `plt.style.available`.

In [43]:

```
print(plt.style.available)
```

```
['Solarize_Light2', '_classic_test_patch', '_mpl-gallery', '_mpl-gallery-nogrid', 'bmh', 'classic', 'dark_background', 'fast', 'fivethirtyeight', 'ggplot', 'grayscale', 'seaborn-v0_8', 'seaborn-v0_8-bright', 'seaborn-v0_8-colorblind', 'seaborn-v0_8-dark', 'seaborn-v0_8-dark-palette', 'seaborn-v0_8-darkgrid', 'seaborn-v0_8-deep', 'seaborn-v0_8-muted', 'seaborn-v0_8-notebook', 'seaborn-v0_8-paper', 'seaborn-v0_8-pastel', 'seaborn-v0_8-poster', 'seaborn-v0_8-talk', 'seaborn-v0_8-ticks', 'seaborn-v0_8-white', 'seaborn-v0_8-whitegrid', 'tableau-colorblind10']
```

Advanced Plotting Techniques

3D Plotting

Creating 3D Axes:

- To create a 3D plot, start with a standard Matplotlib Figure.
- Add a subplot with the `projection` argument set to `'3d'`:

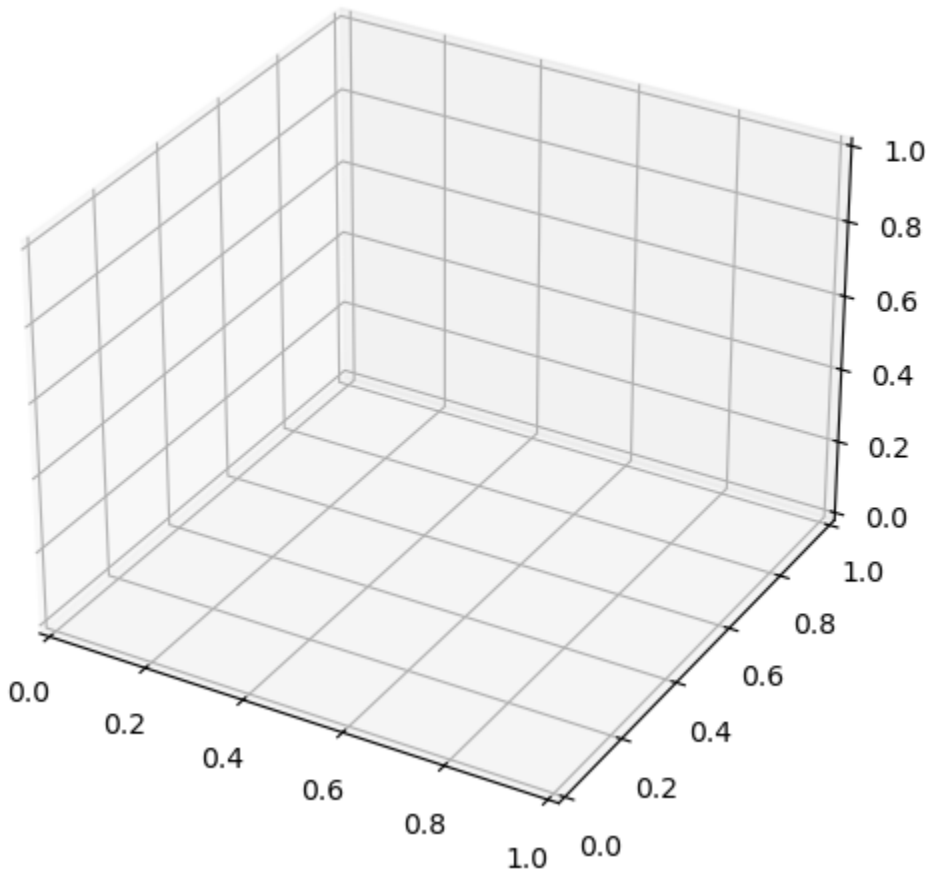
```
fig = plt.figure()  
ax = fig.add_subplot(projection='3d')
```

In [47]:

```
# Create a new figure
fig = plt.figure(figsize=(8, 6))

# Add a 3D subplot
ax = fig.add_subplot(projection='3d')

# You can now plot 3D data on this 'ax' object
plt.show()
```



Plotting Lines in 3D:

- Use the `ax.plot3D(x, y, z, ...)` method to plot lines in 3D.
- It requires three arguments: x, y, and z coordinates.
- You can apply styling options similar to `plt.plot()` (e.g., `color`, `linestyle`, `linewidth`, `marker`).

In [48]:

```
# Create a new figure and a 3D subplot
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(projection='3d')

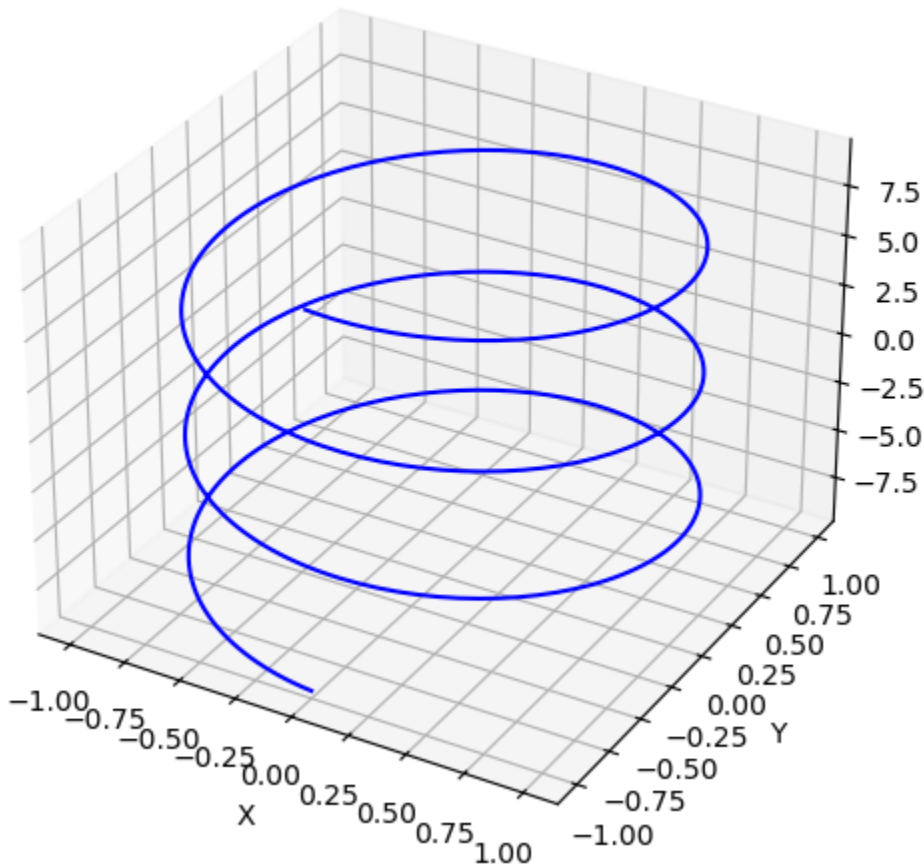
# Sample 3D data: a helix
t = np.linspace(-3 * np.pi, 3 * np.pi, 500)
x = np.sin(t)
y = np.cos(t)
z = t
```

```
# Plot the helix
ax.plot3D(x, y, z, 'blue')

# Set labels for the axes
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')
ax.set_title('3D Line Plot: Helix')

plt.show()
```

3D Line Plot: Helix



Plotting Scatter Plots in 3D:

- Use the `ax.scatter3D(x, y, z, ...)` method to plot individual points in 3D.
- It also takes x, y, and z coordinates as arguments.
- Customize markers using parameters like `s` (size), `c` (color, single or array), `marker`, and `alpha`.

In [49]:

```
# Create a new figure and a 3D subplot
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(projection='3d')

# Sample 3D scatter data
np.random.seed(42)
n = 100
```

```

x = np.random.randn(n)
y = np.random.randn(n)
z = np.random.randn(n)
colors = np.random.rand(n)
sizes = 50 * np.random.rand(n)

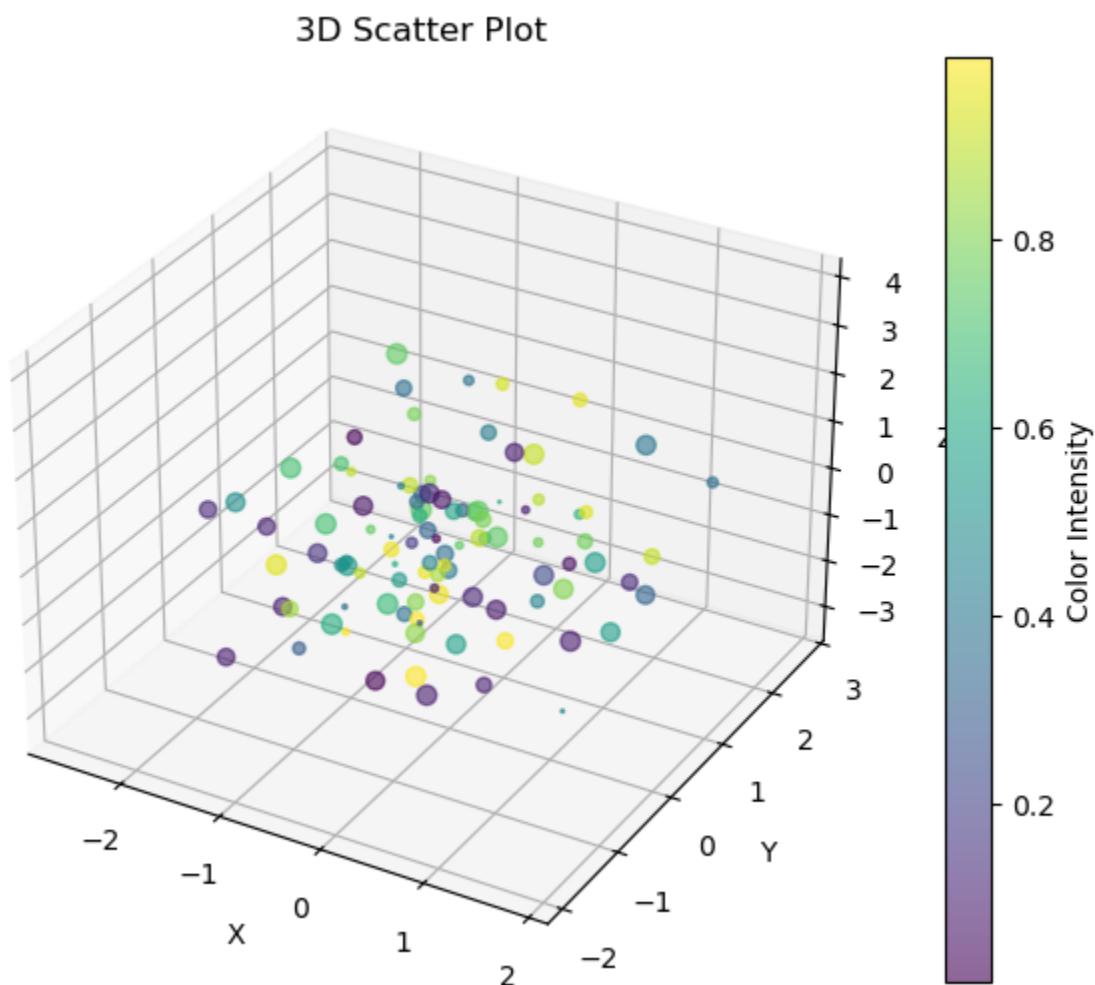
# Plot the 3D scatter
scatter = ax.scatter3D(x, y, z, c=colors, s=sizes, cmap='viridis', alpha=0.6)

# Add a colorbar
fig.colorbar(scatter, label='Color Intensity')

# Set labels
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')
ax.set_title('3D Scatter Plot')

plt.show()

```



Plotting Surfaces in 3D:

- Visualize 3D surfaces using 2D arrays for x, y coordinates and corresponding z values (height).
- Use methods like `ax.plot_surface()` or `ax.plot_wireframe()`.
 - `ax.plot_surface(X, Y, Z, **kwargs)` : Creates a filled surface plot.
 - `ax.plot_wireframe(X, Y, Z, **kwargs)` : Creates a wireframe representation.

In [53]:

```
# Create a new figure and a 3D subplot
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(projection='3d')

# Sample data for a surface
num1 = np.arange(-5, 5, 0.25)
num2 = np.arange(-5, 5, 0.25)
X, Y = np.meshgrid(num1, num2)
R = np.sqrt(X**2 + Y**2)
Z = np.sin(R)

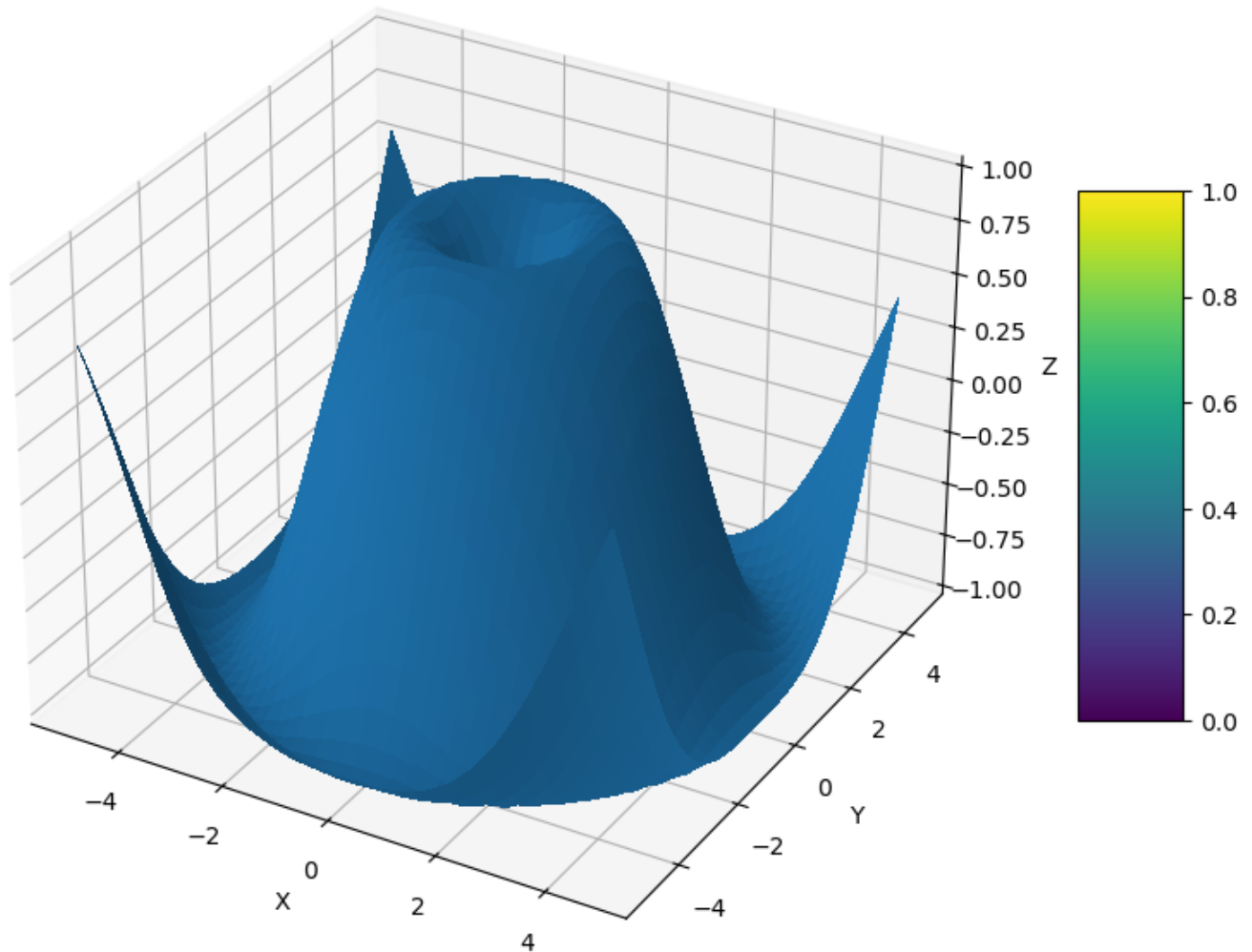
# Plot the surface
surf = ax.plot_surface(X, Y, Z, linewidth=0, antialiased=False)

# Add a colorbar
fig.colorbar(surf, shrink=0.5, aspect=5)

# Set labels
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')
ax.set_title('3D Surface Plot')

plt.show()
```

3D Surface Plot



In [51]:

```
import matplotlib.pyplot as plt
import numpy as np

# Create a new figure and a 3D subplot
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(projection='3d')

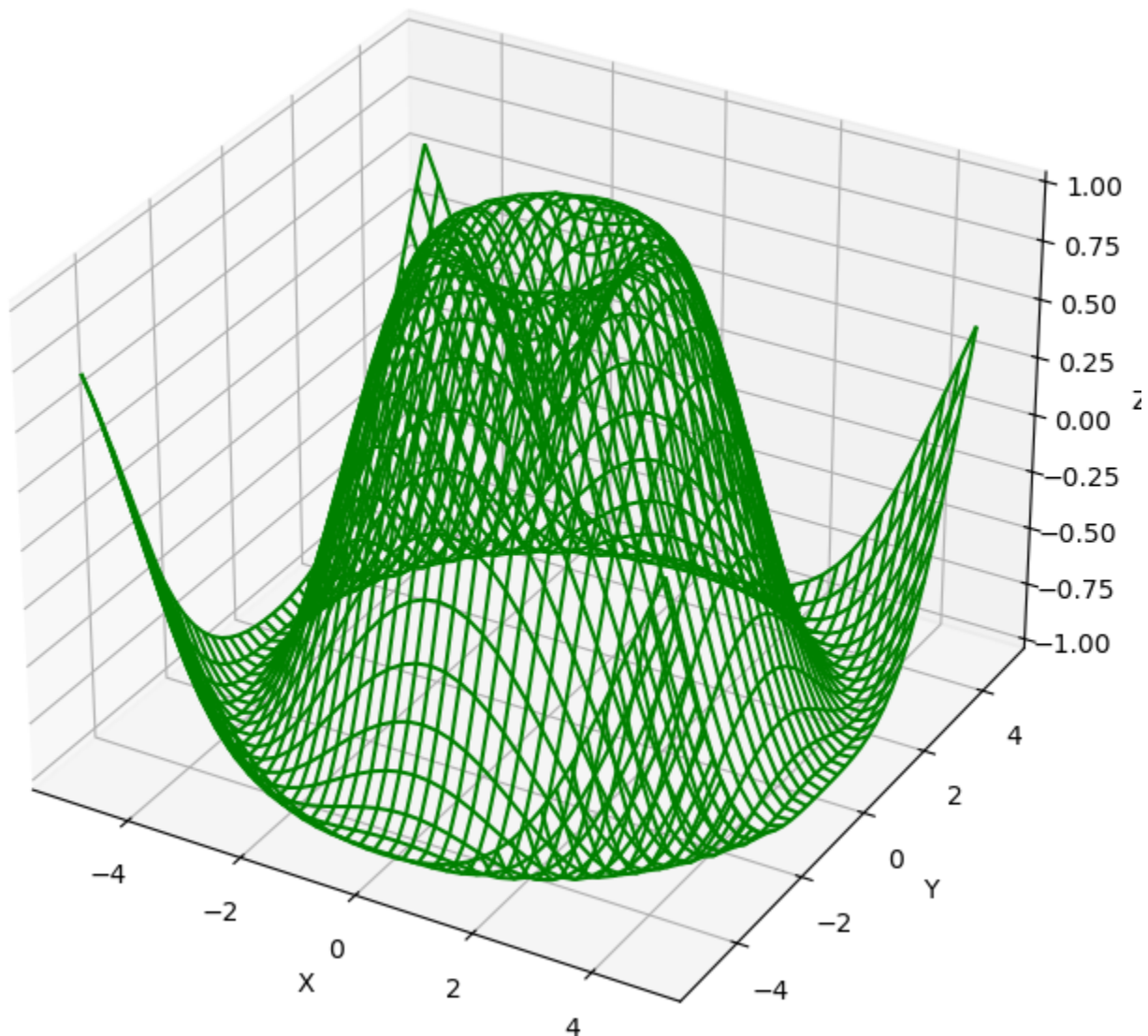
# Sample data for a wireframe
num1 = np.arange(-5, 5, 0.25)
num2 = np.arange(-5, 5, 0.25)
X, Y = np.meshgrid(num1, num2)
R = np.sqrt(X**2 + Y**2)
Z = np.sin(R)

# Plot the wireframe
ax.plot_wireframe(X, Y, Z, color='green')

# Set labels
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')
ax.set_title('3D Wireframe Plot')
```

```
plt.show()
```

3D Wireframe Plot



Contour Plots

Visualizing 3D Data in 2D using `plt.contour()`:

- The `plt.contour(X, Y, Z, levels=None, **kwargs)` function creates contour lines (isolines) of a 3D surface projected onto a 2D plane.
- **X and Y**: 2D arrays of the x and y coordinates.
- **Z**: 2D array of z values (surface height) corresponding to the (X, Y) grid.
- **levels**: Optional argument specifying the z-values for the contour lines. If omitted, Matplotlib chooses automatically.

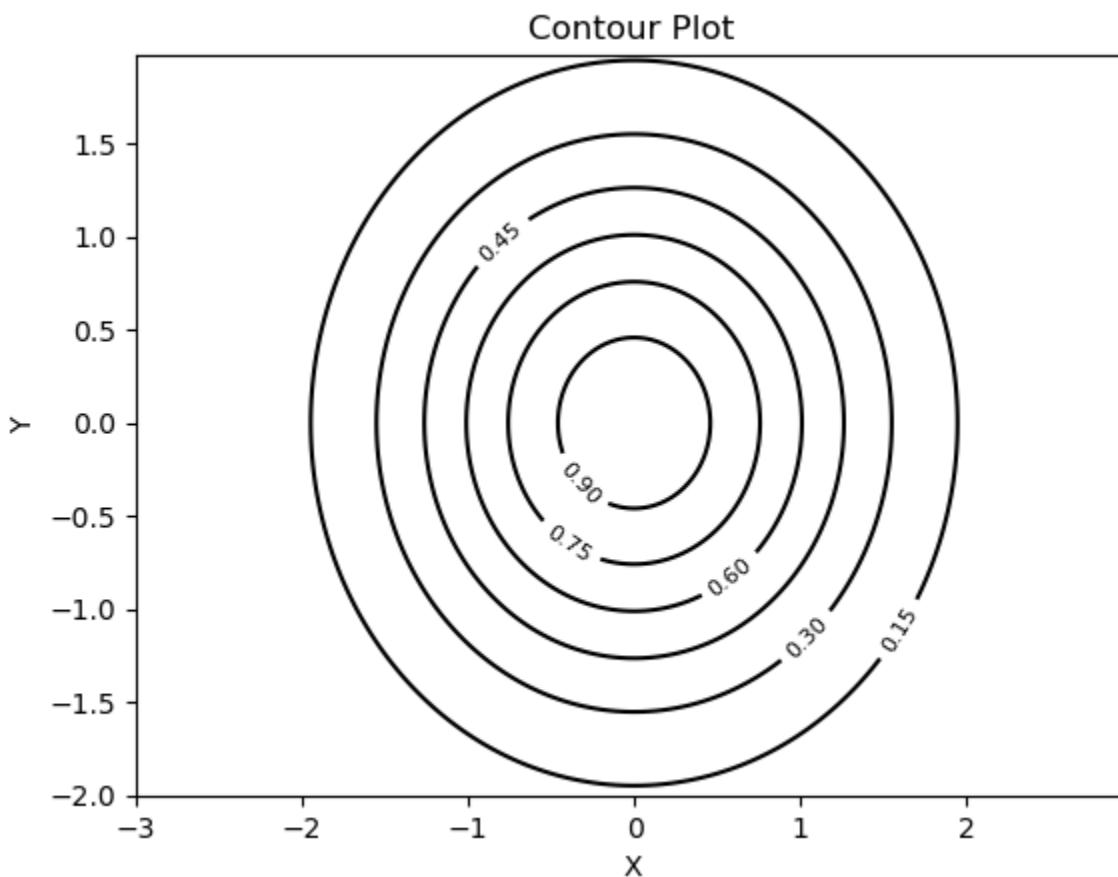
In [54]:

```
# Sample 3D data
delta = 0.025
x = np.arange(-3.0, 3.0, delta)
y = np.arange(-2.0, 2.0, delta)
X, Y = np.meshgrid(x, y)
Z = np.exp(-(X**2 + Y**2) / 2)

# Create a contour plot
CS = plt.contour(X, Y, Z, 6, colors='k') # 6 contour lines, black color

# Add labels to the contour lines
plt.clabel(CS, inline=True, fontsize=8)

plt.title('Contour Plot')
plt.xlabel('X')
plt.ylabel('Y')
plt.show()
```



Visualizing 3D Data in 2D using `plt.contourf()` :

- The `plt.contourf(X, Y, Z, levels=None, cmap=None, **kwargs)` function creates filled contour plots. The areas between contour lines are colored.
- Arguments are similar to `plt.contour()` .
- **cmap** : Optional argument to specify the colormap for filling the contours.

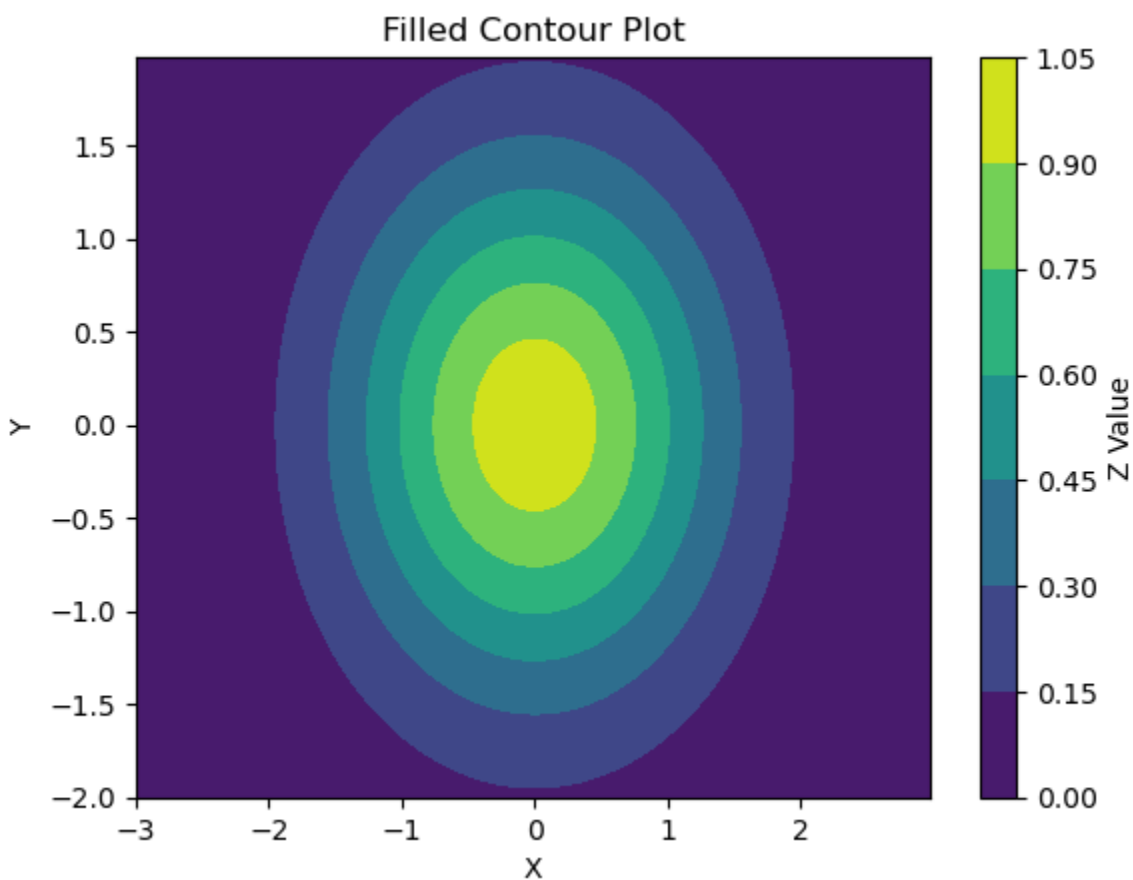
In [55]:


```
# Sample 3D data (same as above)
delta = 0.025
x = np.arange(-3.0, 3.0, delta)
y = np.arange(-2.0, 2.0, delta)
X, Y = np.meshgrid(x, y)
Z = np.exp(-(X**2 + Y**2) / 2)

# Create a filled contour plot
CF = plt.contourf(X, Y, Z, 6, cmap='viridis') # 6 filled levels, using 'viridis' colormap

# Add a colorbar to show the color mapping
plt.colorbar(CF, label='Z Value')

plt.title('Filled Contour Plot')
plt.xlabel('X')
plt.ylabel('Y')
plt.show()
```



Customizing Contour Lines:

You can customize the appearance of contour lines in `plt.contour()` using these keyword arguments:

- **colors** : Specifies the color(s) of the lines (e.g., `'red'`, `['red', 'blue', 'green']`).
- **linewidths** : Specifies the width(s) of the lines.
- **linestyles** : Specifies the style(s) of the lines (e.g., `'--'`, `['-', '--', ':']`).

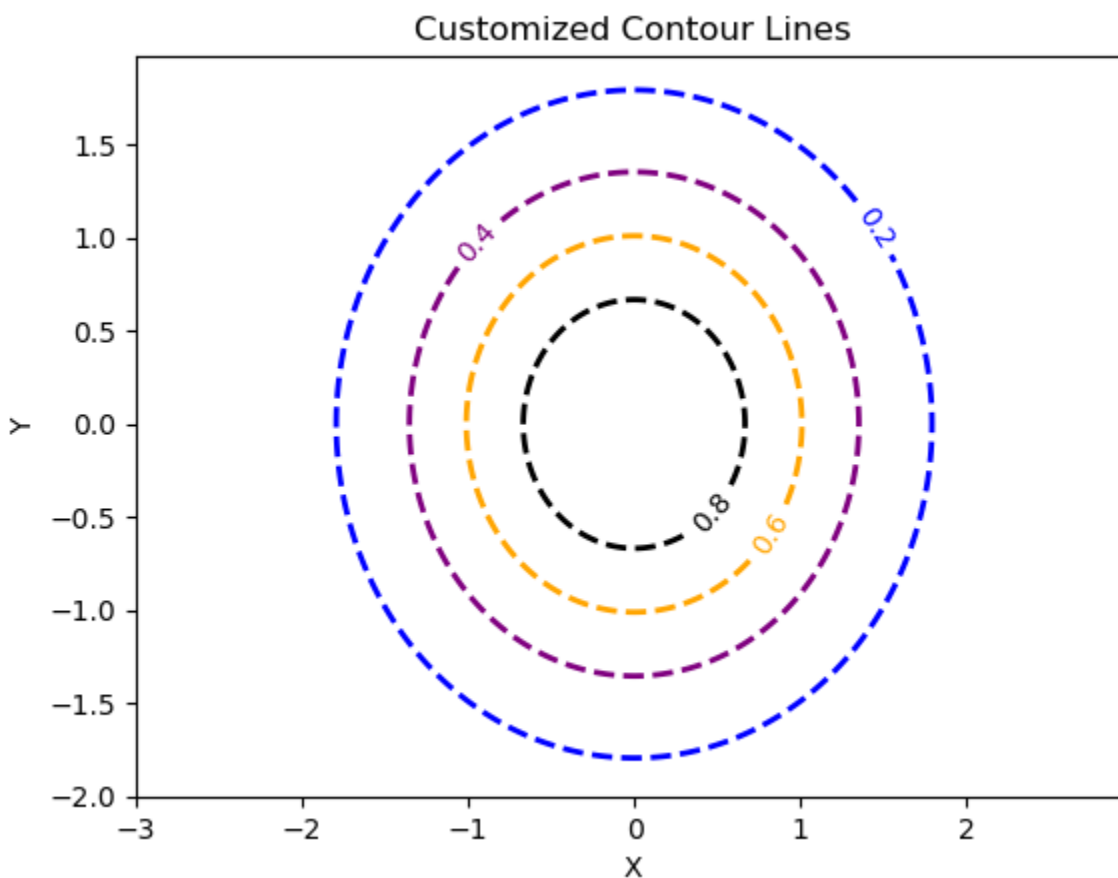
In [56]:

```
# Sample 3D data
delta = 0.025
x = np.arange(-3.0, 3.0, delta)
y = np.arange(-2.0, 2.0, delta)
X, Y = np.meshgrid(x, y)
Z = np.exp(-(X**2 + Y**2) / 2)

# Create a contour plot with customizations
CS = plt.contour(X, Y, Z, levels=[-0.2, 0, 0.2, 0.4, 0.6, 0.8],
                 colors=['red', 'green', 'blue', 'purple', 'orange', 'black'],
                 linewidths=2, linestyle='--')

# Add labels to the contour lines
plt.clabel(CS, inline=True, fontsize=10)

plt.title('Customized Contour Lines')
plt.xlabel('X')
plt.ylabel('Y')
plt.show()
```



Customizing Filled Areas:

For `plt.contourf()`, you can customize the filled areas using:

- **cmap**: Specifies the colormap for the filled regions.
- **alpha**: Controls the transparency of the filled areas.
- **levels**: Allows you to define the boundaries of the filled regions.

In [57]:

```
# Sample 3D data
delta = 0.025
x = np.arange(-3.0, 3.0, delta)
y = np.arange(-2.0, 2.0, delta)
X, Y = np.meshgrid(x, y)
Z = np.exp(-(X**2 + Y**2) / 2)

# Create a filled contour plot with customizations
CF = plt.contourf(X, Y, Z, levels=np.linspace(Z.min(), Z.max(), 10),
                  cmap='plasma', alpha=0.7)

# Add a colorbar
plt.colorbar(CF, label='Z Value')

plt.title('Customized Filled Contour Plot')
plt.xlabel('X')
plt.ylabel('Y')
plt.show()
```

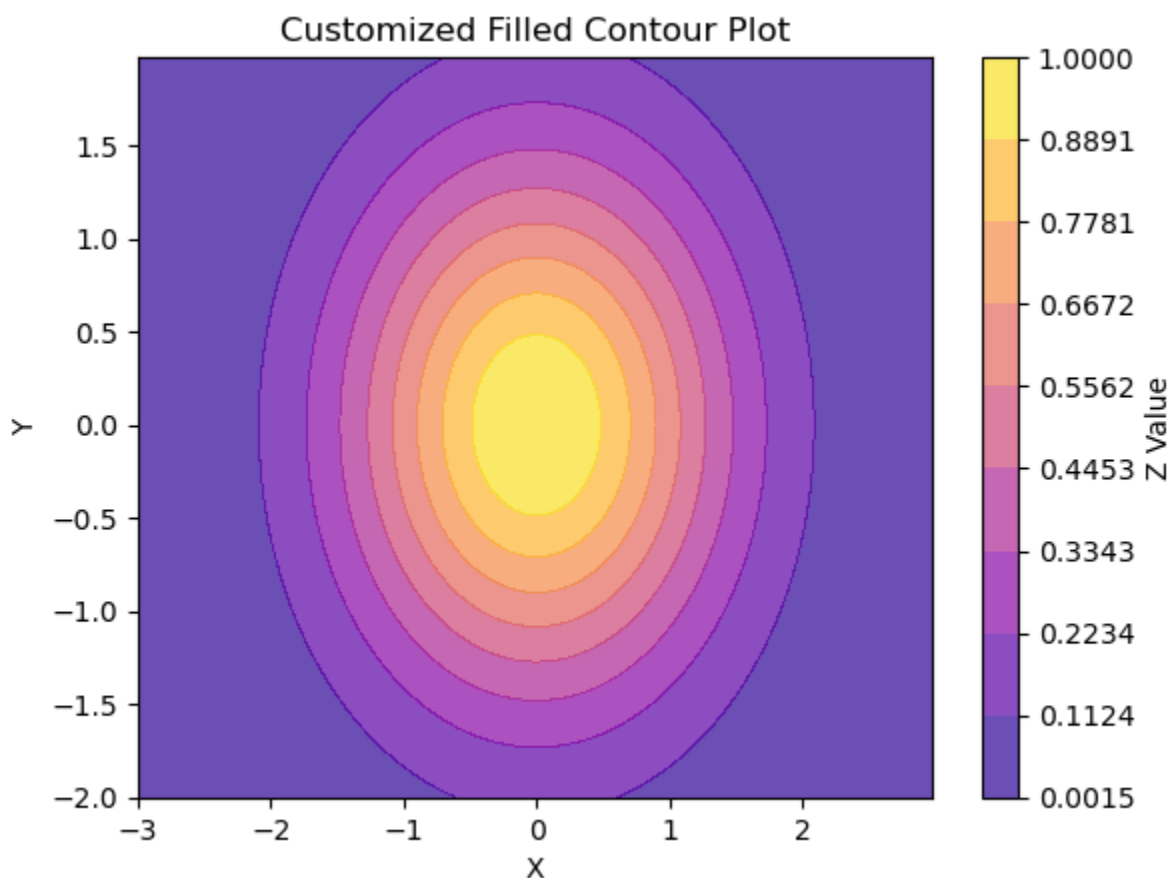


Image Display

Displaying Images using `plt.imshow()`:

- The `plt.imshow(X, cmap=None, aspect=None, interpolation=None, origin=None, extent=None, **kwargs)` function displays an array as an image.
- **X**: The image data.
 - 2D array: Grayscale image (values represent intensity).

- 3D array: RGB or RGBA image (third dimension for color channels).
- **cmap** : Colormap for grayscale images. Not typically used for RGB/RGBA.
- **aspect** : Controls the image's aspect ratio ('auto' , 'equal').
- **interpolation** : Algorithm for resizing ('nearest' , 'bilinear' , 'bicubic' , etc.).
- **origin** : Sets the image origin ('upper' (default), 'lower').
- **extent** : Specifies image coordinates in data space (left , right , bottom , top).

In [58]:

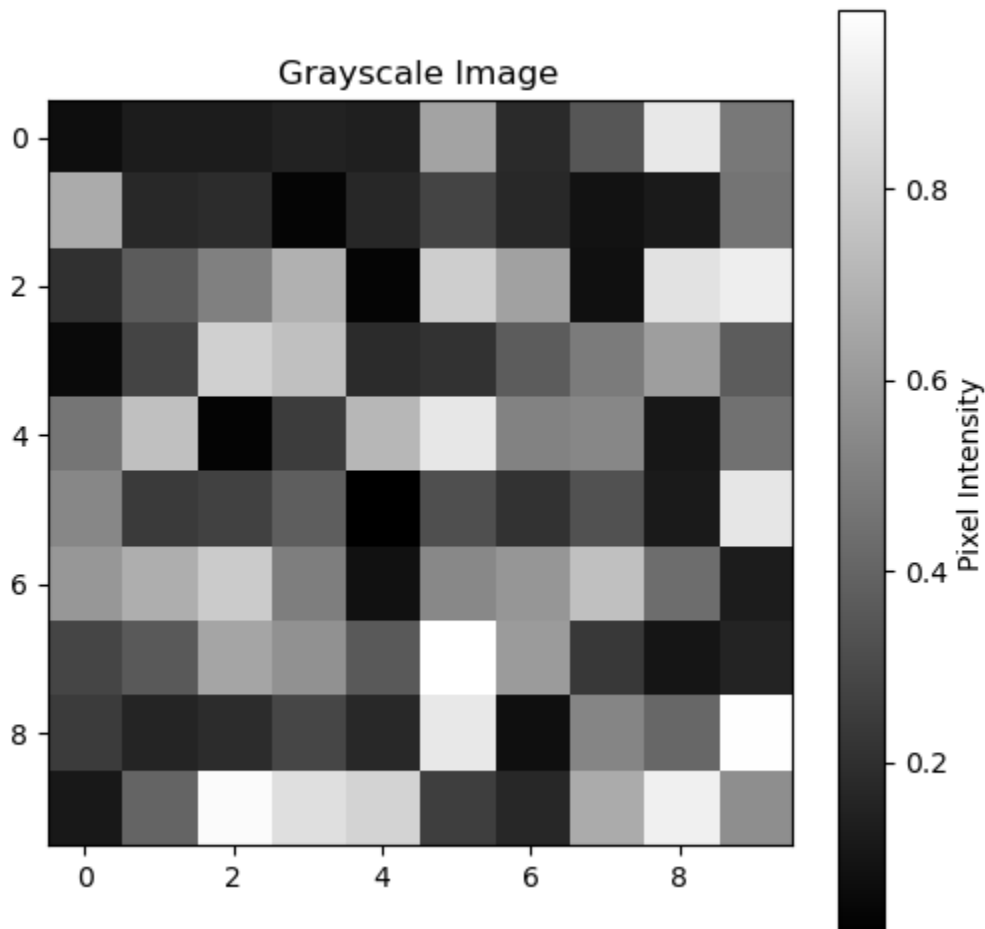
```
import matplotlib.pyplot as plt
import numpy as np

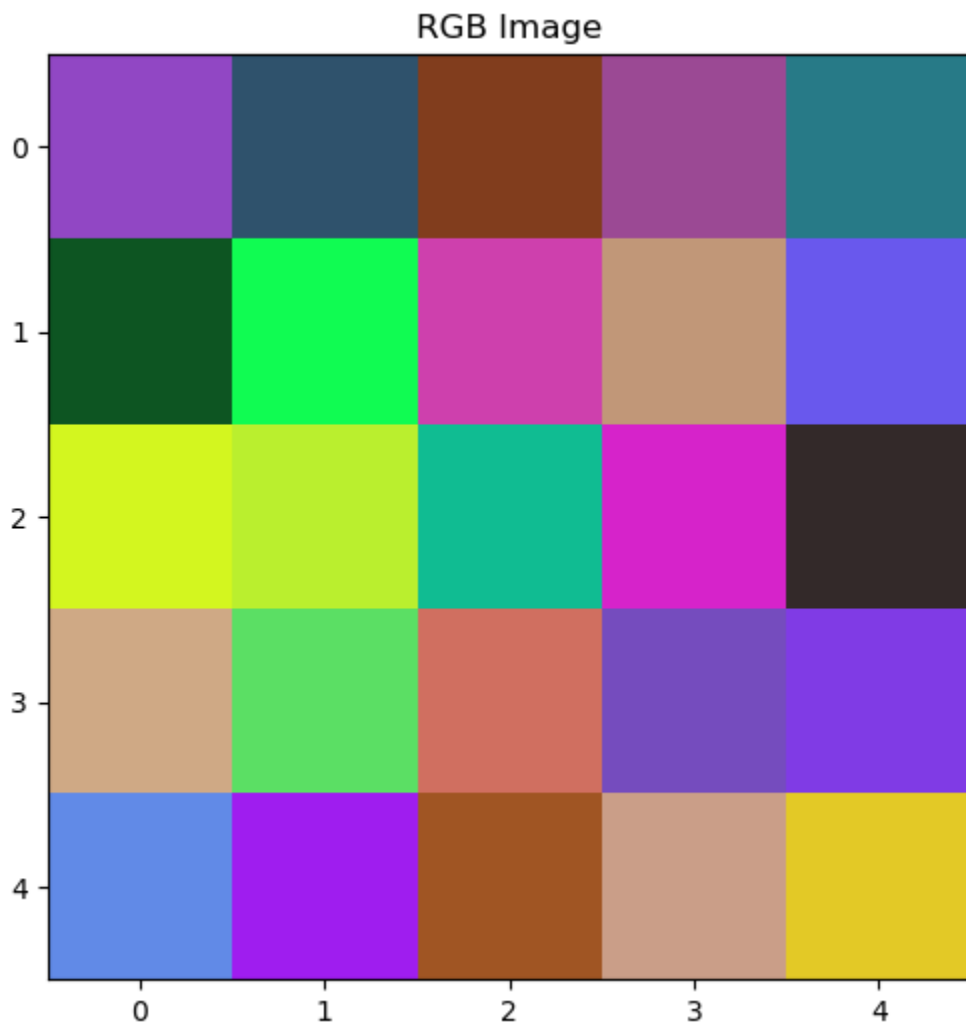
# Sample grayscale image data (10x10 array of random values)
grayscale_image = np.random.rand(10, 10)

plt.figure(figsize=(6, 6))
plt.imshow(grayscale_image, cmap='gray')
plt.colorbar(label='Pixel Intensity')
plt.title('Grayscale Image')
plt.show()

# Sample RGB image data (5x5x3 array of random values)
rgb_image = np.random.rand(5, 5, 3)

plt.figure(figsize=(6, 6))
plt.imshow(rgb_image)
plt.title('RGB Image')
plt.show()
```





Customizing Colormaps:

- For grayscale images displayed with `plt.imshow()`, the `cmap` parameter is crucial for visual interpretation.
- Matplotlib provides a wide variety of colormaps to choose from.

In [59]:

```
import matplotlib.pyplot as plt
import numpy as np

# Sample grayscale image data
grayscale_image = np.random.rand(10, 10)

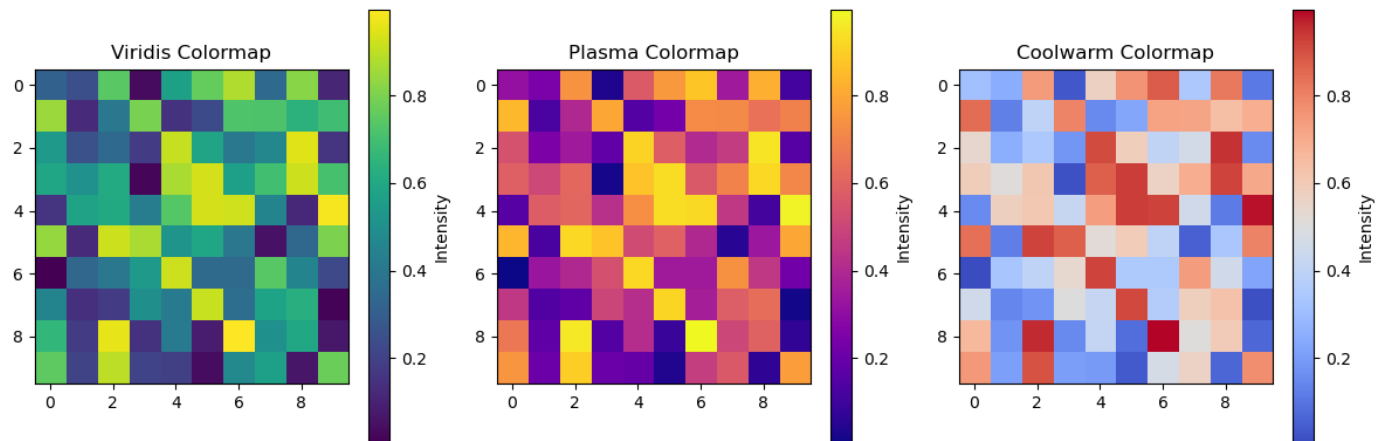
# Display with different colormaps
plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)
plt.imshow(grayscale_image, cmap='viridis')
plt.title('Viridis Colormap')
plt.colorbar(label='Intensity')

plt.subplot(1, 3, 2)
plt.imshow(grayscale_image, cmap='plasma')
plt.title('Plasma Colormap')
plt.colorbar(label='Intensity')
```

```
plt.subplot(1, 3, 3)
plt.imshow( grayscale_image, cmap='coolwarm')
plt.title('Coolwarm Colormap')
plt.colorbar(label='Intensity')

plt.tight_layout()
plt.show()
```



Customizing Interpolation:

- The `interpolation` parameter in `plt.imshow()` determines how pixel values are estimated during image resizing for display.
- Different methods can produce smoother or sharper results.

In [60]:

```
# Sample grayscale image data (small size to see interpolation effects)
small_image = np.array([[0, 5, 10],
                        [15, 20, 25],
                        [30, 35, 40]])

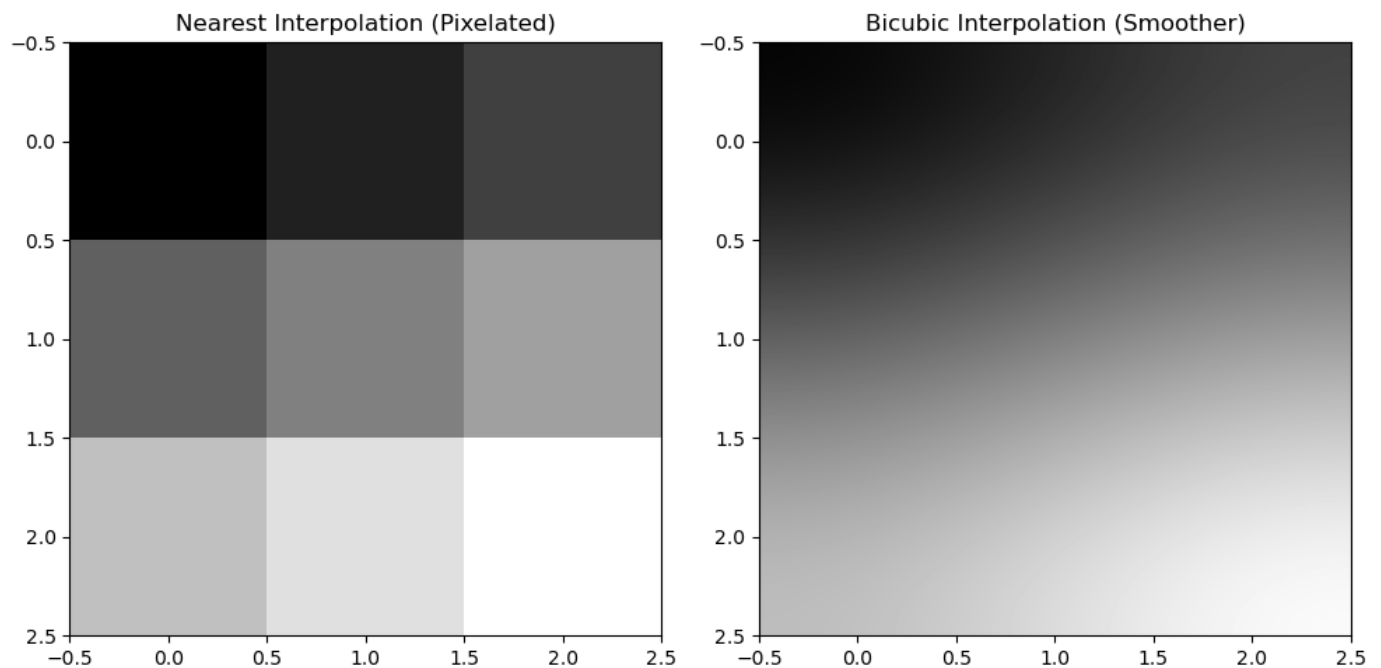
# Display with different interpolation methods
plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.imshow(small_image, cmap='gray', interpolation='nearest')
plt.title('Nearest Interpolation (Pixelated)')

plt.subplot(1, 2, 2)
plt.imshow(small_image, cmap='gray', interpolation='bicubic')
plt.title('Bicubic Interpolation (Smoother)')

plt.tight_layout()
plt.show()

# Explore other interpolation options: 'bilinear', 'gaussian', 'spline16', etc.
```



Animations (Brief Introduction)

Creating Simple Animations using `matplotlib.animation`:

- Matplotlib animations involve updating plot data at intervals and combining frames into a sequence.
- The `matplotlib.animation.FuncAnimation` class is commonly used.
- `FuncAnimation(fig, func, frames=None, init_func=None, fargs=None, save_count=None, kwargs)**`
 - **fig**: The Matplotlib Figure to animate.
 - **func**: Function called for each frame to update plot data. Takes the frame number as the first argument.
 - **frames**: Number of frames or an iterable of frame values. If `None`, animation runs indefinitely.
 - **init_func**: Optional function called once to initialize the plot.
 - **fargs**: Optional tuple of arguments passed to `func` after the frame number.

In [61]:

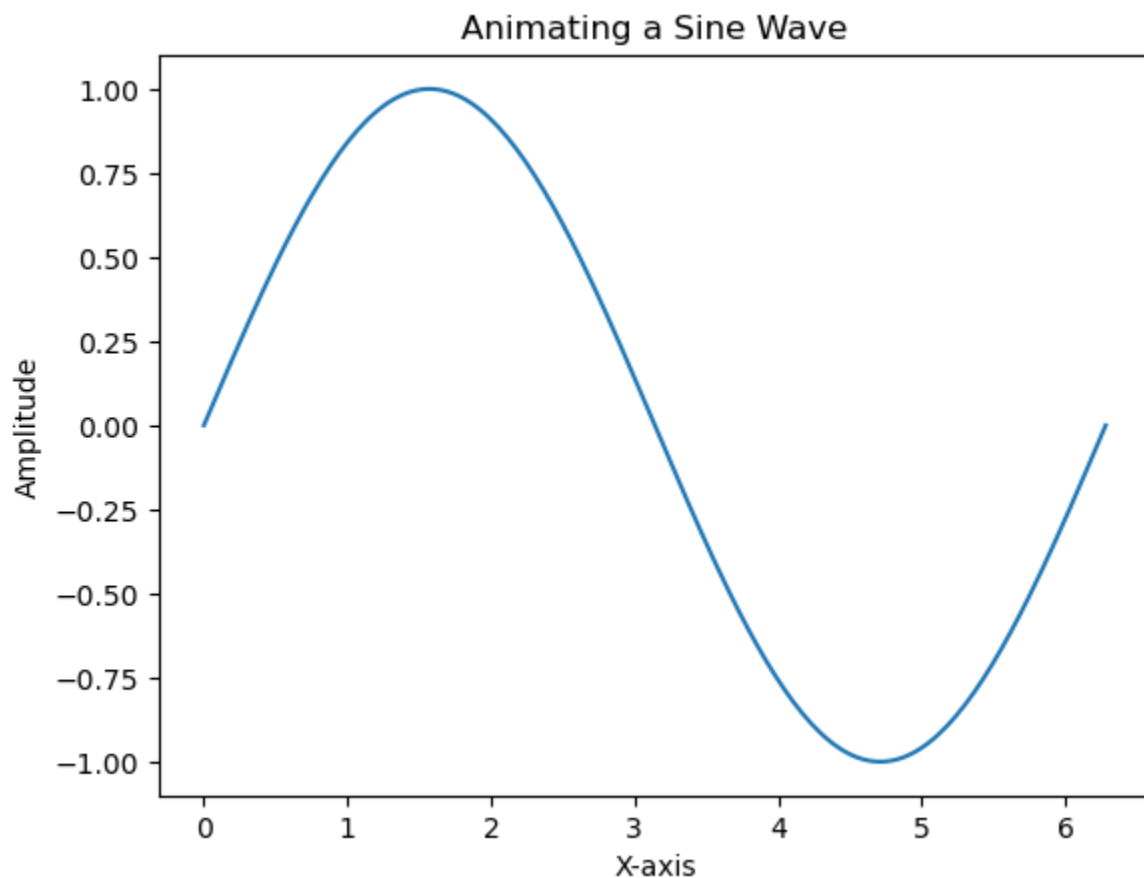
```
import matplotlib.pyplot as plt
import numpy as np
from matplotlib.animation import FuncAnimation

# Create a figure and axes
fig, ax = plt.subplots()
x = np.linspace(0, 2 * np.pi, 200)
line, = ax.plot(x, np.sin(x))

# Define the animation function
def animate(i):
    line.set_ydata(np.sin(x + i / 10.0)) # Update the y-data
    return line,

# Create the animation
ani = FuncAnimation(fig, animate, frames=100, interval=20, blit=True)
```

```
plt.title('Animating a Sine Wave')
plt.xlabel('X-axis')
plt.ylabel('Amplitude')
plt.show()
```



In [62]:

```
# To save the animation (requires a writer like ffmpeg)
ani.save('sine_wave_animation.gif', writer='pillow', fps=30)
```

Working with Different Data Types

Plotting Data from NumPy Arrays:

- NumPy arrays are fundamental for numerical work in Python.
- Matplotlib is designed to work seamlessly and efficiently with NumPy arrays.
- You can directly pass NumPy arrays as input to most Matplotlib plotting functions.

In [63]:

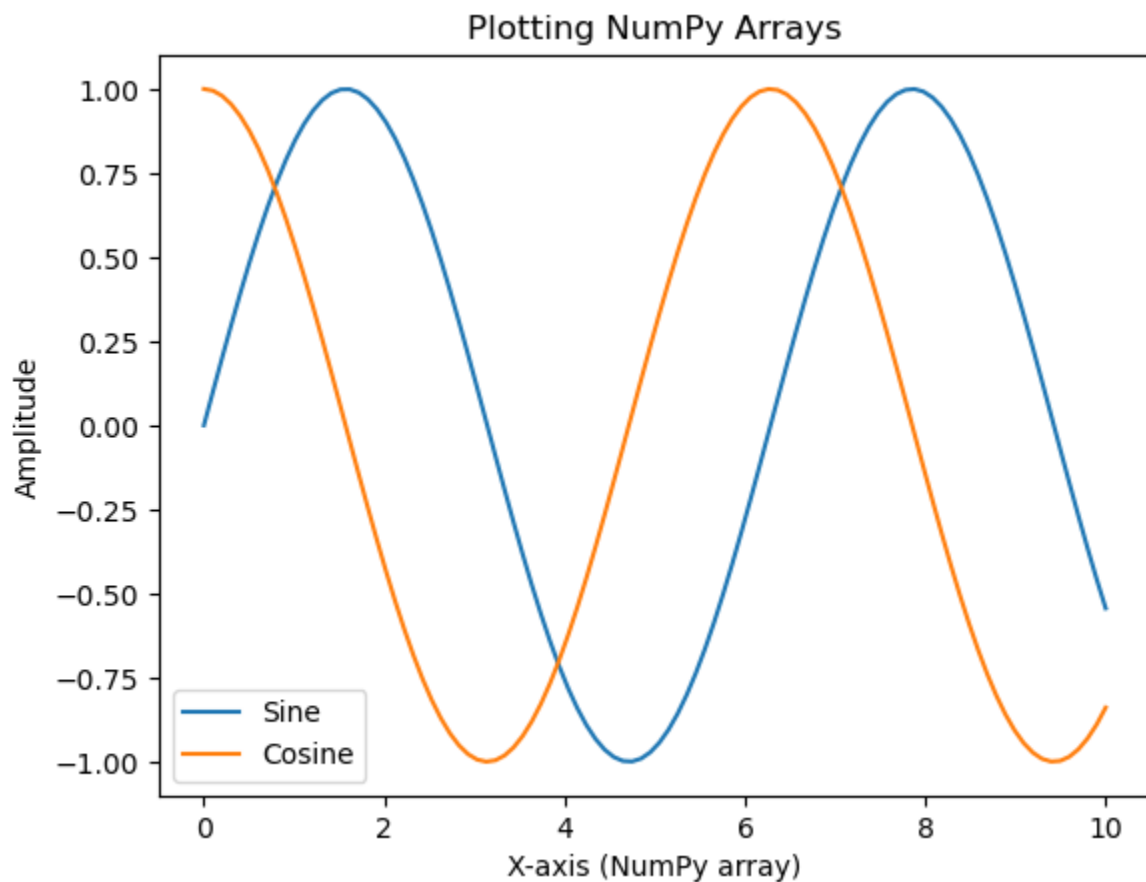
```
# Sample data as NumPy arrays
x = np.linspace(0, 10, 100)
y1 = np.sin(x)
y2 = np.cos(x)

# Directly plot NumPy arrays
plt.plot(x, y1, label='Sine')
plt.plot(x, y2, label='Cosine')

plt.xlabel('X-axis (NumPy array)')
```



```
plt.ylabel('Amplitude')
plt.title('Plotting NumPy Arrays')
plt.legend()
plt.show()
```



Plotting Data from Pandas DataFrames and Series:

- Pandas DataFrames (tabular data) and Series (1D labeled arrays) are powerful data structures.
- Matplotlib can directly plot data from these objects.
- Often, column labels are used as axis labels or legend entries.

In [64]:

```
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

# Sample Pandas DataFrame
data = {'X': np.linspace(0, 10, 50),
        'Y1': np.exp(-0.1 * np.linspace(0, 10, 50)),
        'Y2': np.sin(np.linspace(0, 10, 50))}
df = pd.DataFrame(data)

# Directly plot columns from the DataFrame
plt.plot(df['X'], df['Y1'], label='Exponential Decay')
plt.plot(df['X'], df['Y2'], label='Sine Wave')

plt.xlabel('X-values')
plt.ylabel('Y-values')
plt.title('Plotting from Pandas DataFrame')
plt.legend()
```

```
plt.show()
```

```
# You can also use the .plot() method directly on DataFrames and Series
```

```
df.plot(x='X', y=['Y1', 'Y2'])
```

```
plt.title('Plotting with DataFrame .plot()')
```

```
plt.show()
```

```
# Plotting a Pandas Series
```

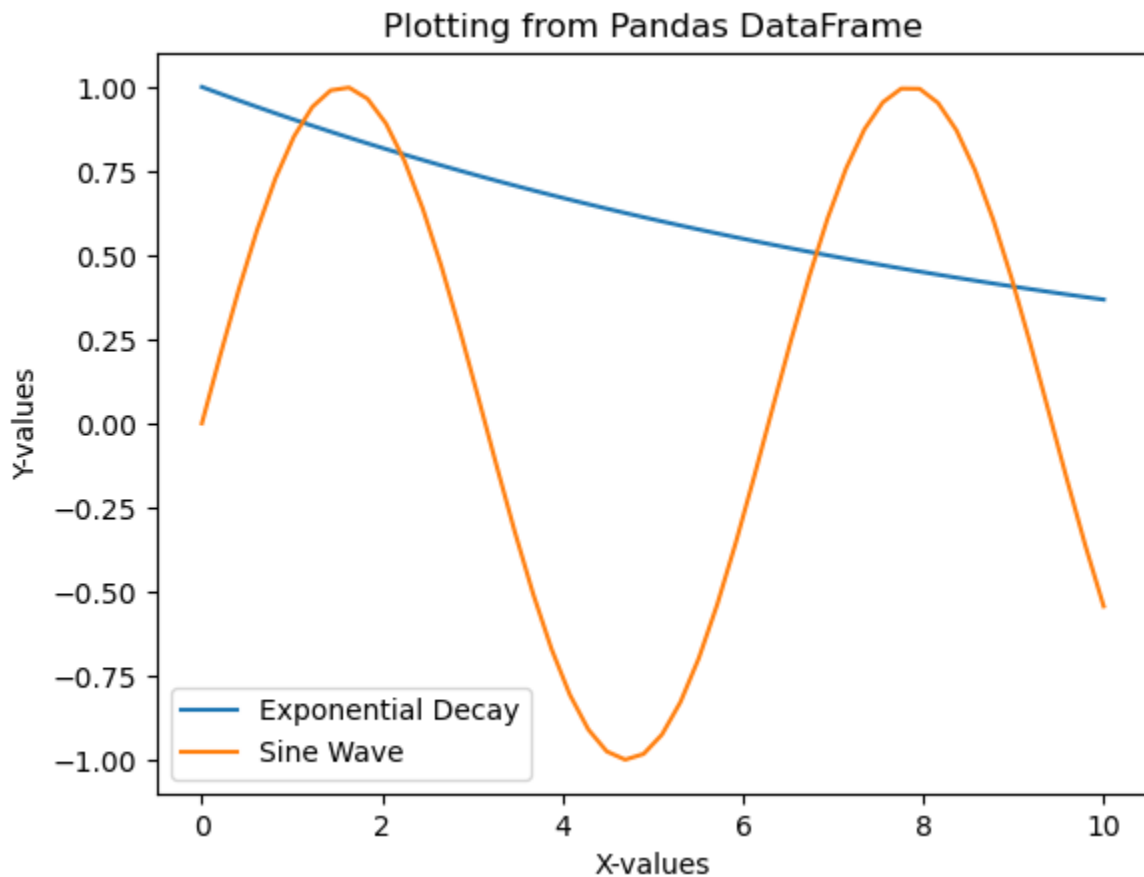
```
series = pd.Series(np.random.randn(100).cumsum())
```

```
series.plot(label='Random Walk')
```

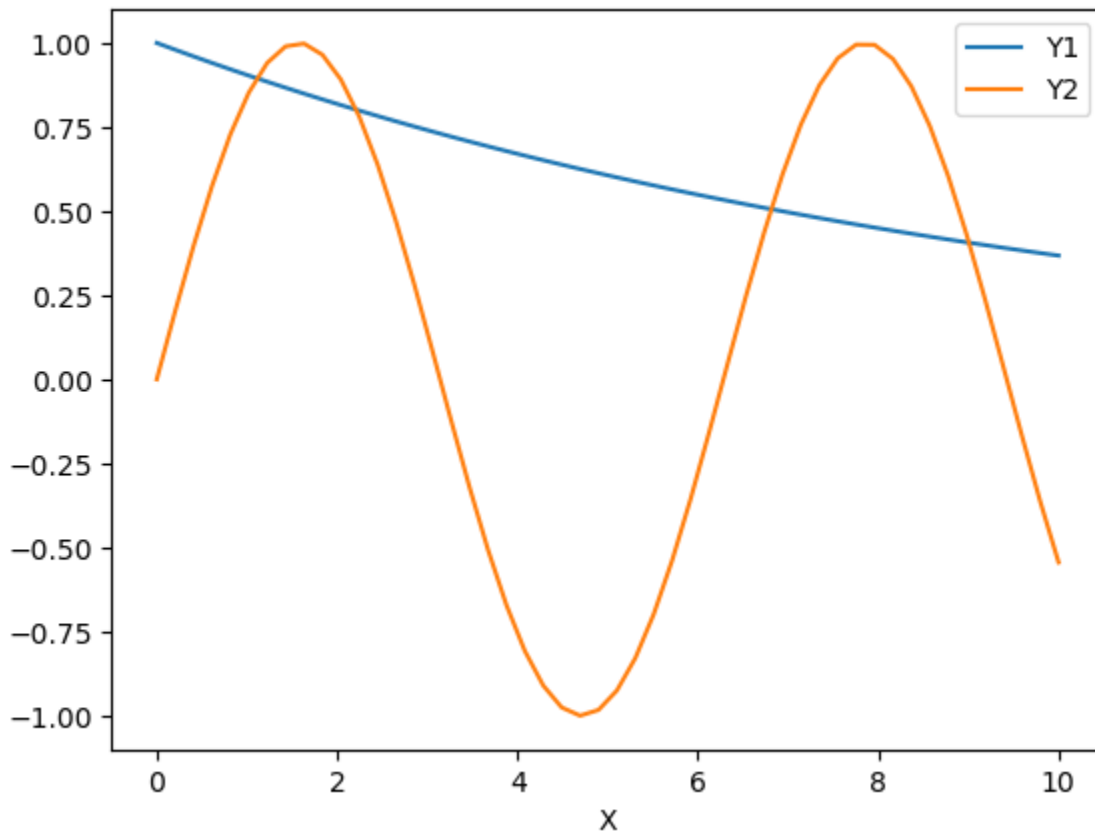
```
plt.legend()
```

```
plt.title('Plotting a Pandas Series')
```

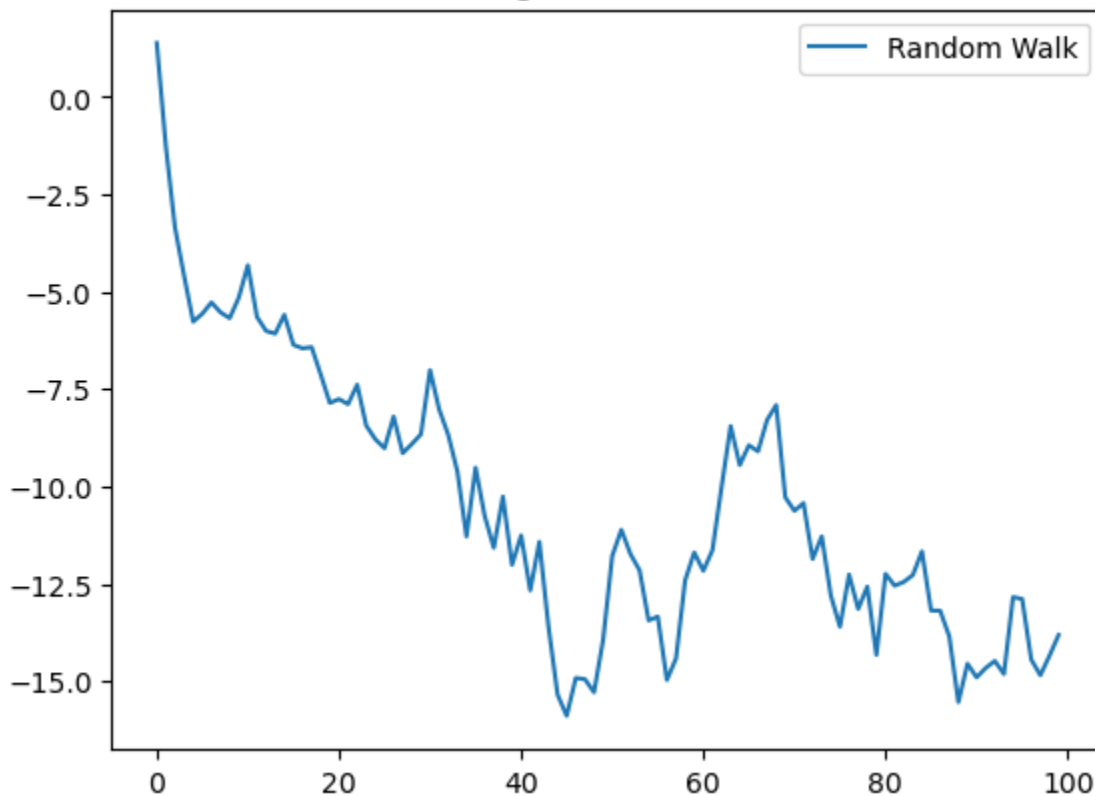
```
plt.show()
```



Plotting with DataFrame .plot()



Plotting a Pandas Series



Customizing Plots Based on DataFrame Columns:

- When plotting from Pandas DataFrames, you can leverage column data for visual customization directly within Matplotlib functions. For instance, you can map column values to colors, sizes, or other

aesthetic properties of your plot elements.

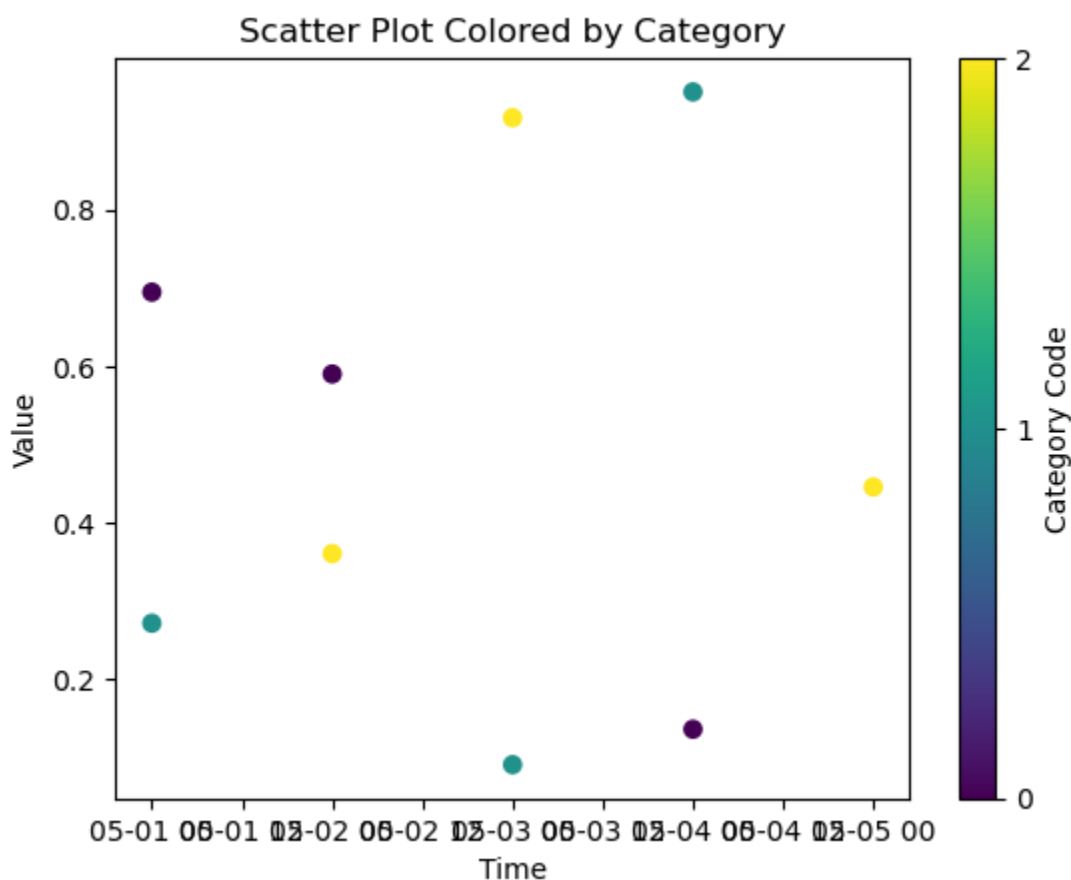
In [65]:

```
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

# Sample DataFrame with categories
data = {'Category': ['A', 'B', 'A', 'C', 'B', 'C', 'A', 'B', 'C'],
        'Value': np.random.rand(9),
        'Time': pd.to_datetime(['2025-05-01', '2025-05-01', '2025-05-02', '2025-05-02',
                                '2025-05-03', '2025-05-03', '2025-05-04', '2025-05-04',
                                '2025-05-05'])}

df = pd.DataFrame(data)

# Scatter plot colored by category
scatter = plt.scatter(df['Time'], df['Value'], c=df['Category'].astype('category').cat.codes)
plt.xlabel('Time')
plt.ylabel('Value')
plt.title('Scatter Plot Colored by Category')
plt.colorbar(scatter, ticks=df['Category'].astype('category').cat.codes.unique(),
             label='Category Code')
plt.show()
```



Handling Time Series Data

Plotting Data with Datetime Objects on the Axes:

- Matplotlib can directly handle datetime objects (from Python's `datetime` module or Pandas) for axis values.

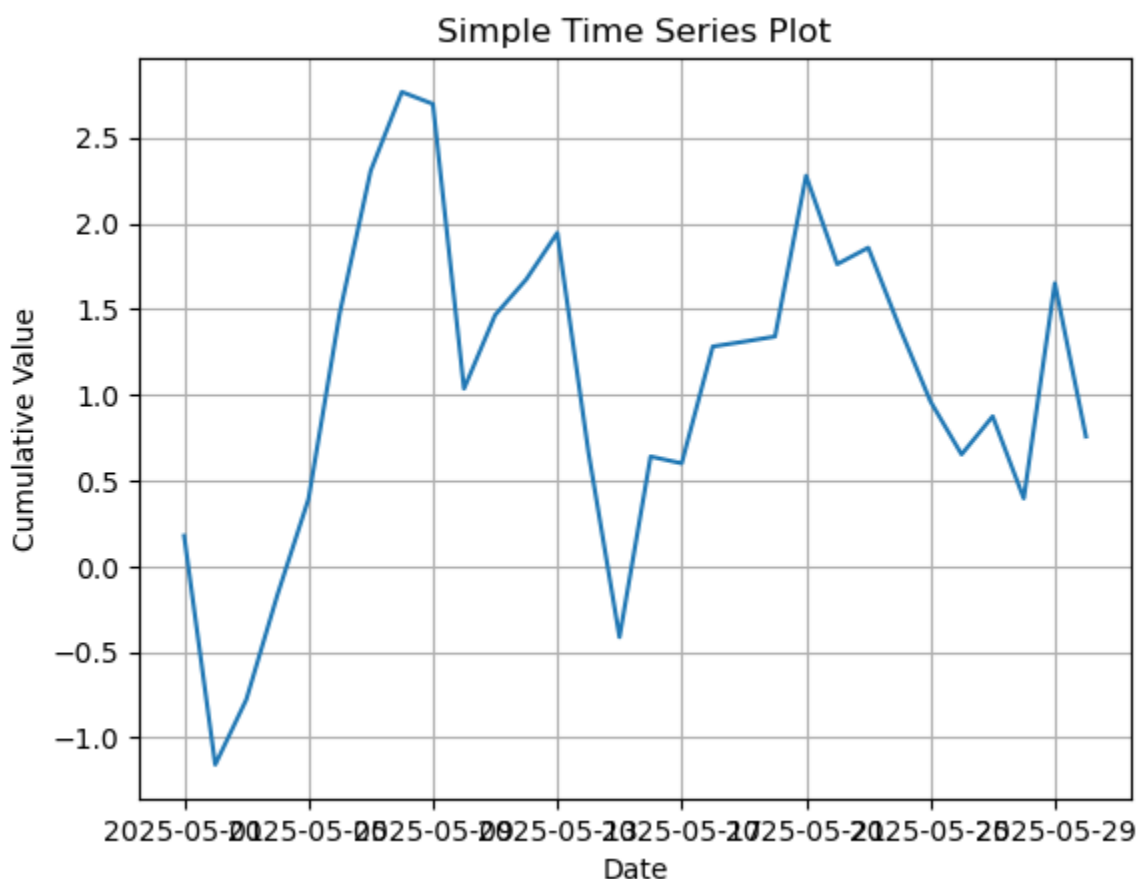
- When plotting against a datetime axis, Matplotlib automatically formats the tick labels for readability.

In [66]:

```
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

# Sample time series data using Pandas
dates = pd.to_datetime(pd.date_range('2025-05-01', periods=30, freq='D'))
values = np.random.randn(30).cumsum()

plt.plot(dates, values)
plt.xlabel('Date')
plt.ylabel('Cumulative Value')
plt.title('Simple Time Series Plot')
plt.grid(True)
plt.show()
```



Formatting Date and Time Tick Labels:

- You can customize the format of date and time tick labels using the `matplotlib.dates` module and the `DateFormatter` class.

In [67]:

```
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
import matplotlib.dates as mdates

# Sample time series data
```

```

dates = pd.to_datetime(pd.date_range('2025-05-01', periods=10, freq='D'))
values = np.random.randn(10).cumsum()

plt.plot(dates, values)
plt.xlabel('Date')
plt.ylabel('Cumulative Value')
plt.title('Time Series with Custom Date Formatting')
plt.grid(True)

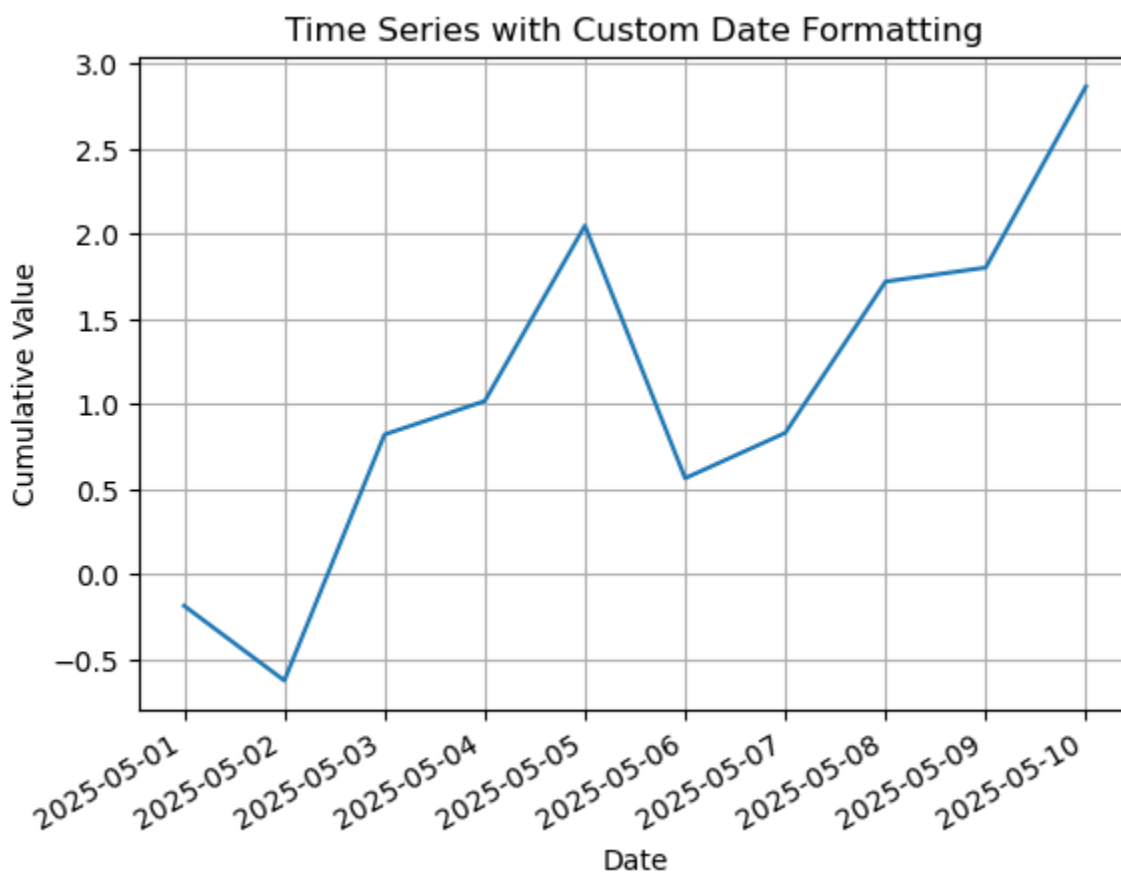
# Define the date format
date_format = mdates.DateFormatter('%Y-%m-%d')

# Apply the format to the x-axis
plt.gca().axis.set_major_formatter(date_format)

# Rotate the x-axis labels for better readability
plt.gcf().autofmt_xdate()

plt.show()

```



Subplots and Layout Management

`plt.subplot()`: Creating Multiple Subplots in a Grid Layout

The `plt.subplot(nrows, ncols, index)` function enables you to create a grid of subplots and select one for plotting.

- **nrows**: Number of rows in the subplot grid.
- **ncols**: Number of columns in the subplot grid.
- **index**: The position of the current subplot within the grid (1-based index, numbered row by row).

For example, `plt.subplot(2, 2, 1)` creates a 2x2 grid of subplots and selects the first subplot (top-left) for subsequent plotting commands.

In [68]:

```
import matplotlib.pyplot as plt
import numpy as np

# Sample data
x = np.linspace(0, 5, 50)

# Create a figure with 2 rows and 2 columns of subplots
plt.figure(figsize=(10, 8))

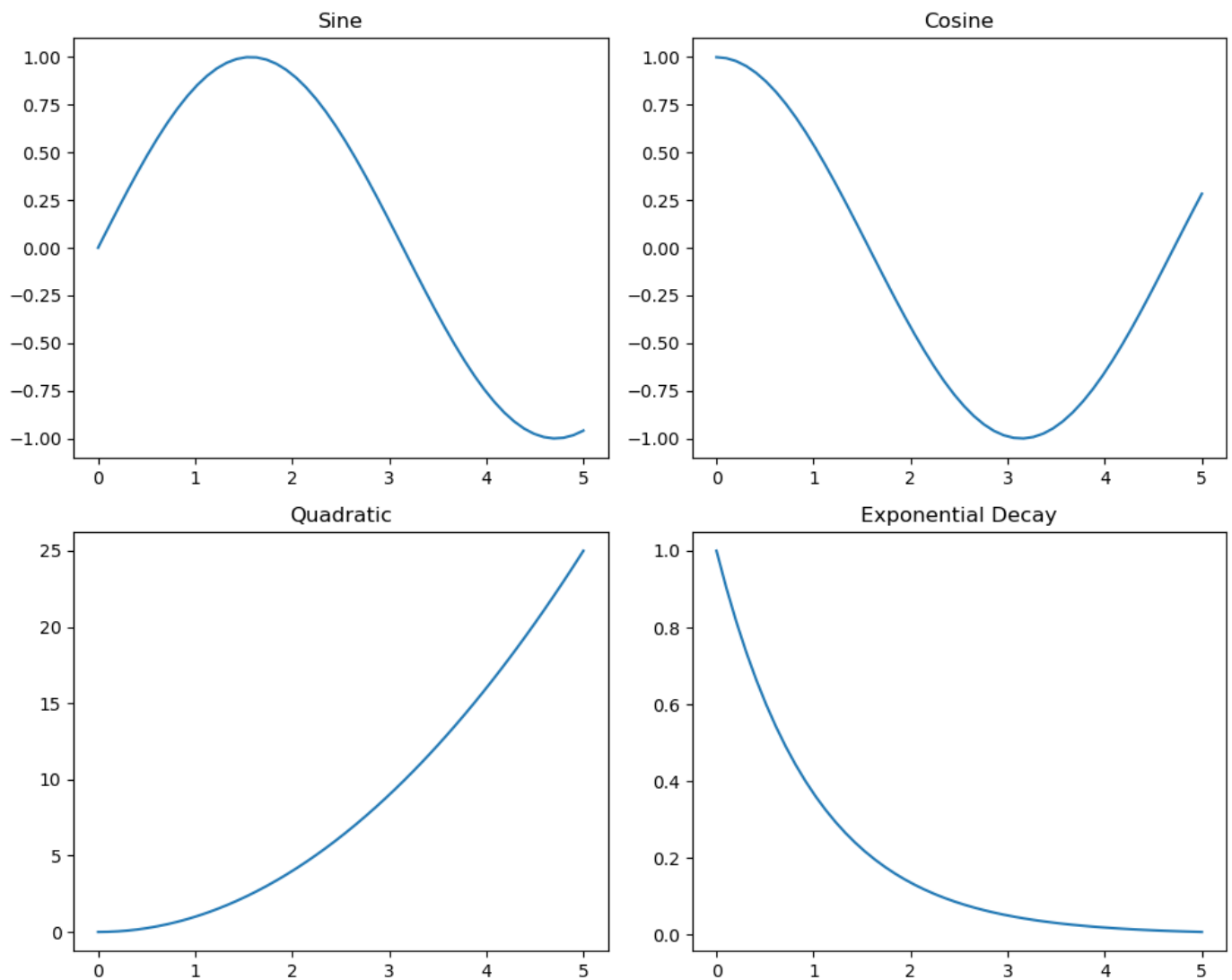
# First subplot (top-left)
plt.subplot(2, 2, 1)
plt.plot(x, np.sin(x))
plt.title('Sine')

# Second subplot (top-right)
plt.subplot(2, 2, 2)
plt.plot(x, np.cos(x))
plt.title('Cosine')

# Third subplot (bottom-left)
plt.subplot(2, 2, 3)
plt.plot(x, x**2)
plt.title('Quadratic')

# Fourth subplot (bottom-right)
plt.subplot(2, 2, 4)
plt.plot(x, np.exp(-x))
plt.title('Exponential Decay')

plt.tight_layout() # Adjust subplot parameters for a tight layout
plt.show()
```



`plt.subplots()` : More Flexible Way to Create Multiple Subplots

The `plt.subplots(nrows=1, ncols=1, sharex=False, sharey=False, squeeze=True, subplot_kw=None, gridspec_kw=None, **fig_kw)` function offers a more streamlined approach to creating figures with multiple subplots.

- It returns the **Figure** object and either:
 - A single **Axes** object (if `nrows=1` and `ncols=1`).
 - A NumPy array of **Axes** objects (for multiple subplots).
- **sharex** and **sharey** : Boolean flags to share the x or y axis limits and ticks among subplots.
- **squeeze** : A boolean. If `True` (default):
 - If only one subplot is created, the result is a single Axes object (not an array).
 - If only one row or one column of subplots is created, the returned Axes object is a 1D NumPy array.
 - If multiple rows and columns are created, the returned object is a 2D NumPy array.

Setting `squeeze=False` always returns a 2D array of Axes, even for a single subplot.

In [69]:


```

import matplotlib.pyplot as plt
import numpy as np

# Create a figure with 2 rows and 2 columns
fig, axes = plt.subplots(2, 2, figsize=(10, 8))
x = np.linspace(0, 5, 50)

# Access individual axes using the axes array
axes[0, 0].plot(x, np.sin(x))
axes[0, 0].set_title('Sine')

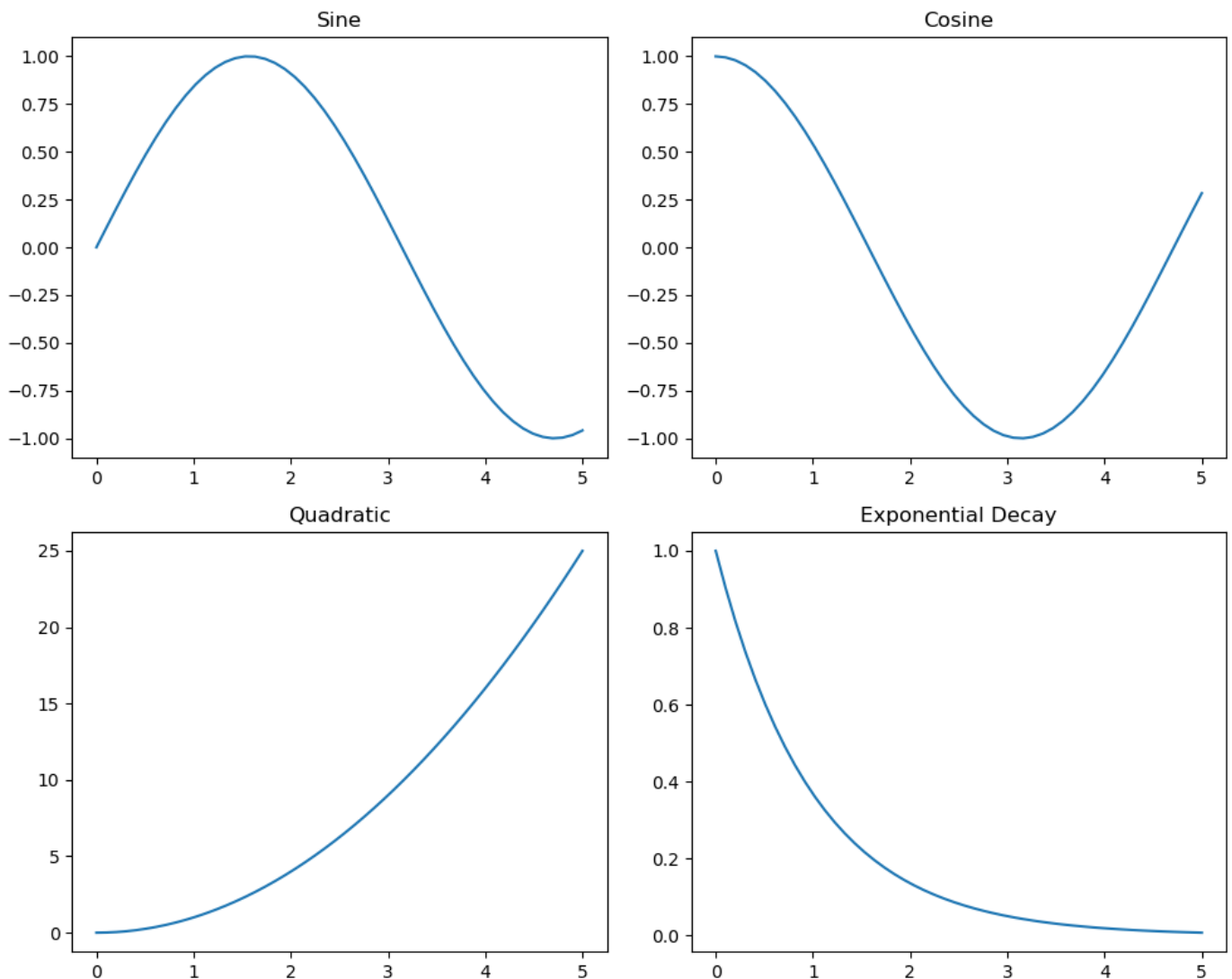
axes[0, 1].plot(x, np.cos(x))
axes[0, 1].set_title('Cosine')

axes[1, 0].plot(x, x**2)
axes[1, 0].set_title('Quadratic')

axes[1, 1].plot(x, np.exp(-x))
axes[1, 1].set_title('Exponential Decay')

plt.tight_layout()
plt.show()

```



Adjusting Subplot Spacing using `plt.tight_layout()`:

- Overlapping subplot titles, labels, or tick labels can clutter plots.
- `plt.tight_layout()` automatically adjusts spacing between subplots to prevent overlaps.
- Typically called after subplot creation.

GridSpec : Creating More Complex and Customized Subplot Layouts:

- `matplotlib.gridspec.GridSpec` allows for intricate subplot arrangements beyond simple grids.
- Define a grid and specify subplot size and position.
- Subplots can span multiple rows or columns.

In [70]:

```
import matplotlib.pyplot as plt
import numpy as np
import matplotlib.gridspec as gridspec

# Create a figure
fig = plt.figure(figsize=(10, 8))

# Define a 2x2 grid
gs = gridspec.GridSpec(2, 2)

# Create subplots in the defined grid
ax1 = fig.add_subplot(gs[0, 0]) # Top-left (row 0, column 0)
ax2 = fig.add_subplot(gs[0, 1]) # Top-right (row 0, column 1)
ax3 = fig.add_subplot(gs[1, :]) # Bottom spanning both columns (row 1, all columns)

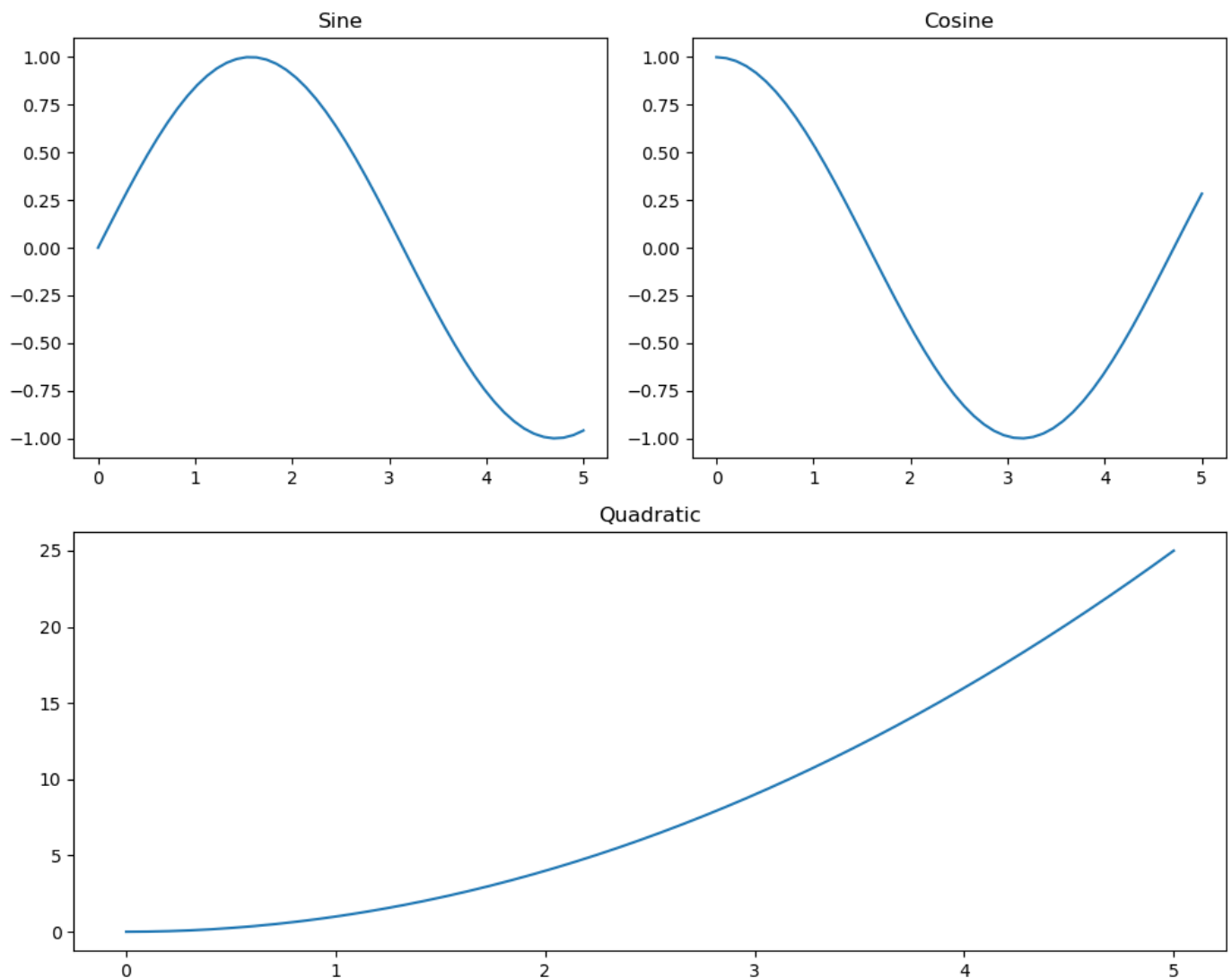
# Sample data and plotting
x = np.linspace(0, 5, 50)

ax1.plot(x, np.sin(x))
ax1.set_title('Sine')

ax2.plot(x, np.cos(x))
ax2.set_title('Cosine')

ax3.plot(x, x**2)
ax3.set_title('Quadratic')

plt.tight_layout()
plt.show()
```



Spanning Rows and Columns with `GridSpec` :

- Use slicing on the `GridSpec` object to create subplots that span multiple rows or columns.
- The syntax `gs[row_start:row_end, col_start:col_end]` defines the subplot's extent within the grid.

`fig.add_axes()` : Adding Axes at Arbitrary Locations within a Figure:

- The `fig.add_axes([left, bottom, width, height])` method offers maximum flexibility in placing Axes.
- `left`, `bottom`, `width`, and `height` are specified as fractions of the Figure's width and height (ranging from 0 to 1).
- This allows you to position and size Axes precisely, even overlapping existing ones if needed.

In [71]:

```
import matplotlib.pyplot as plt
import numpy as np

# Create a figure
fig = plt.figure(figsize=(8, 6))

# Main plot (occupying most of the figure)
```

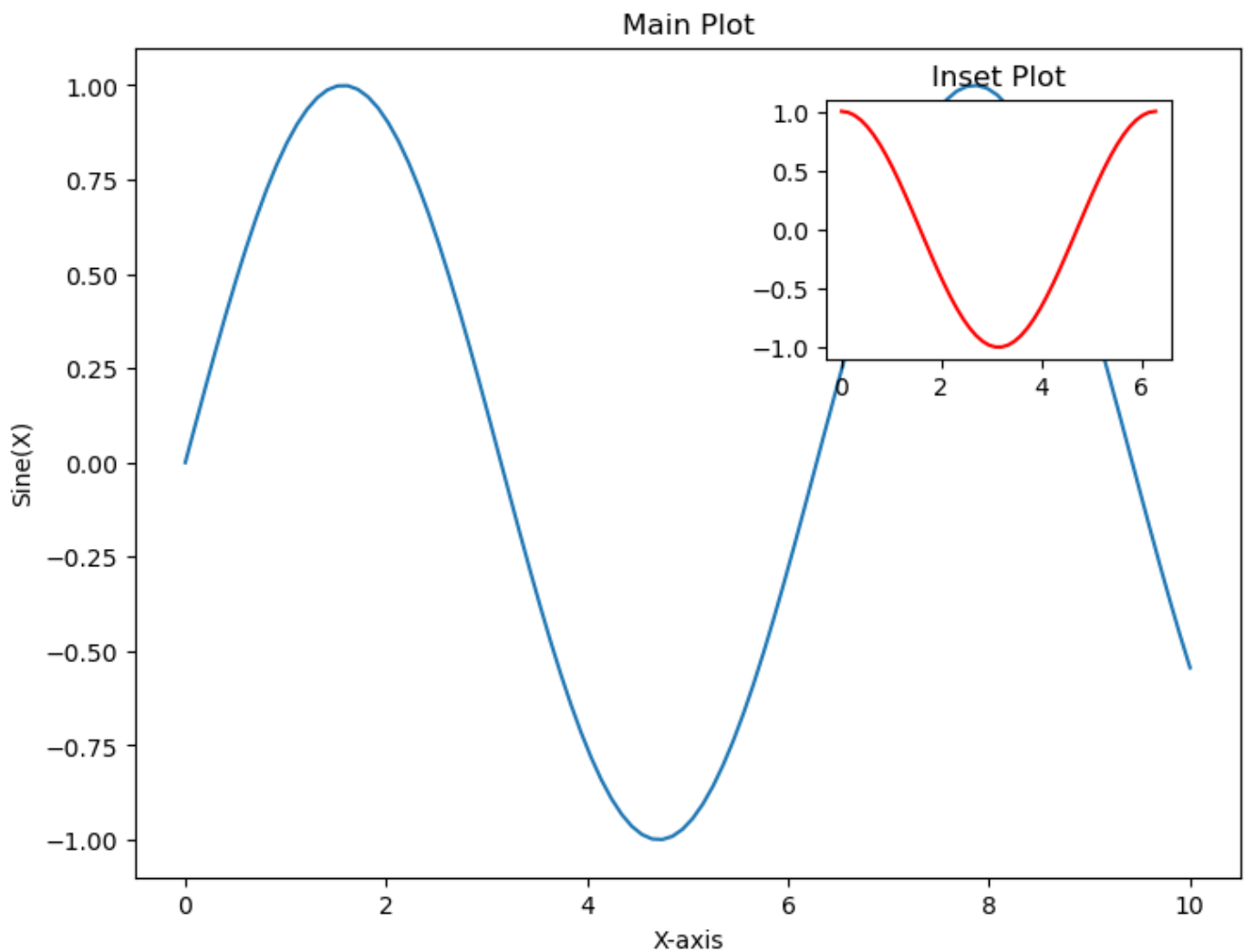
```

ax1 = fig.add_axes([0.1, 0.1, 0.8, 0.8]) # [left, bottom, width, height]
x = np.linspace(0, 10, 100)
y = np.sin(x)
ax1.plot(x, y)
ax1.set_xlabel('X-axis')
ax1.set_ylabel('Sine(X)')
ax1.set_title('Main Plot')

# Inset plot (smaller plot inside the main one)
ax2 = fig.add_axes([0.6, 0.6, 0.25, 0.25]) # [left, bottom, width, height]
x_inset = np.linspace(0, 2 * np.pi, 50)
y_inset = np.cos(x_inset)
ax2.plot(x_inset, y_inset, color='red')
ax2.set_title('Inset Plot')

plt.show()

```



In []:

```


```